

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM**



Mẫu thiết kế - SE401.Q21 Báo cáo cuối kì

GIẢNG VIÊN BỘ MÔN: Trần Anh Dũng

SINH VIÊN THỰC HIỆN:

23521224 Trương Hoàng Phúc

23521140 Lê Minh Phát

22520243 Ya Đạt

TP. HỒ CHÍ MINH, NĂM 2026

MỤC LỤC

Chương 1. Các mẫu tạo lập	1
1.1. Abstract Factory (Phức)	1
1.2. Builder (Đặt)	3
1.3. Factory Method (Đặt)	21
1.4. Prototype (Phát)	37
1.5. Singleton (Phát)	62
Chương 2. Các mẫu cấu trúc	77
2.1. Adapter (Phát)	77
2.2. Bridge (Đặt)	97
2.3. Composite (Phức)	111
2.4. Decorator (Phức)	113
2.5. Facade (Phát)	116
2.6. Flyweight (Phát)	136
2.7. Interpreter (Phức)	158
2.8. Proxy (Đặt)	160
Chương 3. Các mẫu hành vi	174
3.1. Chain of Responsibility (Phát)	174
3.2. Command (Đặt)	198
3.3. Iterator (Đặt)	211
3.4. Mediator (Phức)	221
3.5. Memento (Phát)	223

3.6. Observer (Đạt)	244
3.7. State (Phúc)	252
3.8. Strategy (Đạt)	254
3.9. Template Method (Phát)	265
3.10. Visitor (Phúc)	282

DANH MỤC HÌNH

Hình 1.1	Cấu trúc của mẫu Abstract Factory	1
Hình 1.2	Cấu trúc UML tổng quát của Prototype Pattern	43
Hình 1.3	Cấu trúc Prototype Registry	43
Hình 1.4	Luồng clone object từ prototype	44
Hình 1.5	Luồng chạy code ví dụ clone Enemy prototype	51
Hình 1.6	Class diagram của ví dụ Enemy trong game	53
Hình 1.7	Cấu trúc UML tổng quát của Singleton Pattern	65
Hình 1.8	Class diagram của ví dụ Logger Singleton	65
Hình 1.9	Luồng truy cập instance duy nhất	66
Hình 1.10	Luồng chạy code ví dụ Logger Singleton	71
Hình 2.11	Cấu trúc UML tổng quát của Adapter Pattern	82
Hình 2.12	Luồng hoạt động giữa Client, Adapter và Adaptee	83
Hình 2.13	Luồng chạy code ví dụ CheckoutService và StripePaymentAdapter	89
Hình 2.14	Class diagram của ví dụ StripePaymentAdapter	90
Hình 2.15	Cấu trúc của mẫu Composite	112
Hình 2.16	Cấu trúc của mẫu Decorator	114
Hình 2.17	Cấu trúc UML tổng quát của Facade Pattern	121
Hình 2.18	Class diagram của ví dụ Checkout Facade	121
Hình 2.19	Luồng chạy code ví dụ CheckoutFacade	129
Hình 2.20	Cấu trúc UML tổng quát của Flyweight Pattern	141
Hình 2.21	Class diagram của ví dụ rừng cây	141

Hình 2.22	Luồng tạo và tái sử dụng Flyweight qua Factory	143
Hình 2.23	Luồng chạy code ví dụ Forest, Tree và TreeTypeFactory	149
Hình 2.24	Class diagram của ví dụ marker bản đồ	157
Hình 2.25	Cấu trúc của mẫu Interpreter	158
Hình 3.26	Cấu trúc UML tổng quát của Chain of Responsibility	180
Hình 3.27	Luồng request qua các handler trong chain	181
Hình 3.28	Luồng chạy code ví dụ request pipeline	190
Hình 3.29	Class diagram của pipeline xử lý request	191
Hình 3.30	Cấu trúc của mẫu Mediator	222
Hình 3.31	Cấu trúc UML tổng quát của Memento Pattern	227
Hình 3.32	Class diagram của ví dụ Text Editor	227
Hình 3.33	Luồng backup và undo bằng Memento	229
Hình 3.34	Luồng chạy code ví dụ TextEditor undo	236
Hình 3.35	Cấu trúc của mẫu State	253
Hình 3.36	Cấu trúc UML tổng quát của Template Method	270
Hình 3.37	Class diagram của ví dụ PaymentProcessor	270
Hình 3.38	Luồng chạy template method trong PaymentProcessor	271
Hình 3.39	Luồng chạy code ví dụ PaymentProcessor.Pay	276
Hình 3.40	Cấu trúc của mẫu Visitor	283

Chương 1. Các mẫu tạo lập

1.1. Abstract Factory (Phức)

1.1.1. Tên và Phân loại

Abstract Factory - Mẫu tạo (Creational Pattern)

1.1.2. Mục đích, ý định

Cung cấp giao diện để tạo các họ đối tượng liên quan mà không cần chỉ định rõ các lớp cụ thể của chúng.

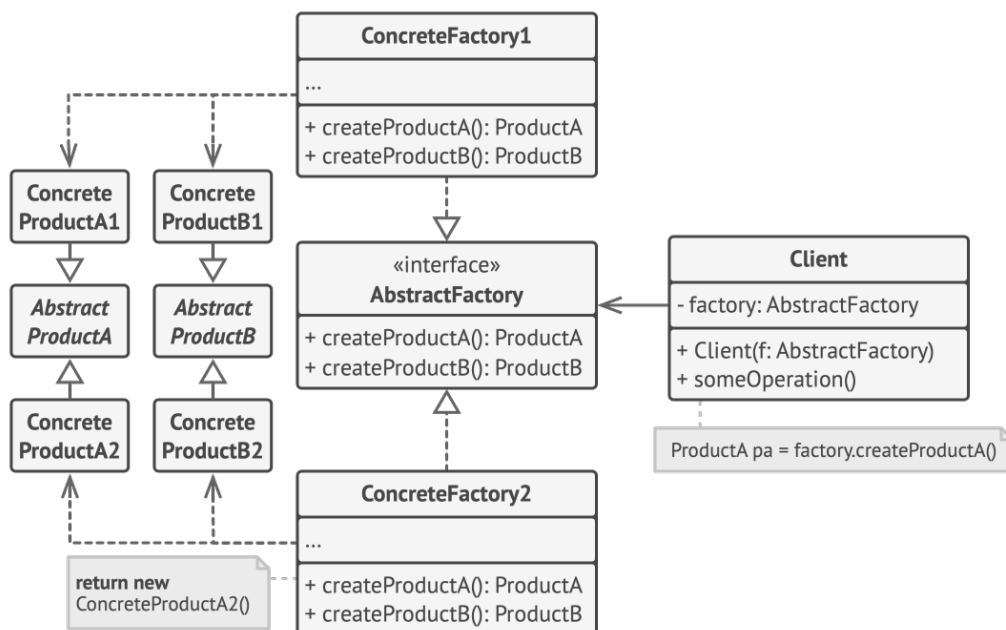
1.1.3. Motivation

Cần tạo các sản phẩm tương thích nhau từ các họ khác nhau. Thay vì khởi tạo trực tiếp từng lớp, sử dụng factory để quản lý việc tạo.

1.1.4. Khả năng ứng dụng

- Hệ thống cần độc lập từ cách các sản phẩm được tạo
- Cần đảm bảo sử dụng các sản phẩm từ cùng một họ
- Cần linh hoạt chuyển đổi giữa các họ sản phẩm khác nhau

1.1.5. Cấu trúc



Hình 1.1: Cấu trúc của mẫu Abstract Factory

AbstractFactory khai báo giao diện tạo sản phẩm. ConcreteFactory triển khai giao diện để tạo các sản phẩm cụ thể.

1.1.6. Các thành viên

- AbstractFactory: giao diện khai báo phương thức tạo sản phẩm
- ConcreteFactory: triển khai tạo sản phẩm cụ thể
- AbstractProduct: giao diện sản phẩm
- ConcreteProduct: sản phẩm cụ thể
- Client: sử dụng factory để tạo sản phẩm

1.1.7. Sự cộng tác

Client gọi phương thức của AbstractFactory. ConcreteFactory tạo ra ConcreteProduct tương ứng.

1.1.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Tách logic tạo từ client; dễ chuyển đổi họ sản phẩm. **Nhược điểm:** Khó thêm sản phẩm mới vào họ; code phức tạp hơn.

1.1.9. Chú ý liên quan đến việc cài đặt

- Xác định rõ cách tạo từng sản phẩm
- Có thể dùng singleton cho factory

1.1.10. Mã nguồn minh họa

Factory pattern thường dùng qua interface IFactory, các implement tạo đối tượng cụ thể.

1.1.11. Ví dụ về các hệ thống thực tế

- UI framework (Swing, Qt) tạo các component khác nhau tùy platform
- Database connection factories cho SQL Server, MySQL, Oracle
- Trò chơi tạo các character từ những họ khác nhau

1.1.12. Các mẫu liên quan

Khác Factory Method (tạo một sản phẩm). Có thể dùng với Prototype, Builder.

1.2. Builder (Đạt)

1.2.1. Tên và Phân loại

Builder Pattern là một mẫu thiết kế thuộc nhóm **Creational Pattern** (nhóm khởi tạo).

Mẫu này tập trung vào việc tách quá trình xây dựng đối tượng phức tạp ra khỏi biểu diễn cuối cùng của đối tượng đó. Thay vì gọi một constructor dài với nhiều tham số hoặc viết logic khởi tạo rải rác, Builder gom quy trình tạo object thành các bước rõ ràng, có thứ tự.

1.2.2. Mục đích, ý định

Mục đích chính của Builder là xây dựng một đối tượng phức tạp từng bước một và cho phép tạo ra nhiều biến thể của đối tượng bằng cùng một quy trình.

Ý định của pattern:

- Tách phần “cách xây” khỏi “sản phẩm được tạo”.
- Dễ kiểm soát thứ tự tạo các phần của object.
- Tránh constructor quá dài hoặc telescoping constructor.
- Tạo nhiều cấu hình sản phẩm khác nhau từ cùng một builder.

1.2.3. Motivation

Giả sử ta có một sản phẩm gồm nhiều thành phần tùy chọn: A, B, C.

Trong nhiều tình huống thực tế, ta cần:

- Bản tối thiểu chỉ có A.
- Bản đầy đủ có A + B + C.
- Bản tùy chỉnh có A + B.

Nếu không dùng Builder, logic tạo object dễ bị phân tán và lặp lại:

- Mỗi nơi tự quyết định add phần nào.
- Dễ quên thứ tự hoặc thiếu bước reset giữa hai lần tạo.
- Khó tái sử dụng quy trình tạo “chuẩn” cho nhiều sản phẩm.

Builder giải quyết bằng cách đóng gói quy trình tạo thành các bước (addA, addB, addC) và dùng Director để định nghĩa recipe (kịch bản build) cố định.

1.2.4. Khả năng ứng dụng

Nên dùng Builder khi:

- Đối tượng có nhiều thuộc tính tùy chọn.
- Cần tạo nhiều biến thể từ cùng một họ sản phẩm.
- Quy trình tạo object gồm nhiều bước và cần kiểm soát thứ tự.
- Muốn tách code khởi tạo phức tạp khỏi business logic.
- Muốn tái sử dụng quy trình build “min”, “max”, “custom”.

Ví dụ phù hợp:

- Tạo cấu hình request/response phức tạp.
- Tạo đối tượng UI (dialog, form, dashboard) theo nhiều preset.
- Tạo document/report có nhiều phần tùy chọn.
- Tạo object domain lớn (Order, Invoice, Profile) với nhiều dữ liệu tùy chọn.

1.2.5. Cấu trúc

Builder Pattern thường gồm các vai trò:

- Product: đối tượng kết quả cần tạo.
- Builder: interface định nghĩa các bước tạo.
- ConcreteBuilder: cài đặt các bước cụ thể và giữ trạng thái sản phẩm đang tạo.
- Director: định nghĩa thứ tự các bước build cho từng biến thể.
- Client: chọn builder, gọi director hoặc tự build tay.

Sơ đồ lớp trong project có thể mô tả như sau:

classDiagram

```
class Product1{
```

```

+parts[] : String
+report()
}

```

```

class Builder{

  <<interface>>

  +reset()

  +addA()

  +addB()

  +addC()

}

```

```

Builder <|-- ConcreteBuilder1
Product1 <-- ConcreteBuilder1

class ConcreteBuilder1{

  -product : Product1

  +ConcreteBuilder1()

  +reset()

  +addA()

  +addB()

  +addC()

  +getProd()

}

```

```
Builder <-- Director
```

```
class Director {  
  
    -builder : Builder  
  
    +Director(builder)  
  
    +build_min()  
  
    +build_max()  
  
}
```

1.2.6. Các thành viên

Product1

- Đại diện sản phẩm cuối cùng.
- Chứa parts để lưu các thành phần đã được lắp.
- Hàm report() dùng để hiển thị cấu trúc sản phẩm.

Builder

- Interface mô tả bộ bước tạo sản phẩm: reset(), addA(), addB(), addC().
- Giúp Director làm việc qua abstraction thay vì phụ thuộc class cụ thể.

ConcreteBuilder1

- Cài đặt chi tiết các bước xây dựng.
- Nắm giữ sản phẩm hiện tại qua biến product.
- getProd() trả sản phẩm hiện tại và reset để sẵn sàng build sản phẩm mới.

Director

- Định nghĩa các kịch bản build tiêu chuẩn.
- build_min() tạo bản tối thiểu.

- `build_max()` tạo bản đầy đủ.

Client

- Tạo builder/director.
- Chọn kịch bản build hoặc tự gọi từng bước để build custom.

1.2.7. Sự cộng tác

Luồng cộng tác tiêu biểu:

1. Client tạo ConcreteBuilder1.
2. Client truyền builder vào Director.
3. Client gọi một recipe như `build_min()` hoặc `build_max()`.
4. Director gọi tuần tự các bước trên builder.
5. Client gọi `getProd()` để nhận sản phẩm hoàn chỉnh.
6. Builder tự `reset()` để có thể build vòng tiếp theo.

Với nhu cầu đặc biệt, client có thể bỏ qua director và gọi trực tiếp:

`addA -> addB -> getProd`

1.2.8. Các hệ quả mang lại, Ưu nhược điểm

1.2.9. Ưu điểm

- Tách rõ quy trình tạo và biểu diễn sản phẩm.
- Giảm constructor quá dài và khó đọc.
- Dễ tạo nhiều cấu hình sản phẩm từ cùng mã build.
- Dễ mở rộng thêm builder mới cho loại sản phẩm mới.
- Dễ test từng bước build hoặc từng recipe.

1.2.10. Nhược điểm

- Tăng số lượng class (Builder, ConcreteBuilder, Director).
- Có thể hơi “nặng” với bài toán rất đơn giản.

- Cần quản lý trạng thái builder cẩn thận (đặc biệt chuyện reset).

1.2.11. Chú ý liên quan đến việc cài đặt

- Nên quy định rõ khi nào builder reset (tại `getProd()` hay do client gọi).
- Tránh để client đọc trạng thái trung gian không hợp lệ.
- Nếu sản phẩm immutable, có thể tạo object mới ở cuối quá trình build.
- Có thể kết hợp fluent API để code ngắn gọn hơn:

```
builder.addA().addB().addC().build()
```

- Với môi trường multi-thread, không nên dùng chung một builder mutable cho nhiều luồng.

1.2.12. Một số so sánh nếu có

1.2.13. Builder và Abstract Factory

- Builder: tập trung **quy trình từng bước** để tạo một object phức tạp.
- Abstract Factory: tập trung **họ object liên quan** (family) và tạo chúng nhanh qua factory method.

1.2.14. Builder và Factory Method

- Factory Method trả về object thường trong một bước.
- Builder phù hợp khi object cần nhiều bước cấu hình trước khi hoàn chỉnh.

1.2.15. Builder và Prototype

- Prototype tạo object mới bằng cách clone object mẫu.
- Builder tạo object mới bằng quy trình lắp ghép từng phần.

1.2.16. Mã nguồn minh họa

Đoạn code sau bám theo demo trong project Builder:

```
function print(str) { console.log(str) }
```

```
class Product1 { parts = []; report() { print(this.parts) } }
```

```
class Builder { reset() { }; addA() { }; addB() { }; addC() { } }
```

```
class ConcreteBuilder1 extends Builder {  
    product; constructor() { super(); this.reset() };  
  
    reset() { this.product = new Product1() }  
    addA() { this.product.parts.push('PartA1') }  
    addB() { this.product.parts.push('PartB1') }  
    addC() { this.product.parts.push('PartC1') }  
  
    getProd() { const result = this.product; this.reset(); return result }  
}
```

```
class Director {  
    builder; constructor(builder) { this.builder = builder }  
  
    build_min() { this.builder.addA() }  
    build_max() { this.builder.addA(); this.builder.addB();  
this.builder.addC() }  
}
```

```
const bld = new ConcreteBuilder1();
```

```

const drc = new Director(bld);

print('Standard basic product:'); drc.build_min()

bld.getProd().report()

print('Standard full featured product:'); drc.build_max()

bld.getProd().report()

print('Custom product:'); bld.addA(); bld.addB()

bld.getProd().report()

```

Kết quả thể hiện rõ ba kiểu tạo:

- Basic product.
- Full featured product.
- Custom product.

1.2.17. Ví dụ về các hệ thống thực tế

- SQL/HTTP query builder trong backend framework.
- UI builder (tạo cây component theo cấu hình).
- Document/report builder (PDF, HTML, DOCX).
- Test data builder (tạo object mẫu cho unit test).
- Cloud infrastructure builder (tạo cấu hình tài nguyên theo từng bước).

1.2.18. Các mẫu liên quan

- **Abstract Factory**: có thể phối hợp để tạo builder tương ứng từng họ sản phẩm.
- **Factory Method**: thường dùng bên trong Builder để tạo từng thành phần con.

- **Prototype**: có thể clone sản phẩm nên rồi tiếp tục build thêm bước mới.
- **Composite**: sản phẩm do Builder tạo ra thường có cấu trúc cây (tree object).

Tóm lại, Builder là lựa chọn tốt khi đối tượng có nhiều bước tạo và nhiều biến thể cấu hình, giúp code rõ ràng, mở rộng tốt và giảm lỗi khi khởi tạo đối tượng phức tạp.

1.2.19. Phân tích sâu quy trình xây dựng

Trong Builder Pattern, điểm quan trọng không chỉ là có các hàm addA, addB, addC, mà còn là quản lý **vòng đời của builder**.

Một vòng đời chuẩn thường gồm ba pha:

1. **Khởi tạo trạng thái build** (reset).
2. **Lắp ráp từng phần** (addX).
3. **Hoàn tất và trả sản phẩm** (getProd/build).

Nếu bỏ qua pha reset hoặc không định nghĩa rõ thời điểm reset, sản phẩm sau có thể bị “dính” dữ liệu từ sản phẩm trước.

1.2.20. Trạng thái nội bộ của Builder

Builder thường chứa state mutable, ví dụ:

- Danh sách parts đã thêm.
- Các flag kiểm soát bước bắt buộc/không bắt buộc.
- Bộ validate tạm thời trước khi xuất sản phẩm.

Trong ví dụ hiện tại, ConcreteBuilder1 lưu state qua biến product. Cách này đơn giản, dễ hiểu, nhưng cần chú ý:

- Không tái sử dụng cùng builder trong nhiều luồng cùng lúc.
- Không để client giữ reference vào product trung gian.
- Nên có quy ước reset nhất quán sau khi trả sản phẩm.

1.2.21. Vai trò của Director trong hệ thống lớn

Ở ví dụ nhỏ, Director có vẻ dư thừa vì client tự gọi builder được. Tuy nhiên trong hệ thống lớn, Director giúp:

- Đóng gói recipe chuẩn của domain.
- Giảm duplicated code trong nhiều service.
- Đảm bảo mọi sản phẩm “chuẩn” luôn qua cùng quy trình.

Ví dụ domain e-commerce:

- `build_default_checkout()`
- `build_guest_checkout()`
- `build_express_checkout()`

Mỗi recipe tương ứng một quy tắc nghiệp vụ rõ ràng.

1.2.22. Biến thể triển khai nâng cao

Builder không chỉ có một kiểu cổ điển. Có nhiều biến thể phù hợp từng loại bài toán.

1.2.23. Fluent Builder

Đặc trưng:

- Mỗi bước trả về `this`.
- Dễ đọc khi chain method.

Ví dụ:

```
const product = builder
    .addA()
    .addB()
    .addC()
    .build();
```

Ưu điểm: code ngắn, trực quan.

Lưu ý: vẫn phải xử lý validate và reset rõ ràng trong build().

1.2.24. Immutable Builder

Đặc trưng:

- Mỗi lần gọi addX() trả về builder mới thay vì sửa builder cũ.
- Tránh side-effect.

Phù hợp với môi trường cần an toàn cao hoặc code theo phong cách functional.

Đánh đổi: tốn thêm bộ nhớ do tạo nhiều object trung gian.

1.2.25. Step Builder

Đặc trưng:

- Ép thứ tự build ở mức type/interface.
- Không cho phép gọi build() nếu thiếu bước bắt buộc.

Ví dụ ý tưởng:

StartStep -> RequiredStepA -> RequiredStepB -> OptionalStep -> BuildStep

Phù hợp khi domain có nhiều ràng buộc cứng, ví dụ hồ sơ pháp lý, giao dịch tài chính.

1.2.26. Builder không có Director

Trong nhiều codebase hiện đại, Director bị lược bỏ để giảm lớp trung gian.

Client hoặc service layer sẽ đóng vai trò điều phối recipe. Cách này ổn khi:

- Số recipe ít.
- Quy trình tạo đơn giản.
- Team ưu tiên ít abstraction.

Ngược lại, nếu recipe tăng mạnh, Director hoặc orchestration class sẽ hữu ích hơn.

1.2.27. Pseudocode tổng quát

```
interface Builder {
```

```
    reset()
```

```
    buildPartA()

    buildPartB()

    buildPartC()

    getResult()
}
```

```
class ConcreteBuilder implements Builder {

    product

    reset()          -> create new Product

    buildPartA()     -> add A

    buildPartB()     -> add B

    buildPartC()     -> add C

    getResult()      -> return product and reset
}
```

```
class Director {

    builder

    makeMinimal()    -> A

    makeFull()       -> A, B, C
}
```

client:

```
b = ConcreteBuilder()
```

```
d = Director(b)
```

```
d.makeFull()
```

```
p = b.getResult()
```

1.2.28. Sequence minh họa chi tiết

sequenceDiagram

```
actor Client
```

```
participant Director
```

```
participant ConcreteBuilder
```

```
participant Product
```

```
Client->>Director: build_max()
```

```
Director->>ConcreteBuilder: addA()
```

```
Director->>ConcreteBuilder: addB()
```

```
Director->>ConcreteBuilder: addC()
```

```
Client->>ConcreteBuilder: getProd()
```

```
ConcreteBuilder-->>Client: Product
```

```
ConcreteBuilder->>ConcreteBuilder: reset()
```

Điểm mấu chốt trong sequence trên là builder tự reset sau khi trả product, đảm bảo build tiếp theo không lẫn state cũ.

1.2.29. Chiến lược kiểm thử cho Builder

Với Builder, testing nên chia theo ba lớp:

1.2.30. Unit test cho ConcreteBuilder

Mục tiêu:

- Mỗi hàm addX() thêm đúng part.
- getProd() trả đúng kết quả.
- reset() thực sự tạo trạng thái mới.

Ví dụ kịch bản:

- addA() rồi getProd() -> chỉ có PartA1.
- addA(), addB(), addC() -> đúng thứ tự parts.
- getProd() hai lần liên tiếp -> lần hai là sản phẩm rỗng (nếu chưa add gì).

1.2.31. Unit test cho Director

Có thể dùng mock builder để kiểm tra thứ tự gọi hàm:

- build_min() chỉ gọi addA().
- build_max() gọi addA(), addB(), addC() đúng thứ tự.

1.2.32. Integration test

Kết hợp Director + ConcreteBuilder + Product để đảm bảo recipe cho ra output đúng nghiệp vụ.

1.2.33. Lỗi thường gặp và cách tránh

1.2.34. Quên reset builder

Triệu chứng:

- Sản phẩm mới chứa dữ liệu của sản phẩm trước.

Cách tránh:

- Reset ngay trong getProd() hoặc bắt buộc client gọi reset() theo contract rõ ràng.

1.2.35. Builder bị lộ trạng thái nội bộ

Triệu chứng:

- Client chỉnh sửa trực tiếp product.parts trong lúc builder đang build.

Cách tránh:

- Giữ encapsulation, không expose state mutable giữa quá trình build.

1.2.36. Dùng chung builder cho nhiều thread

Triệu chứng:

- Kết quả ngẫu nhiên, part thiếu/thừa do race condition.

Cách tránh:

- Mỗi request một builder instance.
- Hoặc dùng immutable builder nếu cần share logic.

1.2.37. Lạm dụng Builder cho object quá đơn giản

Triệu chứng:

- Tăng số lớp không cần thiết.

Cách tránh:

- Nếu object có 2-3 field đơn giản, constructor/factory có thể đủ tốt.

1.2.38. Hiệu năng và đánh đổi thiết kế

Builder thêm một số overhead:

- Thêm call method cho từng bước.
- Thêm object trung gian (builder/director).

Tuy nhiên trong đa số ứng dụng business, overhead này rất nhỏ so với lợi ích về:

- Khả năng bảo trì.
- Tính rõ ràng.
- Giảm lỗi khởi tạo.

Nếu bài toán nhạy hiệu năng, có thể benchmark giữa:

- Constructor/factory trực tiếp.
- Builder cổ điển.

- Fluent/immutable builder.

1.2.39. Ví dụ mở rộng: HTTP Request Builder

Builder rất phù hợp cho request có nhiều phần tùy chọn như method, headers, query, body, timeout.

```
class HttpRequest {  
    constructor({ method, url, headers, query, body, timeout }) {  
        this.method = method;  
        this.url = url;  
        this.headers = headers;  
        this.query = query;  
        this.body = body;  
        this.timeout = timeout;  
    }  
}
```

```
class HttpRequestBuilder {  
    constructor() { this.reset(); }  
  
    reset() {  
        this.method = "GET";  
        this.url = "";  
        this.headers = {};  
        this.query = {};  
    }  
}
```

```
    this.body = null;

    this.timeout = 5000;

    return this;
}
```

```
setMethod(m) { this.method = m; return this; }

setUrl(u) { this.url = u; return this; }

addHeader(k, v) { this.headers[k] = v; return this; }

addQuery(k, v) { this.query[k] = v; return this; }

setBody(b) { this.body = b; return this; }

setTimeout(ms) { this.timeout = ms; return this; }

build() {

    if (!this.url) throw new Error("URL is required");

    const req = new HttpRequest({

        method: this.method,

        url: this.url,

        headers: this.headers,

        query: this.query,

        body: this.body,

        timeout: this.timeout,

    });

    this.reset();
}
```



```
        return req;
    }
}
```

Ưu điểm của ví dụ trên là có thể thêm option mới mà không phá vỡ client code cũ.

1.2.40. Checklist áp dụng nhanh

Trước khi chọn Builder, có thể tự hỏi:

- Object có nhiều thuộc tính tùy chọn không?
- Có nhiều biến thể sản phẩm từ cùng quy trình không?
- Constructor hiện tại có khó đọc hoặc dễ nhầm thứ tự tham số không?
- Có cần đóng gói recipe chuẩn cho nhiều use case không?
- Team có chấp nhận thêm abstraction để đổi lấy maintainability không?

Nếu phần lớn câu trả lời là “có”, Builder thường là lựa chọn hợp lý.

1.2.41. Câu hỏi thường gặp

1.2.42. Builder có bắt buộc phải có Director không?

Không bắt buộc. Director hữu ích khi có nhiều recipe chuẩn hoặc muốn tách hẳn orchestration khỏi client.

1.2.43. Khi nào nên reset trong build()/getProd()?

Nên reset ngay sau khi trả kết quả nếu builder được tái sử dụng nhiều lần trong cùng vòng đời object.

1.2.44. Có thể dùng Builder với object immutable không?

Có. Đây là trường hợp phổ biến. Builder giữ state mutable tạm thời, sau đó tạo ra immutable product tại build().

1.2.45. Builder có thay thế hoàn toàn Factory Method không?

Không. Hai pattern phục vụ nhu cầu khác nhau. Builder mạnh ở khởi tạo nhiều bước; Factory Method mạnh ở việc chọn loại object để tạo.

1.2.46. Kết luận mở rộng

Builder không chỉ là “một cách viết constructor đẹp hơn”. Điểm giá trị thực sự nằm ở kiến trúc:

- Chuẩn hóa quy trình khởi tạo phức tạp.
- Giảm coupling giữa quá trình build và logic sử dụng.
- Tăng khả năng mở rộng khi domain phát triển.

Trong project hiện tại, demo Product1 + ConcreteBuilder1 + Director đã thể hiện đúng tinh thần cốt lõi của Builder. Khi mở rộng hệ thống, bạn có thể thêm:

- Nhiều ConcreteBuilder cho nhiều loại sản phẩm.
- Director recipes theo từng ngữ cảnh nghiệp vụ.
- Fluent API và validate rule để tăng an toàn khi build.

Nhờ vậy, code khởi tạo sẽ rõ ràng, nhất quán và bền vững hơn theo thời gian.

1.3. Factory Method (Đạt)

1.3.1. Tên và phân loại

Factory Method Pattern trong tiếng Việt thường được gọi là **mẫu Phương thức Factory**.

Factory Method thuộc nhóm **Creational Pattern** vì nó tập trung vào cơ chế khởi tạo đối tượng.

Thay vì client gọi trực tiếp `new ConcreteProduct()`, toàn bộ logic tạo đối tượng được đẩy vào một phương thức ảo (factory method) để các lớp con quyết định kiểu cụ thể nào sẽ được tạo ra.

Tên gọi “Factory Method” thể hiện đúng bản chất: **một phương thức đóng vai trò nhà máy sản xuất đối tượng**, và nhà máy cụ thể là do lớp con định nghĩa.

1.3.2. Mục đích và ý định

Mục đích của Factory Method là **định nghĩa một interface để tạo đối tượng, nhưng để lại cho lớp con quyết định lớp nào sẽ được khởi tạo**.

Factory Method cho phép một lớp trì hoãn việc khởi tạo sang lớp con (defer instantiation to subclasses). Điều này có nghĩa:

- Lớp cha xác định **khi nào** và **cách sử dụng** sản phẩm thông qua interface chung.
- Lớp con xác định **sản phẩm cụ thể** nào được tạo ra bằng cách override factory method.
- Client làm việc hoàn toàn qua abstraction, không phụ thuộc vào class cụ thể.

1.3.3. Vấn đề cần giải quyết (Motivation)

Xét một ứng dụng quản lý phương tiện giao thông (Vehicle). Ban đầu hệ thống chỉ hỗ trợ tàu thuyền (Boat). Theo thời gian cần bổ sung xe đạp (Bicycle), sau đó xe máy, ô tô, v.v.

Nếu client tạo đối tượng trực tiếp:

```
IVehicle vehicle = new Boat();  
  
vehicle.StartEngine();
```

Mỗi lần thêm phương tiện mới, client phải sửa code — vi phạm nguyên lý **Open/Closed Principle**. Đồng thời client bị coupling chặt vào class cụ thể Boat, gây khó khăn khi test và mở rộng.

Giải pháp: giới thiệu lớp VehicleCreator trừu tượng với factory method CreateVehicle().

Mỗi loại creator (BoatCreator, BicycleCreator) tự quyết định sản phẩm nào được tạo. Client chỉ làm việc với VehicleCreator và IVehicle — không biết class cụ thể nào đứng sau.

1.3.4. Khả năng ứng dụng

Factory Method phù hợp trong các trường hợp:

- **Không biết trước loại đối tượng cần tạo:** khi code không thể xác định tại compile-time sẽ dùng class cụ thể nào, mà chỉ biết ở runtime.
- **Muốn lớp con kiểm soát việc tạo đối tượng:** framework/thư viện định nghĩa cấu trúc chung, ứng dụng cụ thể quyết định class nào được dùng.

- **Tái sử dụng object hiện có thay vì tạo mới:** factory method có thể trả về object từ cache/pool thay vì new, mà client không cần biết.
- **Cần tách biệt logic khởi tạo khỏi logic sử dụng:** giúp code dễ test (mock creator), dễ mở rộng (thêm creator mới không sửa code cũ).

1.3.5. Cấu trúc

Cấu trúc tổng quát của Factory Method:

@startuml

```
abstract class Creator {
    +{abstract} FactoryMethod() : Product
    +SomeOperation()
}
```

```
interface Product {
    +DoSomething()
}
```

```
class ConcreteCreatorA {
    +FactoryMethod() : Product
}
```

```
class ConcreteCreatorB {
    +FactoryMethod() : Product
}
```

```
class ConcreteProductA {  
    +DoSomething()  
}
```

```
class ConcreteProductB {  
    +DoSomething()  
}
```

```
Creator <|-- ConcreteCreatorA  
Creator <|-- ConcreteCreatorB  
Product <|-- ConcreteProductA  
Product <|-- ConcreteProductB  
  
ConcreteCreatorA ..> ConcreteProductA : <<creates>>  
ConcreteCreatorB ..> ConcreteProductB : <<creates>>  
  
Creator --> Product  
  
@enduml
```

Cấu trúc cụ thể trong project này:

```
@startuml  
  
abstract class VehicleCreator {  
    +{abstract} CreateVehicle() : IVehicle  
}
```

```

interface IVehicle {
    +StartEngine() : string
}

class BoatCreator {
    +CreateVehicle() : IVehicle
}

class BicycleCreator {
    +CreateVehicle() : IVehicle
}

class Boat {
    +StartEngine() : string
}

class Bicycle {
    +StartEngine() : string
}

VehicleCreator <|-- BoatCreator
VehicleCreator <|-- BicycleCreator
IVehicle <|.. Boat

```

```

IVehicle <|.. Bicycle

BoatCreator ..> Boat : <<creates>>

BicycleCreator ..> Bicycle : <<creates>>

VehicleCreator --> IVehicle

@enduml

```

1.3.6. Các thành viên

Factory Method bao gồm bốn thành phần chính:

Product (IVehicle)

- Interface hoặc lớp trừu tượng định nghĩa hành vi chung cho tất cả sản phẩm.
- Creator làm việc với sản phẩm qua interface này, không biết class cụ thể.
- Trong project: IVehicle với phương thức StartEngine().

ConcreteProduct (Boat, Bicycle)

- Các lớp cụ thể triển khai interface Product.
- Mỗi ConcreteProduct định nghĩa hành vi đặc thù của mình.
- Trong project: Boat.StartEngine() trả về "Boat has been started",
Bicycle.StartEngine() trả về "Your bicycle has engine? That's cool".

Creator (VehicleCreator)

- Lớp trừu tượng (hoặc interface) khai báo factory method.
- Thường chứa các phương thức nghiệp vụ sử dụng sản phẩm được tạo ra bởi factory method.
- Creator **không** bắt buộc phải trừu tượng — có thể cung cấp một sản phẩm mặc định.
- Trong project: VehicleCreator với abstract IVehicle CreateVehicle().

ConcreteCreator (BoatCreator, BicycleCreator)

- Các lớp con override factory method để trả về instance cụ thể.
- Mỗi ConcreteCreator chịu trách nhiệm cho một loại sản phẩm.
- Trong project: BoatCreator trả về new Boat(), BicycleCreator trả về new Bicycle().

1.3.7. Sự cộng tác

Luồng hoạt động của Factory Method diễn ra như sau:

@startuml

participant Client

participant VehicleCreator as Creator <<abstract>>

participant BoatCreator as Concrete

participant Boat as Product

Client -> Concrete : new BoatCreator()

Client -> Concrete : CreateVehicle()

Concrete -> Product : new Boat()

Product --> Concrete : boat : IVehicle

Concrete --> Client : boat : IVehicle

Client -> Product : StartEngine()

Product --> Client : "Boat has been started"

@enduml

Quá trình cộng tác:

1. Client chọn một ConcreteCreator phù hợp (thông qua config, switch expression, DI container, ...).

2. Client gọi `CreateVehicle()` trên creator. Creator thực chất là `VehicleCreator`, nhưng phương thức ảo được dispatch tới `BoatCreator`.
3. `BoatCreator.CreateVehicle()` khởi tạo `new Boat()` và trả về dưới dạng `IVehicle`.
4. Client sử dụng sản phẩm qua interface `IVehicle.StartEngine()` — hoàn toàn không biết đó là `Boat` hay `Bicycle`.

1.3.8. Hệ quả, ưu và nhược điểm

Ưu điểm

- **Loại bỏ coupling với ConcreteProduct:** client làm việc qua interface `IVehicle`, không import/tham chiếu trực tiếp `Boat` hay `Bicycle`.
- **Tuân theo Open/Closed Principle:** thêm phương tiện mới (`Car`, `Motorcycle`) chỉ cần thêm class mới — không sửa code cũ.
- **Single Responsibility Principle:** logic tạo sản phẩm tập trung tại creator, tách biệt khỏi logic sử dụng.
- **Dễ test và mock:** trong unit test, có thể tạo `FakeCreator` trả về `MockVehicle` mà không ảnh hưởng code thật.
- **Tái sử dụng logic khởi tạo:** nếu tạo object cần nhiều bước (`config`, `validate`, ...), logic đó nằm gọn trong creator.

Nhược điểm

- **Tăng số lượng class:** mỗi loại sản phẩm mới cần thêm cả class `Product` lẫn class `Creator` — trong dự án nhỏ có thể là over-engineering.
- **Phân cấp class sâu hơn:** khó theo dõi luồng tạo đối tượng khi có nhiều lớp con creator.
- **Client vẫn phải biết Creator:** client cần biết dùng `BoatCreator` hay `BicycleCreator`, chỉ không cần biết `Boat/Bicycle` cụ thể.

1.3.9. Chú ý khi cài đặt

Cách khai báo Creator

- Creator có thể là abstract class (buộc subclass phải implement factory method) hoặc class thường với một implementation mặc định.
- Nếu dùng abstract, đảm bảo mọi subclass đều override factory method — không để sót.

Không nhất thiết phải có Creator tách biệt

- Đôi khi factory method được đặt ngay trong lớp sử dụng sản phẩm, không cần class Creator riêng. Điều này chấp nhận được khi hệ thống nhỏ.

Trả về interface, không trả về class cụ thể

- Factory method nên khai báo kiểu trả về là IVehicle, không phải Boat. Điều này bảo đảm client không thể vô tình coupling vào class cụ thể.

Đặt tên nhất quán

- Convention phổ biến: Create<ProductName>(), Make<ProductName>(), Build<ProductName>(). Trong project này dùng CreateVehicle().

Kết hợp với Dependency Injection

- Trong ứng dụng thực tế, ConcreteCreator thường được inject qua DI container thay vì khởi tạo thủ công bằng switch/if.

// Thay vì:

```
vehicleCreator = creator switch {  
    nameof(BoatCreator) => new BoatCreator(),  
    ...  
};
```

// Nên dùng:

```
services.AddScoped<VehicleCreator, BoatCreator>();
```

```
var vehicleCreator = serviceProvider.GetRequiredService<VehicleCreator>();
```

Phân biệt Factory Method với static factory

- Static factory (`Vehicle.Create(type)`) không có tính đa hình — không thể override ở subclass. Factory Method yêu cầu instance của creator và khai thác virtual dispatch.

1.3.10. So sánh với các mẫu liên quan

Factory Method vs Abstract Factory

Tiêu chí	Factory Method	Abstract Factory
Phạm vi	Tạo một loại sản phẩm	Tạo một họ sản phẩm liên quan
Cơ chế	Kế thừa (subclass override)	Composition (factory object)
Mở rộng	Thêm subclass creator	Thêm concrete factory
Ví dụ	<code>BoatCreator.CreateVehicle()</code>	<code>MietjetFactory.CreateTicket()</code> + <code>CreateBaggagePolicy()</code>

Factory Method vs Builder

- Builder tập trung vào **từng bước** xây dựng object phức tạp; Factory Method tập trung vào **loại** object được tạo.
- Một `ConcreteCreator` trong Factory Method có thể sử dụng Builder bên trong để xây dựng sản phẩm phức tạp.

Factory Method vs Prototype

- Prototype tạo object bằng cách **clone** một đối tượng mẫu có sẵn.
- Factory Method tạo object từ đầu bằng constructor.
- Prototype linh hoạt hơn khi cần nhiều biến thể nhỏ; Factory Method rõ ràng hơn về kiểu tạo ra.

Factory Method vs Template Method

- Cả hai đều dựa vào kế thừa và virtual method.
- Template Method xác định **bộ khung thuật toán** và để subclass override các bước; Factory Method xác định **điểm tạo đối tượng** và để subclass chọn loại.
- Factory Method thực chất là một trường hợp đặc biệt của Template Method áp dụng cho việc khởi tạo đối tượng.

1.3.11. Mã nguồn minh họa

Dưới đây là toàn bộ mã nguồn của project FactoryMethodDemo:

Interface IVehicle

```
namespace FactoryMethodDemo.Interfaces
{
    internal interface IVehicle
    {
        string StartEngine();
    }
}
```

Abstract class VehicleCreator

```
namespace FactoryMethodDemo.Interfaces
{
```

```

abstract class VehicleCreator
{
    // Factory Method – lớp con quyết định trả về`loại IVehicle nào
    public abstract IVehicle CreateVehicle();
}

```

Concrete products: Boat và Bicycle

```

namespace FactoryMethodDemo.Models
{
    internal class Boat : IVehicle
    {
        public string StartEngine() => "Boat has been started";
    }

    internal class Bicycle : IVehicle
    {
        public string StartEngine() => "Your bicycle has engine? That's
cool";
    }
}

```

Concrete creators: BoatCreator và BicycleCreator

```

namespace FactoryMethodDemo.Implements
{

```

```

internal class BoatCreator : VehicleCreator
{
    public override IVehicle CreateVehicle() => new Boat();
}

internal class BicycleCreator : VehicleCreator
{
    public override IVehicle CreateVehicle() => new Bicycle();
}
}

```

Client (Program.cs)

```

using FactoryMethodDemo.Implements;
using FactoryMethodDemo.Interfaces;

string creator = nameof(BoatCreator);

VehicleCreator vehicleCreator;

vehicleCreator = creator switch
{
    nameof(BoatCreator)    => new BoatCreator(),
    nameof(BicycleCreator) => new BicycleCreator(),
    _                      => throw new Exception()
};

```

```

if (vehicleCreator is not null)
{
    var vehicle = vehicleCreator.CreateVehicle();
    Console.WriteLine(vehicle.StartEngine());
}

```

Kết quả chạy (khi creator = nameof(BoatCreator)):

Boat has been started

Để chuyển sang dùng BicycleCreator, chỉ cần đổi giá trị của biến creator — không cần sửa bất kỳ dòng code nào khác.

Ví dụ mở rộng: thêm phương tiện mới Car chỉ cần:

```

// Models/Car.cs

internal class Car : IVehicle
{
    public string StartEngine() => "Car engine roaring!";
}

// Implements/CarCreator.cs

internal class CarCreator : VehicleCreator
{
    public override IVehicle CreateVehicle() => new Car();
}

```

```
// Program.cs – thêm một case trong switch, không sửa gì khác
```

```
nameof(CarCreator) => new CarCreator(),
```

1.3.12. Ví dụ trong các hệ thống thực tế

ASP.NET Core — ControllerFactory

ASP.NET Core dùng `ControllerFactory` để tạo controller instance cho mỗi request. Framework định nghĩa interface, ứng dụng (hoặc DI container) cung cấp concrete factory. Điều này cho phép thay thế toàn bộ cơ chế khởi tạo controller mà không sửa framework.

Entity Framework Core — DbContext Pooling

`IDbContextFactory<TContext>` là một factory method interface. Khi dùng `AddDbContextFactory`, EF Core tạo context thông qua factory — cho phép pooling, scoped lifetime, v.v. thay vì `new ApplicationDbContext()`.

Logging — ILoggerFactory

`ILoggerFactory.CreateLogger(categoryName)` là một factory method: factory quyết định trả về `ConsoleLogger`, `FileLogger`, hay `NullLogger` tùy theo cấu hình, mà caller không cần biết.

Java — java.util.Calendar.getInstance()

`Calendar.getInstance()` là một static factory trả về `GregorianCalendar`, `BuddhistCalendar`, hay `JapaneseImperialCalendar` tùy Locale của hệ thống — người gọi chỉ biết đang nhận `Calendar`.

Game Engine (Unity) — ObjectPool / Instantiate

`Object.Instantiate(prefab)` trong Unity là một dạng factory: engine quyết định tạo object trong scene từ prefab, trả về instance. `AddressableAssets` nâng cấp thêm thành async factory.

1.3.13. Các mẫu liên quan

- **Abstract Factory:** thường được cài đặt bằng nhiều Factory Method. Khi một factory method phát triển lên và cần tạo nhiều sản phẩm liên quan, nó trở thành Abstract Factory.
- **Template Method:** Factory Method là một đặc trường hợp của Template Method — cả hai dùng kế thừa và virtual dispatch để tùy chỉnh hành vi ở subclass.
- **Prototype:** có thể thay thế Factory Method khi sản phẩm cần clone từ prototype thay vì khởi tạo mới. Đặc biệt hữu ích khi số lượng biến thể lớn.
- **Builder:** Factory Method chỉ cần một bước tạo sản phẩm (`CreateVehicle()`). Khi sản phẩm phức tạp hơn và cần nhiều bước, dùng Builder bên trong factory method.
- **Dependency Injection:** DI container là một Factory Method tổng quát — nó map interface sang concrete class và khởi tạo khi cần. Trong .NET, `IServiceProvider.GetService<T>()` về bản chất là factory method.

1.3.14. Nguyên lý thiết kế được áp dụng

Factory Method là minh họa điển hình cho nhiều nguyên lý trong SOLID và OOP:

- **Open/Closed Principle:** thêm loại phương tiện mới không cần sửa code hiện có, chỉ thêm class mới.
- **Dependency Inversion Principle:** module cấp cao (`Program.cs`) phụ thuộc vào abstraction (`IVehicle`, `VehicleCreator`), không phụ thuộc vào `Boat` hay `Bicycle` cụ thể.
- **Single Responsibility Principle:** `BoatCreator` chỉ chịu trách nhiệm tạo `Boat`; `Bicycle` chỉ chứa logic của xe đạp.
- **Program to interfaces, not implementations:** client nhận `IVehicle`, không nhận `Boat`.

1.3.15. Tóm tắt

Factory Method giải quyết vấn đề **coupling giữa client và class cụ thể** bằng cách giới thiệu một phương thức ảo để tạo đối tượng. Lớp cha định nghĩa **khi nào dùng sản phẩm**; lớp con định nghĩa **sản phẩm nào được tạo**.

Trong project FactoryMethodDemo, pattern được thể hiện rõ ràng qua:

- VehicleCreator — creator trừu tượng với CreateVehicle().
- BoatCreator, BicycleCreator — concrete creator, mỗi cái tạo một loại phương tiện.
- IVehicle — product interface.
- Boat, Bicycle — concrete product với logic riêng.
- Program.cs — client chọn creator lúc runtime, dùng sản phẩm qua interface.

Factory Method phù hợp nhất khi hệ thống cần mở rộng linh hoạt về loại đối tượng tạo ra, muốn tách biệt logic tạo khỏi logic sử dụng, và cần đảm bảo client không coupling với class cụ thể.

1.4. Prototype (Phát)

1.4.1. Tên và phân loại

Prototype Pattern có thể hiểu là **mẫu nguyên mẫu** hoặc **mẫu đối tượng mẫu**. Tên gọi này nhấn mạnh ý tưởng chính của pattern: thay vì tạo object mới từ đầu bằng constructor, ta tạo object mới bằng cách sao chép một object mẫu đã tồn tại.

Prototype thuộc nhóm **Creational Pattern** vì nó tập trung vào cách tạo object. Tuy nhiên, khác với Factory Method hoặc Abstract Factory, Prototype không đặt trọng tâm vào việc chọn constructor hoặc subclass phù hợp, mà đặt trọng tâm vào việc **clone trạng thái của một object mẫu**.

Nói ngắn gọn, Prototype là pattern dùng để tạo object mới bằng cách sao chép object có sẵn, đặc biệt hữu ích khi object cần tạo có cấu hình phức tạp hoặc chi phí khởi tạo cao.

1.4.2. Vấn đề cần giải quyết

Trong nhiều hệ thống, việc tạo object mới không phải lúc nào cũng đơn giản. Có những object cần rất nhiều bước khởi tạo, phải đọc dữ liệu từ database/file/API, phải validate nhiều thông tin, hoặc phải thiết lập một cấu trúc object con phức tạp.

Ví dụ:

- Một ReportTemplate có sẵn header, footer, style, danh sách section, biểu đồ và rule định dạng.
- Một enemy trong game có nhiều thông số như máu, sát thương, tốc độ, AI behavior, sprite, animation.
- Một form đăng ký có nhiều field, rule validation và giá trị mặc định.
- Một cấu hình hệ thống cần đọc từ file hoặc database trước khi sử dụng.
- Một object gần giống object cũ, chỉ khác một vài thuộc tính nhỏ.

Nếu client luôn phải gọi constructor và truyền đầy đủ thông tin, code tạo object sẽ dài, lặp lại và dễ sai. Ngoài ra, client có thể bị phụ thuộc quá mạnh vào class cụ thể, constructor cụ thể và chi tiết khởi tạo bên trong.

Ví dụ không dùng Prototype:

```
var goblin = new Enemy(  
    name: "Goblin",  
    health: 100,  
    damage: 15,  
    speed: 3,
```

```

    sprite: LoadSprite("goblin.png"),
    aiProfile: LoadAI("aggressive"),
    skills: new List<string> { "Slash", "Dodge" }
);

```

```

var strongGoblin = new Enemy(
    name: "Strong Goblin",
    health: 150,
    damage: 20,
    speed: 3,
    sprite: LoadSprite("goblin.png"),
    aiProfile: LoadAI("aggressive"),
    skills: new List<string> { "Slash", "Dodge" }
);

```

Hai object gần như giống nhau nhưng code khởi tạo bị lặp lại. Khi logic khởi tạo thay đổi, nhiều nơi trong code cũng phải sửa theo.

Vấn đề cốt lõi là: **làm sao tạo nhanh object mới dựa trên một object đã được cấu hình sẵn mà không cần lặp lại toàn bộ logic khởi tạo?**

Prototype Pattern giải quyết vấn đề này bằng cách cho chính object biết cách sao chép nó. Client chỉ cần chọn object mẫu và gọi Clone().

1.4.3. Mục đích và ý định

Mục đích chính của Prototype là **tạo object mới bằng cách clone từ object mẫu đã có sẵn.**

Prototype thường được dùng để:

- Giảm chi phí khởi tạo object phức tạp.
- Tránh lặp lại code tạo object gần giống nhau.
- Tạo nhiều biến thể object từ một object mẫu.
- Giảm phụ thuộc của client vào constructor cụ thể.
- Cho phép thêm loại object mới bằng cách đăng ký prototype mới thay vì sửa nhiều logic tạo object.
- Hỗ trợ cơ chế template, preset, copy object, duplicate item hoặc clone configuration.

Prototype đặc biệt phù hợp khi object mẫu đã có trạng thái gần đúng với object cần tạo. Sau khi clone, client chỉ cần chỉnh sửa một vài thuộc tính khác biệt.

1.4.4. Định nghĩa

Prototype Pattern là một creational design pattern cho phép tạo object mới bằng cách sao chép một object hiện có, gọi là prototype, thay vì khởi tạo object từ đầu.

Định nghĩa ngắn gọn:

- Prototype khai báo phương thức `Clone()`.
- ConcretePrototype tự cài đặt cách sao chép chính nó.
- Client gọi `Clone()` trên object mẫu để tạo object mới.
- Prototype Registry có thể được dùng để lưu và tra cứu nhiều prototype theo key.

Nói đơn giản:

Thay vì hỏi: "Làm sao tạo object này từ đâu?"

Prototype hỏi: "Đã có object mẫu chưa? Nếu có, clone nó rồi chỉnh lại phần cần khác."

1.4.5. Ý tưởng cốt lõi

Ý tưởng chính của Prototype Pattern là: **nếu việc tạo object mới phức tạp hoặc tốn kém, hãy tạo sẵn một object mẫu, sau đó sao chép nó khi cần.**

Quy trình tư duy thường là:

1. Tạo một object mẫu với cấu hình đầy đủ.
2. Lưu object mẫu đó ở nơi phù hợp, ví dụ registry hoặc service.
3. Khi cần object mới, clone object mẫu thay vì dựng lại từ đầu.
4. Chỉnh sửa các thuộc tính khác biệt trên object vừa clone.

Ví dụ trong game:

- goblinPrototype chứa cấu hình chuẩn của Goblin.
- Khi sinh Goblin mới, game gọi `goblinPrototype.Clone()`.
- Sau đó game chỉ cần chỉnh vị trí, level hoặc vài chỉ số riêng.

Ví dụ trong hệ thống báo cáo:

- `monthlyReportTemplate` chứa layout, style, sections, charts.
- Khi tạo báo cáo tháng 5, hệ thống clone template rồi thay data tháng 5.
- Khi tạo báo cáo tháng 6, hệ thống clone template rồi thay data tháng 6.

Điểm quan trọng là object clone phải đủ độc lập để việc chỉnh sửa object mới không làm hỏng prototype gốc.

1.4.6. Shallow copy và Deep copy

Khi nói đến Prototype, vấn đề quan trọng nhất là phân biệt **shallow copy** và **deep copy**.

1.4.7. Shallow copy

Shallow copy chỉ sao chép các field cấp đầu tiên. Với field kiểu primitive hoặc value type, giá trị được copy trực tiếp. Nhưng với field là object tham chiếu như list, dictionary, object con, bản clone và bản gốc có thể cùng trỏ đến một object con.

Ví dụ:

report1.Sections ----> List A

report2.Sections ----> List A

Nếu report2 thêm section mới, report1 cũng bị ảnh hưởng vì cả hai dùng chung danh sách. Shallow copy có ưu điểm là nhanh và đơn giản, nhưng dễ gây bug nếu object có nhiều dữ liệu tham chiếu có thể thay đổi.

1.4.8. Deep copy

Deep copy sao chép cả object chính và các object con bên trong. Sau khi clone, bản gốc và bản sao độc lập hơn.

Ví dụ:

report1.Sections ----> List A

report2.Sections ----> List B

Nếu report2 thêm section mới, report1 không bị ảnh hưởng.

Deep copy an toàn hơn trong nhiều trường hợp, nhưng phức tạp hơn vì phải quyết định object con nào cần clone, object nào có thể dùng chung, object nào là immutable.

1.4.9. Khi nào dùng loại nào?

- Dùng shallow copy khi object đơn giản, không có object con mutable, hoặc dữ liệu dùng chung là an toàn.
- Dùng deep copy khi object có cấu trúc lồng nhau và bản clone cần độc lập với bản gốc.
- Có thể dùng copy có chọn lọc: clone phần dễ thay đổi, chia sẻ phần immutable hoặc tài nguyên nặng.

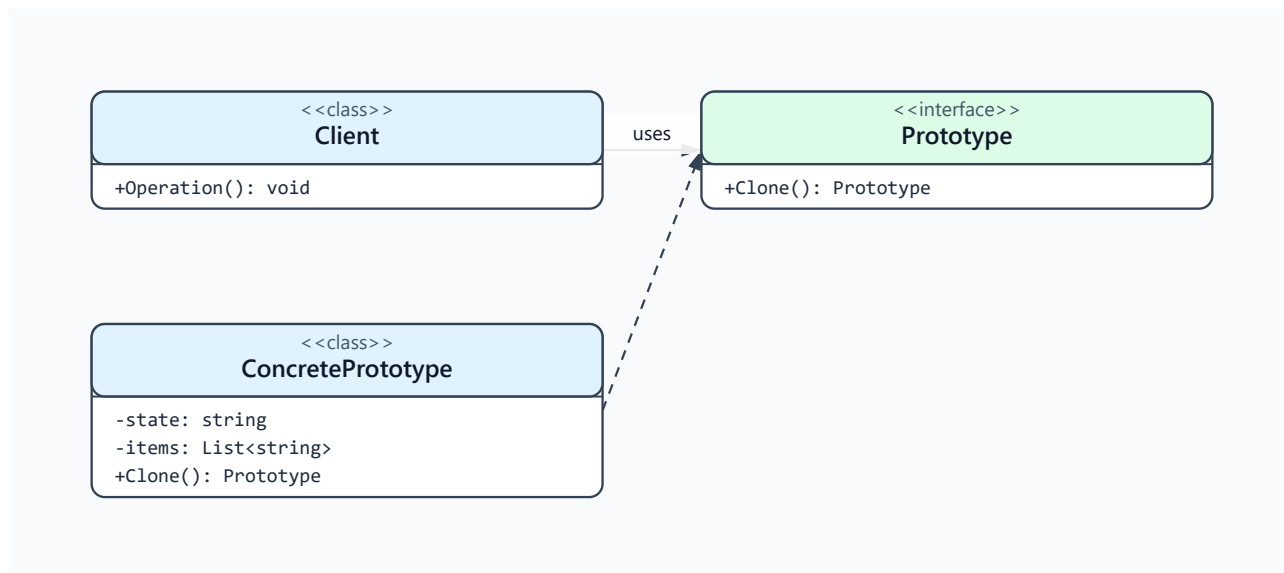
1.4.10. Thành phần chính

Các thành phần chính của Prototype Pattern gồm:

Thành phần	Vai trò
Prototype	Interface hoặc abstract class khai báo phương thức Clone(). Nó định nghĩa contract chung cho các object có khả năng tự sao chép.
ConcretePrototype	Class cụ thể có state và cài đặt logic clone. Mỗi class tự quyết định clone shallow, deep hay clone có chọn lọc.
Client	Đối tượng sử dụng prototype để tạo object mới. Client không cần biết chi tiết constructor phức tạp.
Prototype Registry	Kho lưu các prototype theo key. Thành phần này không bắt buộc, nhưng rất hữu ích khi hệ thống có nhiều object mẫu cần quản lý.

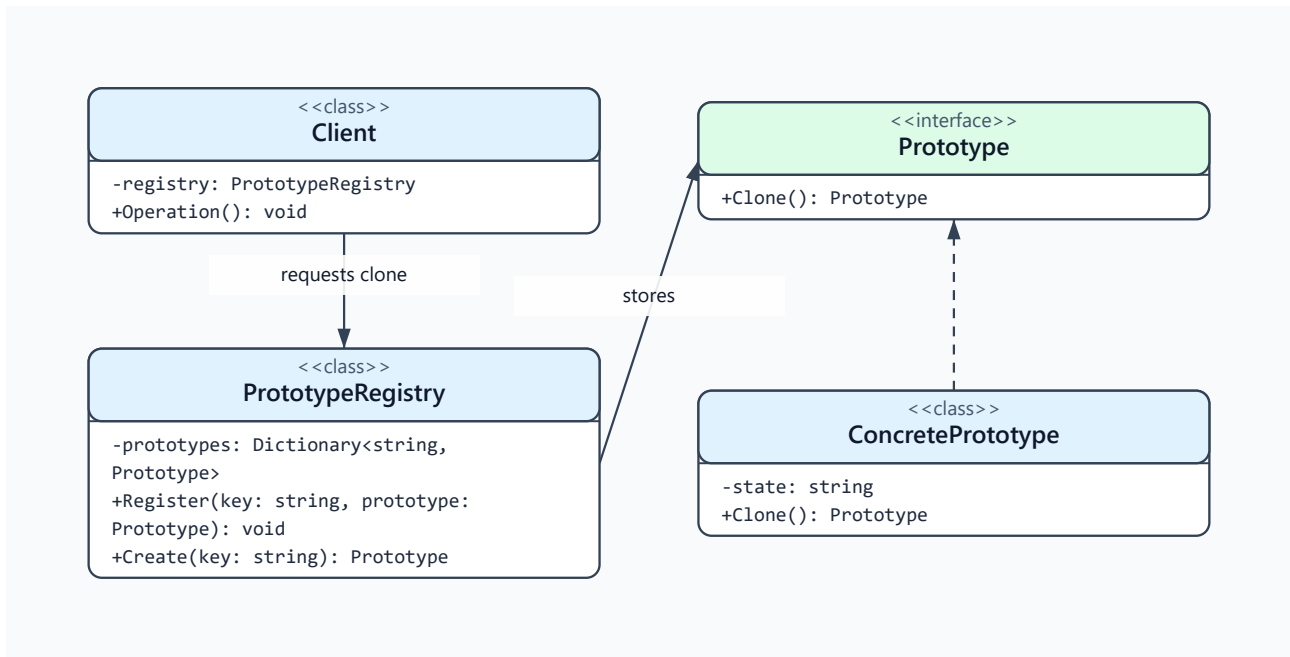
1.4.11. UML tổng quát bằng PlantUML

Sơ đồ tổng quát:



Hình 1.2: Cấu trúc UML tổng quát của Prototype Pattern

Sơ đồ có Prototype Registry:



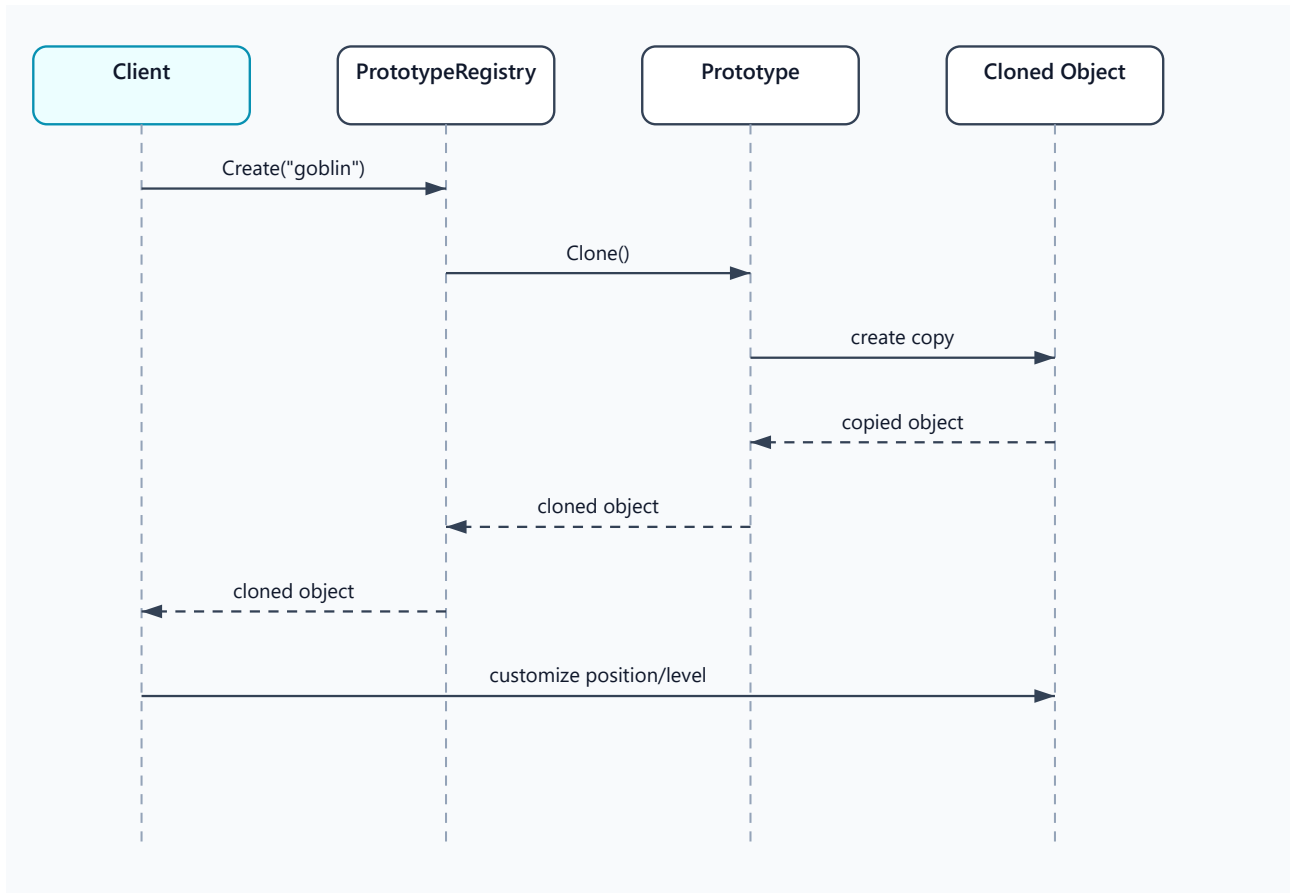
Hình 1.3: Cấu trúc Prototype Registry

1.4.12. Luồng hoạt động

Luồng hoạt động cơ bản của Prototype Pattern:

1. Client cần tạo một object mới.
2. Client chọn một prototype phù hợp.
3. Client gọi Clone() trên prototype.
4. Prototype tự sao chép dữ liệu của nó sang object mới.
5. Client nhận object mới.
6. Client có thể tùy chỉnh thêm một vài thuộc tính nếu cần.

Biểu diễn dạng sequence:



Hình 1.4: Luồng clone object từ prototype

Nếu không dùng registry, client có thể giữ trực tiếp prototype và gọi Clone() trên prototype đó.

1.4.13. Ví dụ minh họa: Enemy trong game

Giả sử game cần tạo nhiều loại enemy. Mỗi enemy có nhiều thuộc tính như tên, máu, sát thương, danh sách kỹ năng. Một số enemy gần giống nhau, chỉ khác level hoặc vị trí.

Nếu dùng constructor ở mọi nơi, code sẽ lặp lại nhiều. Ta có thể tạo prototype cho từng loại enemy, sau đó clone khi cần sinh enemy mới.

1.4.14. Code C# minh họa

```

using System;

using System.Collections.Generic;

using System.Linq;
  
```

```

public interface IEnemyPrototype
{
    IEnemyPrototype Clone();
}

public class Enemy : IEnemyPrototype
{
    public string Name { get; set; }
    public int Health { get; set; }
    public int Damage { get; set; }
    public int Level { get; set; }
    public Position Position { get; set; }
    public List<string> Skills { get; set; }

    public Enemy(
        string name,
        int health,
        int damage,
        int level,
        Position position,
        List<string> skills)
    {
        Name = name;

```

```

    Health = health;

    Damage = damage;

    Level = level;

    Position = position;

    Skills = skills;
}

```

```

public IEnemyPrototype Clone()
{
    // Deep copy cho các object mutable như Position và List<string>.
    return new Enemy(
        Name,
        Health,
        Damage,
        Level,
        new Position(Position.X, Position.Y),
        Skills.ToList()
    );
}

```

```

public override string ToString()
{
    return $"{Name} | HP: {Health} | Damage: {Damage} | Level: {Level} |

```

```
Position: ({Position.X}, {Position.Y}) | Skills: {string.Join(", ",
Skills)}}";
```

```
}
```

```
}
```

```
public class Position
```

```
{
```

```
    public int X { get; set; }
```

```
    public int Y { get; set; }
```

```
    public Position(int x, int y)
```

```
    {
```

```
        X = x;
```

```
        Y = y;
```

```
    }
```

```
}
```

```
public class EnemyRegistry
```

```
{
```

```
    private readonly Dictionary<string, IEnemyPrototype> _prototypes =
```

```
new();
```

```
    public void Register(string key, IEnemyPrototype prototype)
```

```

    {
        _prototypes[key] = prototype;
    }

    public Enemy Create(string key)
    {
        if (!_prototypes.ContainsKey(key))
        {
            throw new ArgumentException($"Prototype '{key}' does not
exist.");
        }

        return (Enemy)_prototypes[key].Clone();
    }
}

public class Program
{
    public static void Main()
    {
        var registry = new EnemyRegistry();

        var goblinPrototype = new Enemy(

```

```
        name: "Goblin",  
        health: 100,  
        damage: 15,  
        level: 1,  
        position: new Position(0, 0),  
        skills: new List<string> { "Slash", "Dodge" }  
    );
```

```
registry.Register("goblin", goblinPrototype);
```

```
var goblinA = registry.Create("goblin");  
goblinA.Position.X = 10;  
goblinA.Position.Y = 5;
```

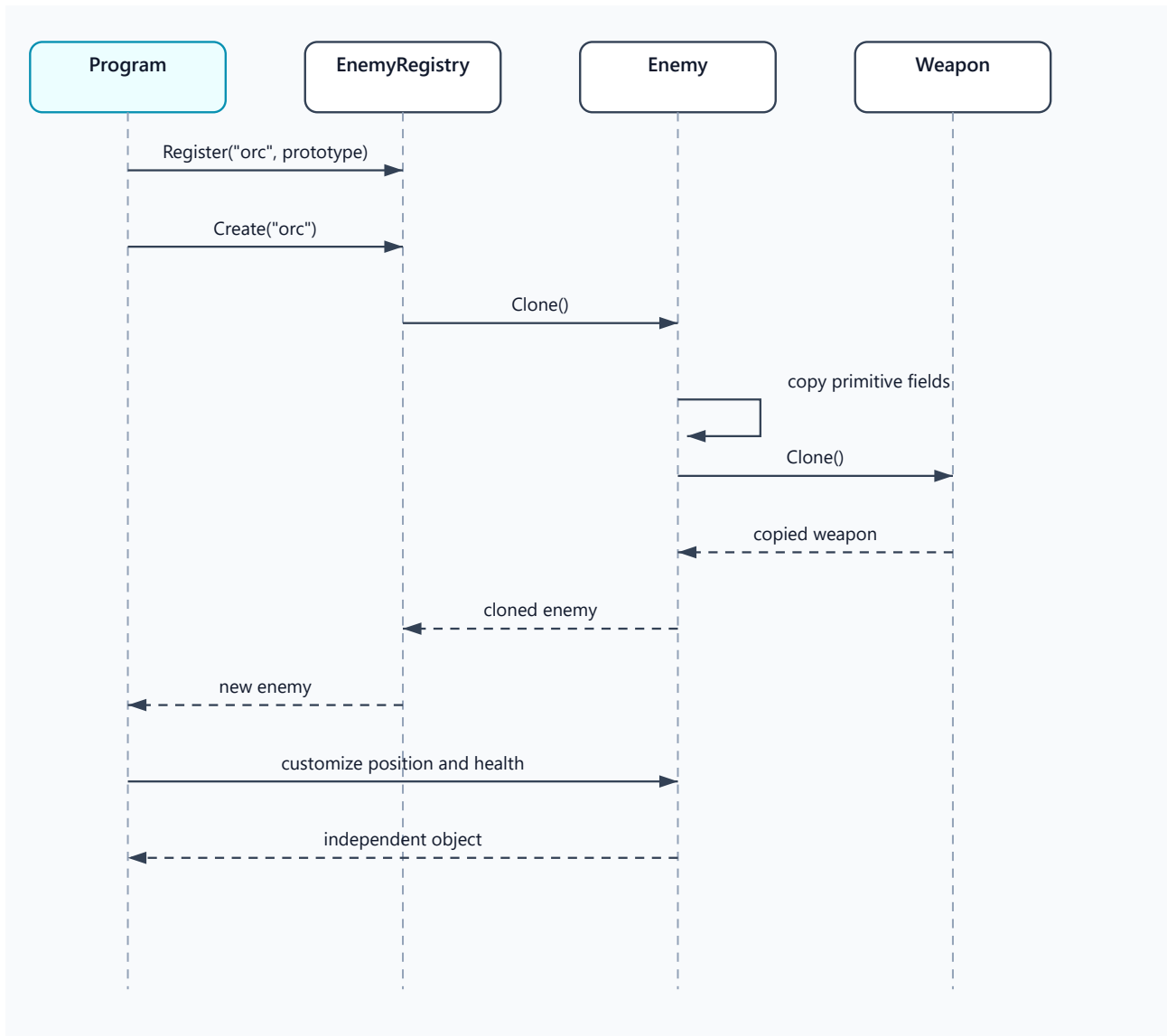
```
var goblinB = registry.Create("goblin");  
goblinB.Level = 3;  
goblinB.Health = 150;  
goblinB.Position.X = 30;  
goblinB.Skills.Add("Poison Attack");
```

```
Console.WriteLine(goblinPrototype);  
Console.WriteLine(goblinA);  
Console.WriteLine(goblinB);
```

```
}  
  
}
```

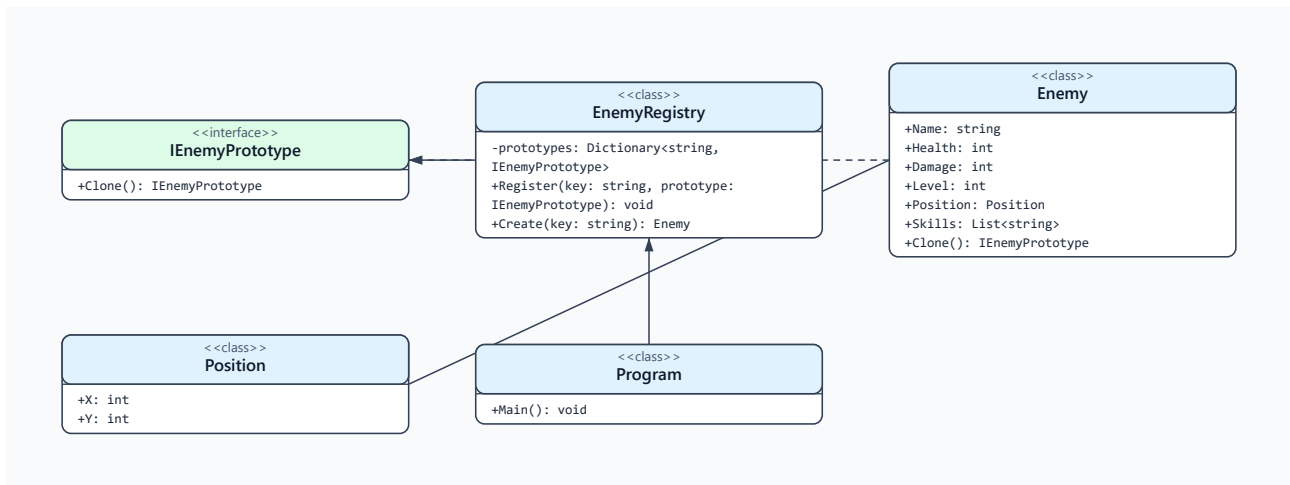
Trong ví dụ trên:

- IEnemyPrototype là Prototype.
- Enemy là ConcretePrototype.
- EnemyRegistry là Prototype Registry.
- Program đóng vai trò Client.
- Clone() tạo bản sao độc lập cho Position và Skills để tránh bug do dùng chung reference.



Hình 1.5: Luồng chạy code ví dụ clone Enemy prototype

1.4.15. Class diagram cho ví dụ



Hình 1.6: Class diagram của ví dụ Enemy trong game

1.4.16. Ví dụ minh họa: Report Template

Một ví dụ gần với hệ thống quản lý là tạo báo cáo. Báo cáo tháng, báo cáo quý, báo cáo doanh thu có thể dùng chung cấu trúc: tiêu đề, danh sách section, biểu đồ, footer. Mỗi lần tạo báo cáo mới, ta chỉ cần clone template rồi thay dữ liệu.

```
using System;

using System.Collections.Generic;

using System.Linq;

public interface IReportPrototype
{
    IReportPrototype Clone();
}

public class Report : IReportPrototype
{

```

```

public string Title { get; set; }

public string Theme { get; set; }

public List<string> Sections { get; set; } = new();

public IReportPrototype Clone()
{
    return new Report
    {
        Title = Title,
        Theme = Theme,
        Sections = Sections.ToList()
    };
}

public void Print()
{
    Console.WriteLine($"Report: {Title} | Theme: {Theme}");
    Console.WriteLine("Sections: " + string.Join(", ", Sections));
}
}

public class Demo
{

```

```

public static void Main()
{
    var monthlyTemplate = new Report
    {
        Title = "Monthly Revenue Report",
        Theme = "Corporate",
        Sections = new List<string> { "Summary", "Revenue Chart", "Top
Products" }
    };

    var mayReport = (Report)monthlyTemplate.Clone();
    mayReport.Title = "Revenue Report - May 2026";
    mayReport.Sections.Add("May Analysis");

    var juneReport = (Report)monthlyTemplate.Clone();
    juneReport.Title = "Revenue Report - June 2026";
    juneReport.Sections.Add("June Analysis");

    monthlyTemplate.Print();
    mayReport.Print();
    juneReport.Print();
}
}

```

Nếu Sections không được copy bằng `ToList()`, template gốc có thể bị thay đổi khi report clone thêm section mới.

1.4.17. Đánh giá

1.4.18. Ưu điểm

Ưu điểm	Giải thích
Giảm chi phí khởi tạo object phức tạp	Object mẫu đã được cấu hình sẵn, client chỉ cần clone thay vì dựng lại từ đầu.
Giảm code lặp	Khi cần nhiều object gần giống nhau, ta không phải lặp lại toàn bộ constructor hoặc logic setup.
Không phụ thuộc trực tiếp vào constructor cụ thể	Client chỉ biết interface <code>Clone()</code> , không cần biết class có bao nhiêu constructor hoặc cần tham số gì.
Dễ tạo biến thể object	Clone object mẫu rồi chỉnh sửa một vài thuộc tính riêng như level, position, title hoặc theme.
Kết hợp tốt với Registry	Có thể lưu nhiều prototype theo key và tạo object mới linh hoạt theo cấu hình.

1.4.19. Nhược điểm

Nhược điểm	Giải thích
Clone object phức tạp không phải lúc nào cũng dễ	Nếu object có nhiều quan hệ, object con, tài nguyên ngoài hoặc vòng tham chiếu, clone sẽ khó cài đặt đúng.
Dễ nhầm giữa shallow copy và deep copy	Nếu copy reference sai cách, object clone và object gốc có thể ảnh hưởng lẫn nhau.
Có thể che giấu logic tạo object	Client chỉ thấy clone, nên đôi khi khó biết object mới thật sự được tạo như thế nào.

Không phù hợp nếu object đơn giản	Nếu object chỉ có vài field và constructor đơn giản, Prototype có thể làm thiết kế rườm rà hơn.
Cần chú ý identity và resource	Các field như ID, connection, file handle, event subscription không nên bị clone máy móc.

1.4.20. Khi nào nên dùng Prototype?

Nên dùng Prototype khi:

- Object cần tạo có chi phí khởi tạo cao.
- Object có nhiều thuộc tính hoặc cấu trúc phức tạp.
- Có nhiều object gần giống nhau, chỉ khác vài thuộc tính nhỏ.
- Muốn tạo object mới dựa trên trạng thái runtime của object hiện có.
- Muốn tránh phụ thuộc trực tiếp vào constructor cụ thể.
- Cần cơ chế duplicate/copy trong ứng dụng, ví dụ copy shape trong editor, clone enemy trong game, clone template báo cáo.
- Muốn quản lý nhiều object mẫu thông qua registry.

Ví dụ thực tế:

- Game clone enemy, item, projectile, NPC template.
- Graphic editor clone shape, layer, component.
- Document editor clone template, paragraph style, slide layout.
- Business app clone report template, form template, workflow template.
- Testing clone object fixture đã setup sẵn.

1.4.21. Khi nào không nên dùng Prototype?

Không nên dùng Prototype khi:

- Object rất đơn giản và constructor rõ ràng.

- Clone object phức tạp hơn tạo mới.
- Object chứa nhiều resource không thể clone an toàn như socket, database connection, stream, lock, thread.
- Object có identity quan trọng và việc clone có thể tạo bug, ví dụ clone cả Id của entity database.
- Hệ thống không có nhu cầu tạo nhiều object tương tự nhau.
- Team dễ nhầm giữa shallow copy và deep copy mà không có convention rõ ràng.

1.4.22. Lưu ý khi triển khai

Khi triển khai Prototype trong dự án thực tế, cần chú ý:

- Xác định rõ clone là shallow copy hay deep copy.
- Không clone máy móc các field định danh như Id, CreatedAt, Version nếu chúng phải unique.
- Với collection mutable, thường nên tạo collection mới.
- Với object con immutable, có thể chia sẻ để tiết kiệm chi phí.
- Với resource bên ngoài như file, socket, database connection, nên tạo lại hoặc bỏ qua.
- Có thể dùng copy constructor, method Clone(), serialization, mapping library hoặc manual copy tùy ngôn ngữ và yêu cầu.
- Nên viết test để đảm bảo thay đổi object clone không ảnh hưởng prototype gốc.

Ví dụ test tư duy:

Clone report template -> thêm section vào report clone

Kỳ vọng: template gốc không có section mới

1.4.23. So sánh Prototype với Factory Method

Tiêu chí	Prototype	Factory Method
----------	-----------	----------------

Cách tạo object	Clone từ object mẫu.	Gọi factory method để tạo object.
Trọng tâm	Sao chép trạng thái có sẵn.	Ẩn logic khởi tạo object.
Phù hợp khi	Object mẫu đã có cấu hình phức tạp.	Muốn subclass quyết định object nào được tạo.
Có cần prototype object không?	Có.	Không.
Ví dụ	<code>goblinPrototype.Clone()</code>	<code>EnemyFactory.CreateEnemy("goblin")</code>

Điểm khác biệt quan trọng: Factory Method tạo object mới dựa trên logic tạo object, còn Prototype tạo object mới dựa trên một object mẫu đã tồn tại.

1.4.24. So sánh Prototype với Abstract Factory

Tiêu chí	Prototype	Abstract Factory
Mục đích chính	Tạo object mới từ object mẫu.	Tạo họ object liên quan với nhau.
Cách tạo	Clone.	Factory tạo object mới.
Phù hợp khi	Có nhiều object gần giống nhau.	Cần tạo nhiều object cùng một family.
Mức độ tổ chức	Tập trung vào từng object mẫu.	Tập trung vào nhóm sản phẩm liên quan.
Ví dụ	Clone enemy mẫu.	Tạo bộ UI Windows/Mac: Button, Checkbox, Dialog.

Abstract Factory phù hợp khi cần đảm bảo nhiều object được tạo ra thuộc cùng một family. Prototype phù hợp khi mỗi object cụ thể có thể được sao chép từ mẫu.

1.4.25. So sánh Prototype với Builder

Tiêu chí	Prototype	Builder
Mục đích chính	Clone object có sẵn.	Tạo object phức tạp từng bước.
Cách tạo object	Sao chép từ mẫu.	Gọi chuỗi bước cấu hình.
Phù hợp khi	Có object mẫu gần đúng.	Cần kiểm soát quá trình build chi tiết.
Khả năng tùy biến	Tùy biến sau khi clone.	Tùy biến trong lúc build.
Ví dụ	Clone một ReportTemplate.	Build Report với title, sections, charts.

Builder tốt khi quá trình tạo object cần nhiều bước rõ ràng. Prototype tốt khi đã có object mẫu hoàn chỉnh và muốn tạo bản sao nhanh.

1.4.26. So sánh Prototype với Flyweight

Tiêu chí	Prototype	Flyweight
Nhóm pattern	Creational.	Structural.
Mục đích chính	Tạo object mới bằng clone.	Chia sẻ object/dữ liệu dùng chung để tiết kiệm bộ nhớ.
Object tạo ra	Object mới, thường độc lập.	Object dùng chung, không nhất thiết tạo mới.
Trạng thái	Clone state cần thiết.	Tách intrinsic và extrinsic state.
Ví dụ	Clone Enemy mẫu thành enemy mới.	Nhiều Tree dùng chung một TreeType.

Prototype tạo thêm object mới. Flyweight cố gắng giảm số object/dữ liệu bị lặp bằng cách dùng chung phần trạng thái giống nhau.

1.4.27. So sánh Prototype với Memento

Tiêu chí	Prototype	Memento
Mục đích chính	Tạo object mới từ object mẫu.	Lưu và khôi phục trạng thái trước đó.
Kết quả	Có một object mới để sử dụng.	Có snapshot để restore state.
Trọng tâm	Object creation.	State history/undo/rollback.
Client dùng khi	Muốn copy/duplicate object.	Muốn quay lại trạng thái cũ.
Ví dụ	Duplicate một shape trong editor.	Undo thao tác chỉnh sửa shape.

Prototype và Memento đều có thể liên quan đến việc sao chép trạng thái, nhưng mục đích khác nhau. Prototype dùng để tạo object mới; Memento dùng để lưu lịch sử trạng thái phục vụ restore.

1.4.28. Kết luận

Prototype Pattern là một mẫu thiết kế hữu ích khi hệ thống cần tạo nhiều object phức tạp hoặc nhiều object gần giống nhau. Thay vì khởi tạo lại từ đầu, ta tạo object mẫu và clone nó khi cần. Pattern này giúp giảm lặp code, giảm chi phí khởi tạo và giúp client ít phụ thuộc hơn vào constructor cụ thể. Tuy nhiên, Prototype cũng yêu cầu lập trình viên hiểu rõ shallow copy, deep copy và các vấn đề liên quan đến object identity, resource, collection mutable.

Có thể ghi nhớ Prototype bằng một câu ngắn:

Prototype = tạo object mới bằng cách sao chép object mẫu đã có.

Khi object đơn giản, constructor rõ ràng thì không cần dùng Prototype. Nhưng khi object phức tạp, khởi tạo tốn kém, hoặc cần tạo nhiều biến thể gần giống nhau, Prototype là một lựa chọn rất đáng cân nhắc.

1.5. Singleton (Phát)

1.5.1. Tên và phân loại

Singleton Pattern là một mẫu thiết kế thuộc nhóm **Creational Pattern**. Nhóm Creational tập trung vào cách tạo object sao cho linh hoạt, kiểm soát được vòng đời object và giảm phụ thuộc trực tiếp giữa client với quá trình khởi tạo.

Singleton giải quyết một nhu cầu rất cụ thể: trong toàn bộ chương trình chỉ nên tồn tại **một instance duy nhất** của một class nào đó, đồng thời toàn hệ thống có thể truy cập đến instance này thông qua một điểm truy cập chung.

Nói ngắn gọn, Singleton là pattern dùng để quản lý một object duy nhất được chia sẻ trong phạm vi ứng dụng.

1.5.2. Vấn đề cần giải quyết

Trong một số hệ thống, có những class không nên được tạo ra nhiều lần. Nếu mỗi nơi trong chương trình đều tự new một object mới, hệ thống có thể gặp các vấn đề như dữ liệu không nhất quán, lãng phí tài nguyên hoặc khó kiểm soát trạng thái toàn cục.

Ví dụ:

- ConfigurationManager: quản lý cấu hình ứng dụng.
- Logger: ghi log tập trung.
- DatabaseConnectionManager: quản lý kết nối hoặc connection pool.
- CacheManager: quản lý cache toàn cục.
- AppSettings: chứa setting dùng chung.
- FileManager: quản lý quyền truy cập đến một file dùng chung.

Giả sử nhiều service trong hệ thống tự tạo logger riêng:

```
var logger1 = new Logger();
```

```
var logger2 = new Logger();
```

```
var logger3 = new Logger();
```

Nếu Logger có trạng thái như đường dẫn file log, cấp độ log hiện tại hoặc buffer ghi log, việc có nhiều instance có thể khiến log bị phân tán, cấu hình không đồng bộ hoặc cạnh tranh tài nguyên.

Vấn đề đặt ra là:

Làm sao đảm bảo một class chỉ có một instance duy nhất, ngăn client tự tạo instance mới, nhưng vẫn cung cấp được một điểm truy cập chung cho toàn hệ thống?

1.5.3. Định nghĩa

Singleton Pattern đảm bảo rằng một class chỉ có một instance duy nhất trong toàn bộ chương trình, đồng thời cung cấp một cách truy cập toàn cục đến instance đó.

Singleton thường gồm ba ý chính:

- Constructor bị ẩn bằng private hoặc protected để client không thể tự new.
- Instance duy nhất được lưu trong chính class đó, thường là một field static.
- Client truy cập object thông qua một static property hoặc static method, ví dụ `Logger.Instance`.

Singleton không chỉ là “biến global”. Điểm quan trọng của Singleton là class tự kiểm soát việc tạo instance, đảm bảo chỉ có một object hợp lệ tồn tại theo thiết kế.

1.5.4. Ý tưởng cốt lõi

Ý tưởng chính của Singleton là:

Ngăn client tự tạo object mới, sau đó cung cấp một instance duy nhất được quản lý bên trong class.

Thay vì để client gọi:

```
var logger = new Logger();
```

Client sẽ gọi:

```
var logger = Logger.Instance;
```

Lần đầu tiên truy cập, Singleton có thể tạo instance. Các lần sau, nó trả về lại cùng object đã được tạo trước đó.

Singleton có thể được cài đặt theo nhiều biến thể:

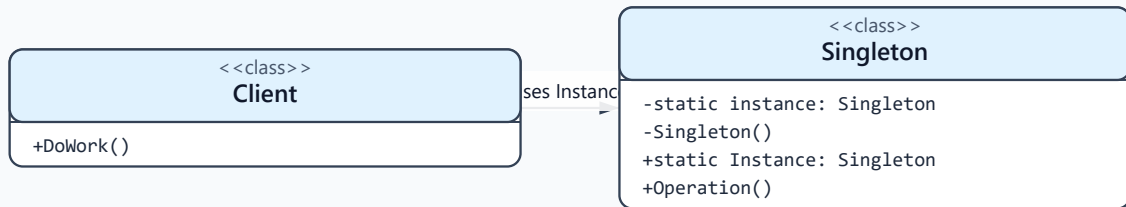
- **Eager initialization:** tạo instance ngay khi class được load.
- **Lazy initialization:** chỉ tạo instance khi có nhu cầu sử dụng lần đầu.
- **Thread-safe Singleton:** đảm bảo an toàn khi nhiều thread cùng truy cập.
- **Singleton qua DI container:** dùng vòng đời singleton trong dependency injection thay vì tự viết `ClassName.Instance`.

1.5.5. Thành phần chính

Thành phần	Vai trò
Singleton Class	Class tự quản lý instance duy nhất của chính nó.
Private Constructor	Ngăn client tạo object mới bằng toán tử <code>new</code> .
Static Instance	Field/property static lưu instance duy nhất.
Static Accessor	Cung cấp điểm truy cập toàn cục, ví dụ <code>Instance</code> hoặc <code>GetInstance()</code> .
Client	Sử dụng instance thông qua accessor thay vì trực tiếp khởi tạo.

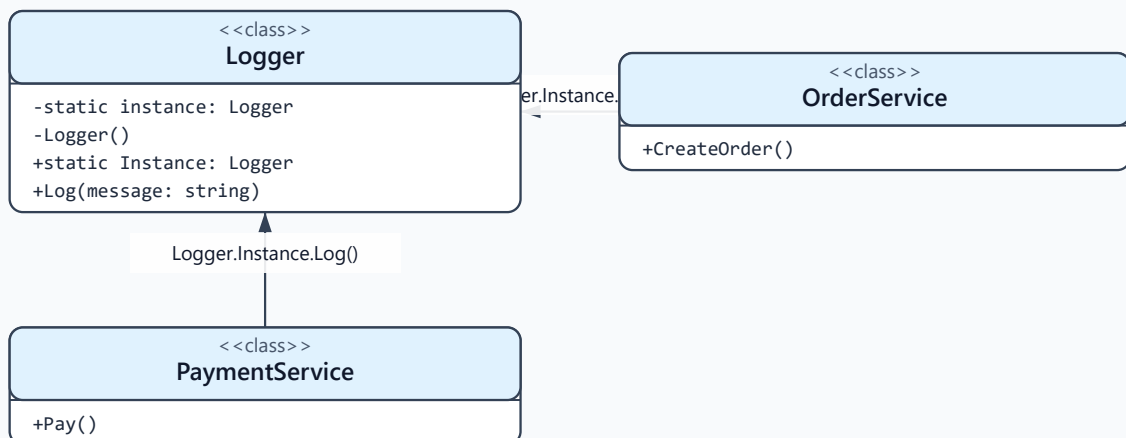
1.5.6. UML bằng PlantUML

1.5.7. UML tổng quát



Hình 1.7: Cấu trúc UML tổng quát của Singleton Pattern

1.5.8. UML ví dụ Logger



Hình 1.8: Class diagram của ví dụ Logger Singleton

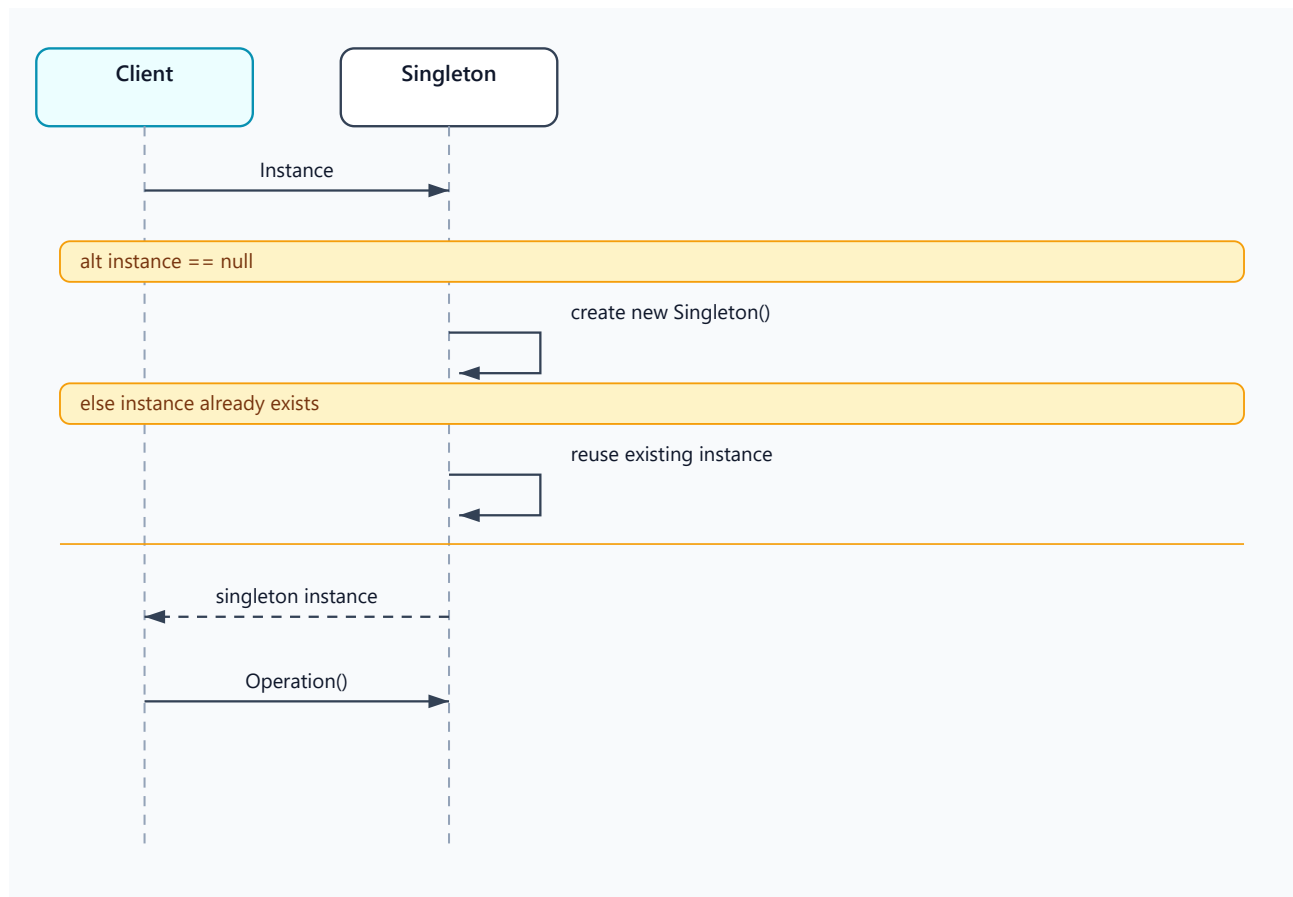
1.5.9. Luồng hoạt động

Luồng hoạt động cơ bản của Singleton:

1. Client cần sử dụng object singleton.
2. Client gọi Singleton.Instance hoặc Singleton.GetInstance().
3. Singleton kiểm tra instance đã được tạo hay chưa.
4. Nếu chưa có, Singleton tạo instance mới.

5. Nếu đã có, Singleton trả về instance cũ.
6. Client dùng instance đó để gọi method cần thiết.
7. Những lần gọi sau đều nhận lại cùng một instance.

Có thể biểu diễn bằng sequence diagram:



Hình 1.9: Luồng truy cập instance duy nhất

1.5.10. Code minh họa C#

1.5.11. Cách cài đặt đơn giản

```
public sealed class Logger
```

```
{
```

```
    private static Logger? _instance;
```

```

private Logger()
{
    Console.WriteLine("Logger created");
}

public static Logger Instance
{
    get
    {
        if (_instance == null)
        {
            _instance = new Logger();
        }

        return _instance;
    }
}

public void Log(string message)
{
    Console.WriteLine($"[LOG] {message}");
}
}

```


Client sử dụng:

```
public class OrderService
{
    public void CreateOrder()
    {
        Logger.Instance.Log("Creating order...");
        // Business logic
        Logger.Instance.Log("Order created successfully.");
    }
}
```

```
public class PaymentService
{
    public void Pay()
    {
        Logger.Instance.Log("Processing payment...");
        // Payment logic
        Logger.Instance.Log("Payment completed.");
    }
}
```

Cách này dễ hiểu nhưng chưa an toàn trong môi trường đa luồng. Nếu hai thread cùng truy cập lần đầu, cả hai có thể cùng thấy `_instance == null` và tạo ra hai object.

1.5.12. Thread-safe Singleton bằng Lazy

Trong C#, cách gọn và an toàn hơn là dùng Lazy<T>:

```
public sealed class AppConfiguration
{
    private static readonly Lazy<AppConfiguration> _instance =
        new Lazy<AppConfiguration>(() => new AppConfiguration());

    private readonly Dictionary<string, string> _settings = new();

    private AppConfiguration()
    {
        _settings["AppName"] = "SE401 Demo";
        _settings["Environment"] = "Development";
    }

    public static AppConfiguration Instance => _instance.Value;

    public string Get(string key)
    {
        return _settings.TryGetValue(key, out var value)
            ? value
            : string.Empty;
    }
}
```

```
    }  
}
```

Client:

```
string appName = AppConfiguration.Instance.Get("AppName");  
Console.WriteLine(appName);
```

Ưu điểm của Lazy<T>:

- Chỉ tạo instance khi cần.
- Thread-safe theo mặc định.
- Code ngắn gọn hơn so với tự viết lock.

1.5.13. Singleton trong ASP.NET Core bằng DI

Trong các ứng dụng hiện đại, đặc biệt là ASP.NET Core, thường nên dùng DI container để quản lý singleton lifetime:

```
public interface IAppLogger
```

```
{  
    void Log(string message);  
}
```

```
public class AppLogger : IAppLogger
```

```
{  
    public void Log(string message)  
    {  
        Console.WriteLine($"[APP] {message}");  
    }  
}
```

```
    }  
}
```

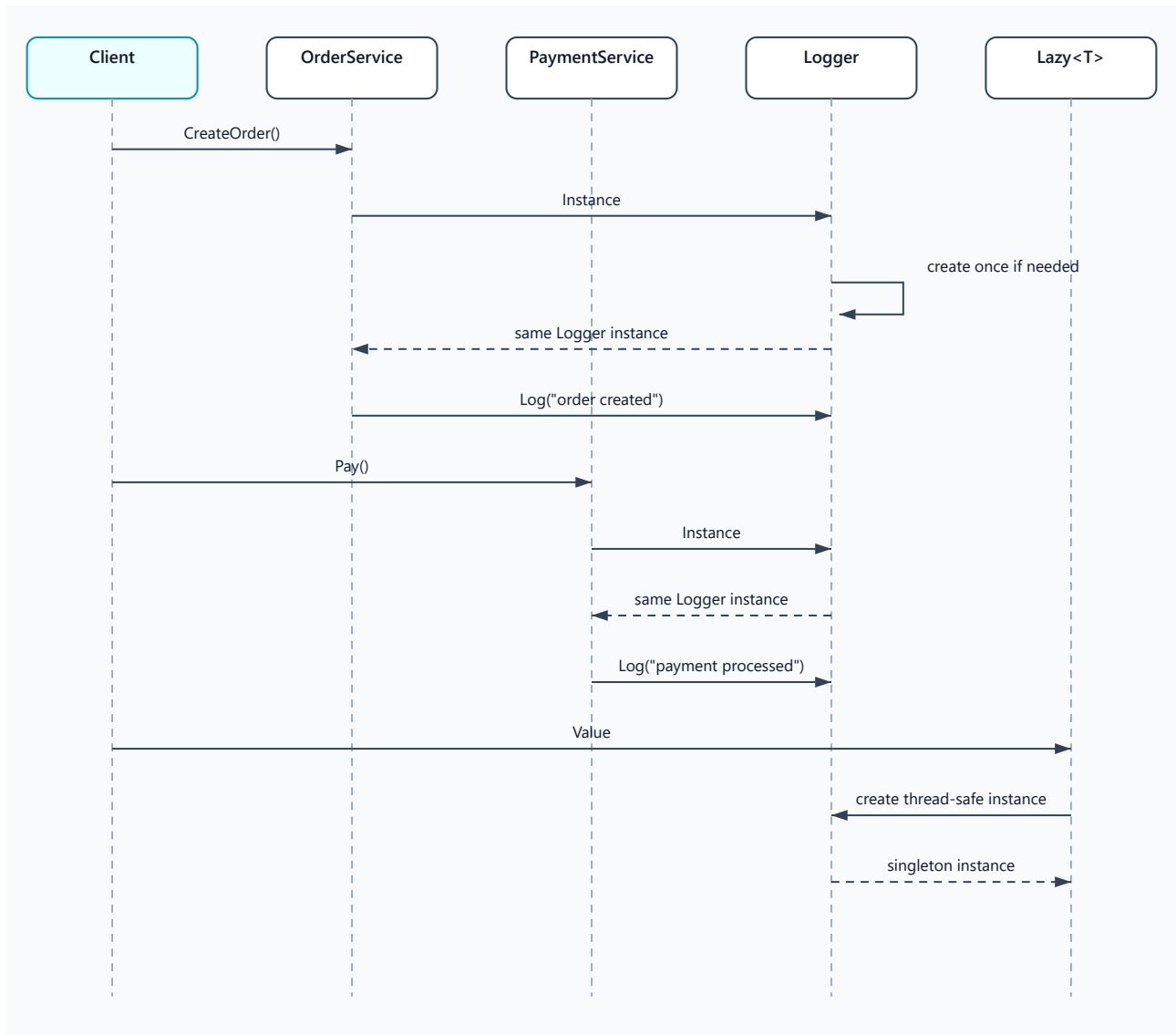
Đăng ký trong Program.cs:

```
builder.Services.AddSingleton<IAppLogger, AppLogger>();
```

Sử dụng qua constructor injection:

```
public class OrderService  
{  
    private readonly IAppLogger _logger;  
  
    public OrderService(IAppLogger logger)  
    {  
        _logger = logger;  
    }  
  
    public void CreateOrder()  
    {  
        _logger.Log("Create order");  
    }  
}
```

Về bản chất, DI container vẫn chỉ tạo một instance AppLogger, nhưng client không gọi AppLogger.Instance. Điều này giúp giảm coupling và dễ test hơn.



Hình 1.10: Luồng chạy code ví dụ Logger Singleton

1.5.14. Đánh giá

1.5.15. Ưu điểm

Ưu điểm	Giải thích
Đảm bảo chỉ có một instance	Hữu ích khi object đại diện cho tài nguyên hoặc trạng thái dùng chung.
Cung cấp điểm truy cập chung	Client có thể lấy instance từ một nơi thống nhất.

Có thể lazy initialization	Chỉ tạo object khi thật sự cần, tránh lãng phí tài nguyên.
Kiểm soát vòng đời object	Class hoặc DI container có thể quản lý thời điểm tạo và tái sử dụng object.
Phù hợp cho một số tài nguyên toàn cục	Ví dụ logger, configuration, cache manager hoặc application context.

1.5.16. Nhược điểm

Nhược điểm	Giải thích
Dễ biến thành global state	Nếu Singleton chứa nhiều state thay đổi, hệ thống có thể khó kiểm soát và khó dự đoán.
Tăng coupling	Client phụ thuộc trực tiếp vào <code>ClassName.Instance</code> , khó thay thế implementation.
Khó test	Unit test khó mock nếu code gọi trực tiếp Singleton thay vì abstraction.
Cần cẩn thận với đa luồng	Lazy initialization không thread-safe có thể tạo lỗi trong môi trường concurrent.
Có thể vi phạm SRP	Class vừa làm nghiệp vụ, vừa chịu trách nhiệm quản lý instance của chính nó.

1.5.17. Khi nào nên dùng và không nên dùng

1.5.18. Nên dùng khi

- Class thật sự chỉ nên có một instance trong toàn bộ ứng dụng.
- Object quản lý tài nguyên dùng chung như configuration, logger, cache.
- Cần kiểm soát chặt chẽ việc tạo object.
- Instance không chứa nhiều mutable state gây khó kiểm soát.

- Có thể triển khai thông qua DI container để giảm coupling.

1.5.19. Không nên dùng khi

- Chỉ muốn truy cập object ở nhiều nơi cho tiện.
- Object chứa trạng thái thay đổi theo từng request/user/session.
- Cần unit test nhiều và cần mock dễ dàng.
- Có thể truyền dependency qua constructor injection.
- Class chỉ chứa hàm tiện ích không state; khi đó static class có thể phù hợp hơn.

1.5.20. So sánh với các pattern/khái niệm liên quan

1.5.21. Singleton vs Static Class

Tiêu chí	Singleton	Static Class
Bản chất	Một object duy nhất.	Tập hợp member static.
Có instance không?	Có instance thật sự.	Không có object theo nghĩa thông thường.
Implement interface	Có thể implement interface.	Không thể implement interface.
Đa hình	Có thể hỗ trợ ở mức nhất định.	Không hỗ trợ đa hình.
DI	Có thể inject nếu đăng ký instance.	Không phù hợp để inject.
Phù hợp khi	Cần object duy nhất có state/hành vi.	Hàm tiện ích không cần state.

1.5.22. Singleton truyền thống vs Singleton lifetime trong DI

Tiêu chí	Singleton truyền thống	Singleton trong DI
Ai quản lý instance?	Chính class đó.	DI container.
Client truy cập	ClassName.Instance.	Constructor injection.
Coupling	Cao hơn.	Thấp hơn vì phụ thuộc abstraction.

Testability	Khó mock hơn.	Dễ mock hơn.
Phù hợp hiện đại	Ít được khuyến khích nếu lạm dụng.	Thường được ưu tiên trong ứng dụng lớn.

1.5.23. Singleton vs Prototype

Tiêu chí	Singleton	Prototype
Nhóm pattern	Creational.	Creational.
Mục đích	Đảm bảo chỉ có một instance.	Tạo object mới bằng cách clone object mẫu.
Số lượng object	Một instance duy nhất.	Nhiều object mới.
Cách tạo	Lấy instance có sẵn.	Clone từ prototype.
Ví dụ	ConfigurationManager.Instance	Clone nhiều Enemy từ enemy mẫu.

1.5.24. Singleton vs Flyweight

Tiêu chí	Singleton	Flyweight
Mục đích	Chỉ có một instance của một class.	Chia sẻ nhiều object/dữ liệu dùng chung theo trạng thái.
Số lượng instance	Một.	Có thể nhiều, mỗi object ứng với một intrinsic state.
Trọng tâm	Kiểm soát instance.	Tiết kiệm bộ nhớ.
Ví dụ	Một Logger.	Nhiều TreeType: Oak, Pine, Cherry.

1.5.25. Singleton vs Factory

Tiêu chí	Singleton	Factory
----------	-----------	---------

Mục đích	Đảm bảo một instance duy nhất.	Tạo object mà không để client biết chi tiết khởi tạo.
Số lượng object	Một.	Có thể nhiều.
Client nhận object bằng	Instance.	Create().
Trọng tâm	Quản lý vòng đời instance.	Đóng gói logic tạo object.
Ví dụ	Logger.Instance.	PaymentFactory.Create("momo").

1.5.26. Kết luận

Singleton là pattern hữu ích khi hệ thống thật sự cần một object duy nhất được chia sẻ, chẳng hạn configuration, logger hoặc cache manager. Tuy nhiên, Singleton rất dễ bị lạm dụng và biến thành global state, làm tăng coupling và gây khó khăn cho unit test.

Trong các ứng dụng hiện đại, thay vì luôn viết Singleton truyền thống bằng `ClassName.Instance`, nên cân nhắc dùng dependency injection với singleton lifetime. Cách này vẫn đảm bảo chỉ có một instance, nhưng giữ code linh hoạt, dễ test và dễ thay thế implementation hơn.

Chương 2. Các mẫu cấu trúc

2.1. Adapter (Phát)

2.1.1. Tên và phân loại

Adapter Pattern còn được gọi là **Wrapper** hoặc **Translator**. Trong tiếng Việt, có thể hiểu đây là **mẫu chuyển đổi giao diện**.

Adapter thuộc nhóm **Structural Pattern** vì trọng tâm của nó không nằm ở thuật toán xử lý nghiệp vụ, mà nằm ở cách tổ chức, ghép nối và làm cho các lớp hoặc object có interface khác nhau có thể làm việc cùng nhau. Nói cách khác, Adapter giải quyết vấn đề tương thích ở tầng cấu trúc giữa các thành phần phần mềm.

2.1.2. Vấn đề cần giải quyết

Trong thực tế, hệ thống phần mềm hiếm khi được xây dựng hoàn toàn từ đầu và hiếm khi mọi thành phần đều có interface khớp với nhau. Khi phát triển hoặc bảo trì hệ thống, ta thường gặp các tình huống như:

- Ứng dụng đang làm việc thông qua một interface chuẩn, ví dụ `IPaymentGateway`, `INotificationSender`, `IJsonParser`, nhưng thư viện bên thứ ba lại cung cấp class với tên hàm, kiểu dữ liệu hoặc quy ước gọi hàm khác.
- Hệ thống cũ trả dữ liệu ở dạng XML, trong khi module mới chỉ nhận và xử lý JSON.
- Client code đã ổn định, đã được kiểm thử, nhưng cần tích hợp thêm một service mới không tuân theo interface hiện tại.
- Một ứng dụng cần kết nối nhiều nhà cung cấp khác nhau, chẳng hạn nhiều cổng thanh toán, nhiều dịch vụ gửi email/SMS, nhiều API vận chuyển; mỗi bên lại đặt tên hàm và định dạng dữ liệu khác nhau.

- Một class cũ vẫn còn giá trị sử dụng, nhưng không thể sửa trực tiếp vì nó thuộc thư viện bên ngoài, hệ thống legacy, hoặc việc sửa có thể gây ảnh hưởng dây chuyền.

Vấn đề cốt lõi là: **làm sao sử dụng được một object có interface không khớp với interface mà client đang mong đợi, nhưng không phải sửa client code và cũng không phải sửa class cũ?**

Nếu giải quyết bằng cách sửa trực tiếp client để gọi class mới, client sẽ phụ thuộc vào chi tiết cụ thể của class đó. Khi có thêm một thư viện khác, client lại phải sửa tiếp. Cách làm này làm tăng coupling, làm giảm khả năng mở rộng và khiến code khó bảo trì.

Adapter Pattern đưa ra một hướng tiếp cận an toàn hơn: giữ nguyên client, giữ nguyên class có sẵn, và đặt một lớp trung gian ở giữa để chuyển đổi lời gọi.

2.1.3. Định nghĩa

Adapter Pattern là một design pattern cho phép các object có interface không tương thích có thể làm việc với nhau bằng cách đặt một lớp trung gian gọi là Adapter. Lớp này cung cấp interface mà client mong muốn, đồng thời bên trong nó chuyển đổi lời gọi sang interface thật sự của object được tái sử dụng, gọi là Adaptee.

Có thể mô tả ngắn gọn như sau:

- Client chỉ biết và chỉ phụ thuộc vào Target interface.
- Adaptee là class có sẵn, có chức năng cần dùng nhưng interface không phù hợp.
- Adapter implement Target, bên trong giữ hoặc kế thừa Adaptee, sau đó chuyển lời gọi từ Target sang lời gọi phù hợp với Adaptee.

Nhờ vậy, client có thể làm việc với Adaptee một cách gián tiếp mà không cần biết Adaptee thật sự hoạt động ra sao.

2.1.4. Ý tưởng cốt lõi

Ý tưởng chính của Adapter là **bọc một object có interface không tương thích bên trong một object mới**, rồi object mới đó cung cấp interface mà client mong muốn.

Adapter thường thực hiện ba việc quan trọng:

1. Nhận request từ client theo interface chuẩn (Target).
2. Chuyển đổi request đó sang dạng mà object cũ hoặc service bên ngoài hiểu được.
3. Gọi phương thức thật sự của Adaptee, sau đó chuyển đổi kết quả trả về nếu cần.

Có thể hình dung Adapter giống như một bộ chuyển đổi ổ cắm điện. Thiết bị điện không cần thay đổi chân cắm, ổ điện trên tường cũng không cần thay đổi cấu trúc. Bộ chuyển đổi đứng ở giữa, nhận một chuẩn đầu vào và biến nó thành chuẩn đầu ra phù hợp.

Trong phần mềm, Adapter cũng đóng vai trò tương tự: nó không làm thay đổi bản chất nghiệp vụ của Adaptee, mà chỉ giúp Adaptee trở nên tương thích với phần còn lại của hệ thống.

2.1.5. Thành phần chính

Một cấu trúc Adapter Pattern điển hình gồm bốn thành phần:

2.1.6. Client

Client là phần code sử dụng chức năng thông qua interface chuẩn. Client không nên biết class cụ thể nào đang xử lý phía sau. Nó chỉ gọi phương thức được định nghĩa trong Target.

Ví dụ, trong một hệ thống thanh toán, client chỉ gọi:

```
paymentGateway.Pay(amount);
```

Client không cần biết phía sau là Stripe, PayPal, MoMo hay một cổng thanh toán nội bộ.

2.1.7. Target

Target là interface mà client mong muốn sử dụng. Đây là hợp đồng chung giữa client và các implementation.

Ví dụ:

```
public interface IPaymentGateway
{
    PaymentResult Pay(decimal amount, string currency);
}
```

Mọi class muốn được client sử dụng như một cổng thanh toán đều cần tuân theo interface này.

2.1.8. Adaptee

Adaptee là class có sẵn, có chức năng thật sự cần dùng, nhưng interface của nó không tương thích với Target.

Ví dụ, một thư viện thanh toán bên thứ ba có thể cung cấp class như sau:

```
public class StripeService
{
    public StripeResponse Charge(long amountInCents, string currencyCode)
    {
        // Gọi API thật sự của Stripe
    }
}
```

Ở đây, StripeService có thể thanh toán được, nhưng nó không có hàm Pay(decimal amount, string currency). Nó dùng Charge(long amountInCents, string currencyCode), nên client không thể dùng trực tiếp theo interface IPaymentGateway.

2.1.9. Adapter

Adapter là class trung gian. Nó implement Target, giữ tham chiếu đến Adaptee, và chuyển lời gọi từ client sang lời gọi mà Adaptee hiểu được.

Ví dụ:

```

public class StripePaymentAdapter : IPaymentGateway
{
    private readonly StripeService _stripeService;

    public StripePaymentAdapter(StripeService stripeService)
    {
        _stripeService = stripeService;
    }

    public PaymentResult Pay(decimal amount, string currency)
    {
        long amountInCents = (long)(amount * 100);

        StripeResponse response = _stripeService.Charge(amountInCents,
currency);

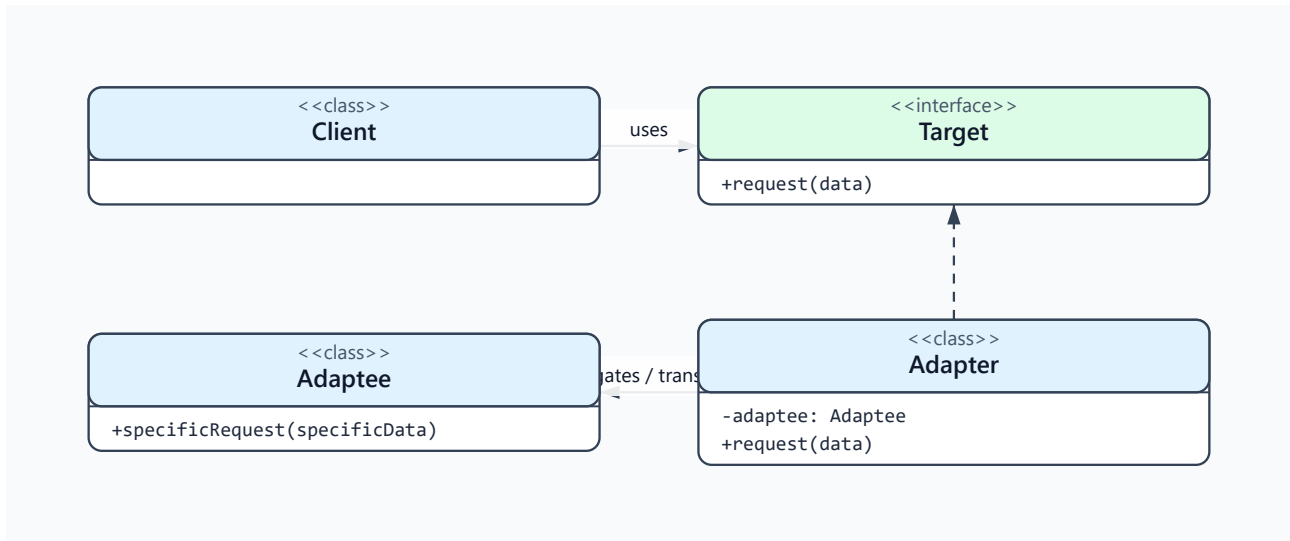
        return new PaymentResult(
            success: response.IsSuccess,
            transactionId: response.TransactionId,
            message: response.Message
        );
    }
}

```

Client vẫn chỉ thấy IPaymentGateway, còn chi tiết chuyển đổi sang StripeService được giấu bên trong adapter.

2.1.10. Cấu trúc UML

Dưới đây là class diagram tổng quát bằng PlantUML:



Hình 2.11: Cấu trúc UML tổng quát của Adapter Pattern

Trong sơ đồ này:

- Client chỉ phụ thuộc vào Target.
- Adapter hiện thực Target, nên có thể được truyền vào client như một implementation bình thường.
- Adapter giữ tham chiếu đến Adaptee và gọi phương thức thật của Adaptee.
- Phần chuyển đổi dữ liệu, đổi tên hàm, đổi kiểu trả về hoặc xử lý mapping nằm trong Adapter.

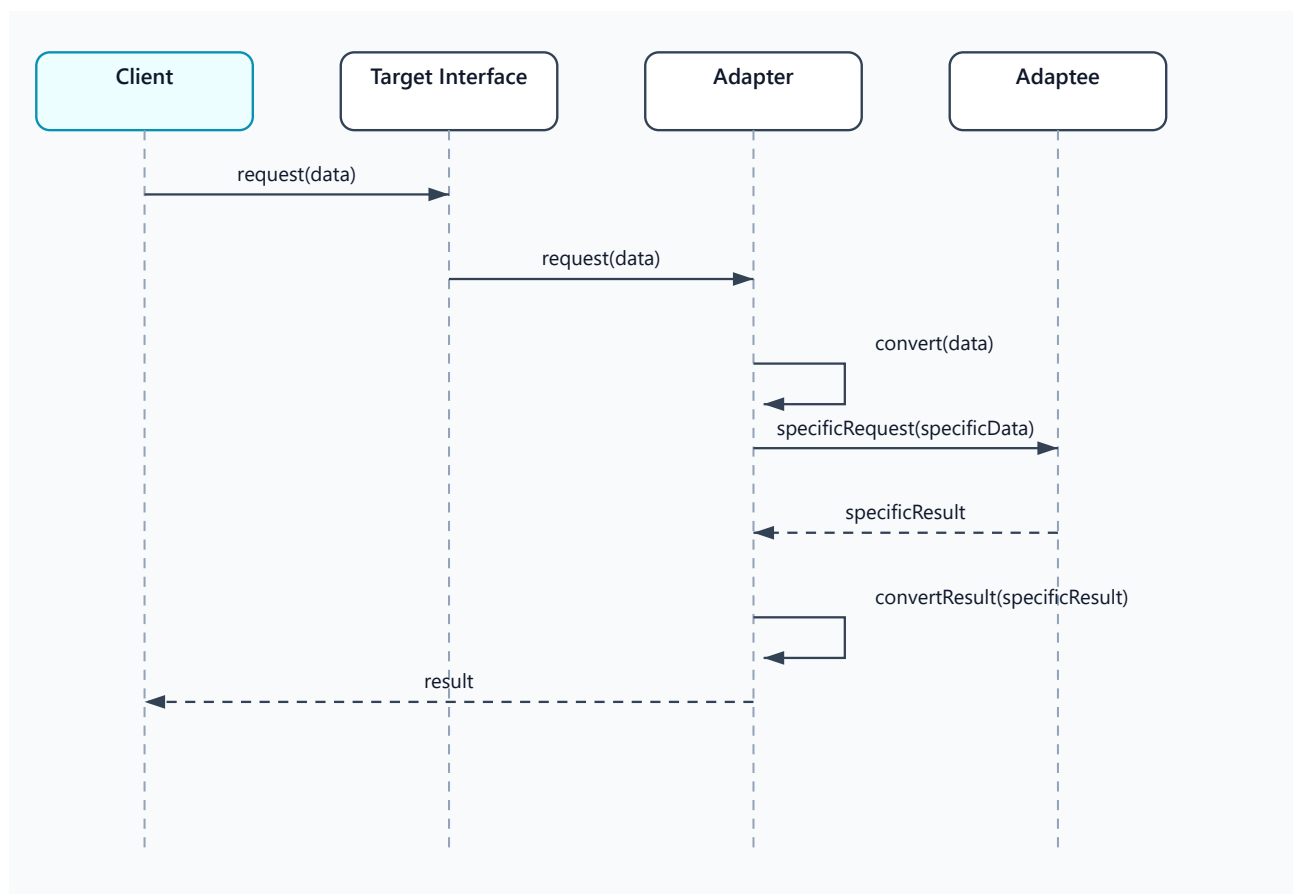
2.1.11. Luồng hoạt động

Luồng hoạt động của Adapter Pattern có thể mô tả theo các bước sau:

1. Client gọi một phương thức thông qua interface Target.
2. Vì object thật được truyền vào client là Adapter, lời gọi đi vào Adapter.

3. Adapter kiểm tra, chuyển đổi hoặc mapping dữ liệu đầu vào sang định dạng mà Adaptee hiểu được.
4. Adapter gọi phương thức thật sự của Adaptee.
5. Adaptee xử lý logic hoặc gọi service bên ngoài.
6. Nếu Adaptee trả về kết quả theo định dạng riêng, Adapter tiếp tục chuyển đổi kết quả đó về định dạng mà Target quy định.
7. Client nhận kết quả theo đúng interface quen thuộc, không cần biết có sự tồn tại của Adaptee phía sau.

Sequence diagram minh họa:



Hình 2.12: Luồng hoạt động giữa Client, Adapter và Adaptee

2.1.12. Phân loại Adapter

Adapter thường có hai biến thể chính: **object adapter** và **class adapter**.

2.1.13. Object Adapter

Object Adapter dùng composition hoặc delegation. Adapter giữ một instance của Adaptee ở bên trong, sau đó ủy quyền lời gọi cho instance đó.

Đây là cách phổ biến trong C#, Java và nhiều ngôn ngữ hướng đối tượng hiện đại vì nó linh hoạt và không phụ thuộc vào đa kế thừa class.

Ưu điểm:

- Linh hoạt vì có thể thay đổi Adaptee lúc runtime.
- Có thể adapter cho nhiều subclass của Adaptee.
- Phù hợp với nguyên tắc ưu tiên composition hơn inheritance.

Nhược điểm:

- Cần viết thêm code chuyển tiếp lời gọi.
- Nếu có nhiều phương thức cần chuyển đổi, adapter có thể dài.

2.1.14. Class Adapter

Class Adapter dùng inheritance. Adapter kế thừa từ Adaptee và đồng thời implement Target.

Cách này chỉ phù hợp với ngôn ngữ hỗ trợ đa kế thừa class hoặc trong trường hợp cấu trúc kế thừa cho phép. Trong C#, class adapter bị hạn chế vì C# không hỗ trợ đa kế thừa class.

Ưu điểm:

- Có thể override hành vi của Adaptee nếu cần.
- Không cần giữ tham chiếu riêng đến Adaptee.

Nhược điểm:

- Kém linh hoạt hơn object adapter.
- Phụ thuộc chặt vào class cụ thể.

- Không phù hợp nếu cần adapter nhiều subclass khác nhau.

2.1.15. Ví dụ minh họa bằng C#

Giả sử hệ thống hiện tại chỉ làm việc với interface `IPaymentGateway`. Sau này hệ thống cần tích hợp Stripe, nhưng thư viện Stripe lại có interface khác.

2.1.16. Target

```
public interface IPaymentGateway
```

```
{  
    PaymentResult Pay(decimal amount, string currency);  
}
```

```
public class PaymentResult
```

```
{  
    public bool Success { get; }  
    public string TransactionId { get; }  
    public string Message { get; }
```

```
    public PaymentResult(bool success, string transactionId, string message)
```

```
{  
    Success = success;  
    TransactionId = transactionId;  
    Message = message;  
}
```

```
}
```

2.1.17. Adaptee

```
public class StripeService
{
    public StripeResponse Charge(long amountInCents, string currencyCode)
    {
        Console.WriteLine($"Charging {amountInCents} cents via Stripe...");

        return new StripeResponse(
            isSuccess: true,
            transactionId: Guid.NewGuid().ToString(),
            message: "Stripe payment completed"
        );
    }
}

public class StripeResponse
{
    public bool IsSuccess { get; }
    public string TransactionId { get; }
    public string Message { get; }

    public StripeResponse(bool isSuccess, string transactionId, string
message)
```

```

{
    IsSuccess = isSuccess;

    TransactionId = transactionId;

    Message = message;
}
}

```

2.1.18. Adapter

```

public class StripePaymentAdapter : IPaymentGateway
{
    private readonly StripeService _stripeService;

    public StripePaymentAdapter(StripeService stripeService)
    {
        _stripeService = stripeService;
    }

    public PaymentResult Pay(decimal amount, string currency)
    {
        long amountInCents = ConvertToCents(amount);

        StripeResponse stripeResponse = _stripeService.Charge(amountInCents,
currency);

        return new PaymentResult(

```

```

        success: stripeResponse.IsSuccess,
        transactionId: stripeResponse.TransactionId,
        message: stripeResponse.Message
    );
}

```

```

private static long ConvertToCents(decimal amount)
{
    return (long)(amount * 100);
}
}

```

2.1.19. Client

```

public class CheckoutService
{
    private readonly IPaymentGateway _paymentGateway;

    public CheckoutService(IPaymentGateway paymentGateway)
    {
        _paymentGateway = paymentGateway;
    }

    public void Checkout(decimal totalAmount)
    {

```

```

        PaymentResult result = _paymentGateway.Pay(totalAmount, "USD");

        if (result.Success)
        {
            Console.WriteLine($"Payment success: {result.TransactionId}");
        }
        else
        {
            Console.WriteLine($"Payment failed: {result.Message}");
        }
    }
}

```

2.1.20. Chạy thử

```

var stripeService = new StripeService();

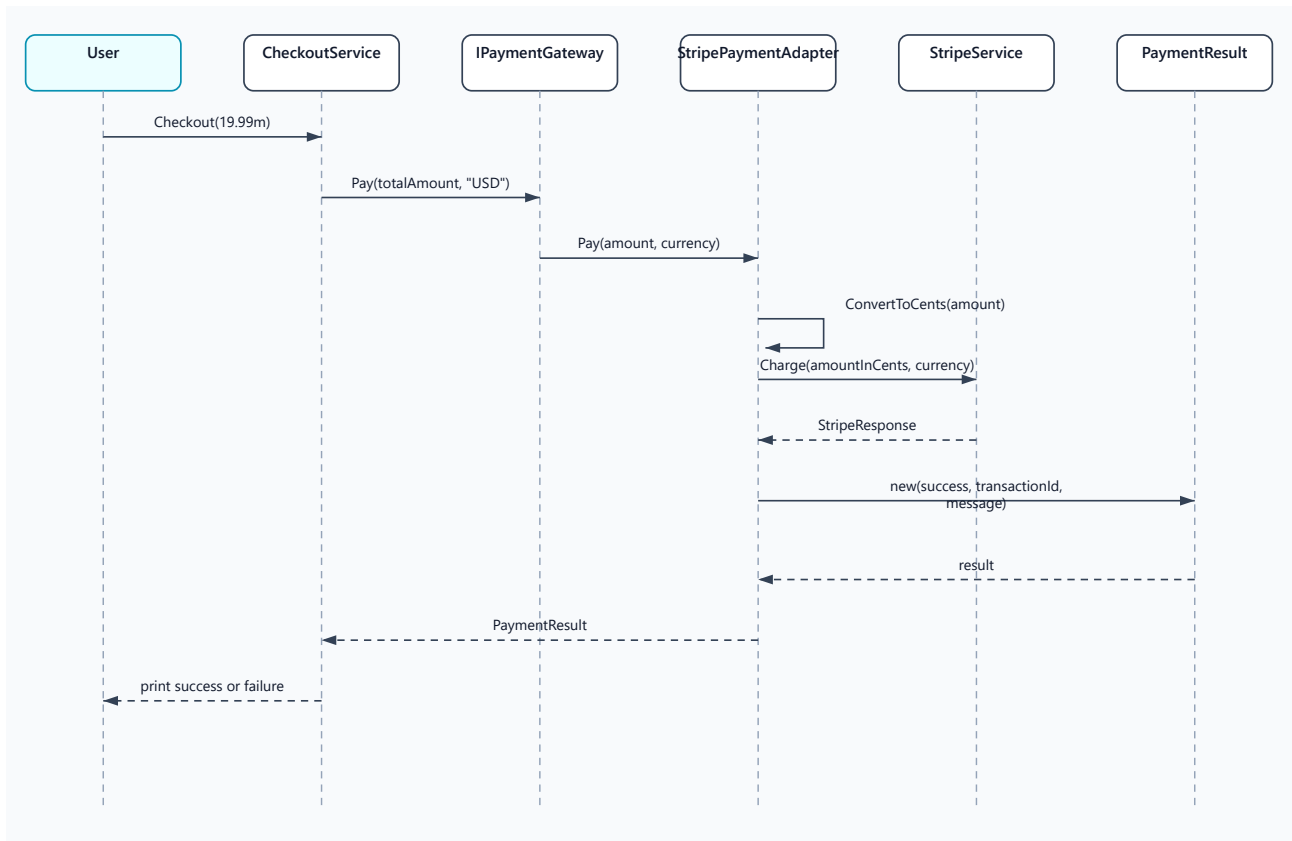
IPaymentGateway gateway = new StripePaymentAdapter(stripeService);

var checkoutService = new CheckoutService(gateway);

checkoutService.Checkout(19.99m);

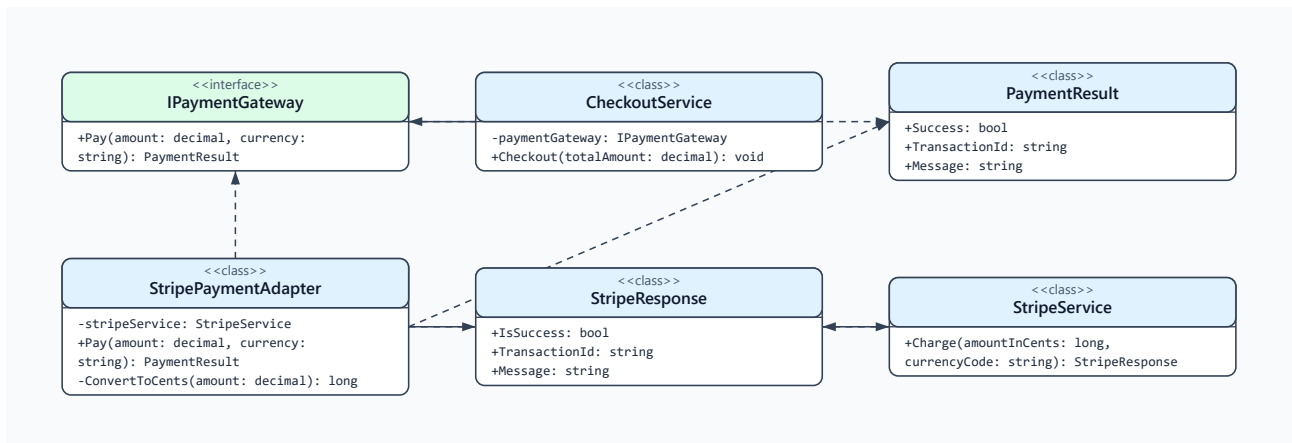
```

Điểm quan trọng là CheckoutService không phụ thuộc vào StripeService. Nếu sau này thay Stripe bằng PayPal hoặc MoMo, ta chỉ cần tạo adapter mới implement IPaymentGateway, client không cần sửa.



Hình 2.13: Luồng chạy code ví dụ CheckoutService và StripePaymentAdapter

2.1.21. UML cho ví dụ thanh toán



Hình 2.14: Class diagram của ví dụ StripePaymentAdapter

2.1.22. Khả năng ứng dụng

Nên cân nhắc dùng Adapter Pattern khi:

- Muốn tái sử dụng một class có sẵn nhưng interface của class đó không khớp với interface hệ thống đang dùng.
- Muốn tích hợp thư viện bên thứ ba mà không làm client code phụ thuộc trực tiếp vào thư viện đó.
- Muốn che giấu sự khác biệt giữa nhiều vendor/service khác nhau bằng một interface thống nhất.
- Muốn giữ nguyên code cũ đã ổn định và chỉ thêm lớp chuyển đổi ở bên ngoài.
- Muốn áp dụng Open/Closed Principle: thêm adapter mới thay vì sửa client cũ.
- Cần chuyển đổi định dạng dữ liệu giữa các module, ví dụ XML sang JSON, DTO bên ngoài sang domain model nội bộ, hoặc response của API bên thứ ba sang response chuẩn của hệ thống.

Không nên lạm dụng Adapter nếu:

- Hai interface đã tương thích hoặc chỉ khác rất nhỏ và có thể thống nhất ngay từ thiết kế ban đầu.
- Adapter phải chứa quá nhiều logic nghiệp vụ, khiến nó không còn là lớp chuyển đổi mà trở thành một service phức tạp.
- Sự khác biệt giữa hai hệ thống không chỉ là interface mà còn là mô hình nghiệp vụ hoàn toàn khác nhau. Khi đó cần thiết kế lại boundary rõ hơn, có thể dùng Anti-Corruption Layer thay vì một adapter đơn giản.

2.1.23. Đánh giá

2.1.24. Ưu điểm

- **Giảm phụ thuộc giữa client và class bên ngoài:** client chỉ phụ thuộc vào Target, không phụ thuộc trực tiếp vào Adaptee.

- **Tái sử dụng được class cũ:** có thể dùng lại thư viện hoặc module legacy mà không cần sửa code gốc.
- **Tuân thủ Open/Closed Principle:** muốn hỗ trợ thêm service mới thì thêm adapter mới, hạn chế sửa code client.
- **Tách biệt logic chuyển đổi:** mọi mapping về tên hàm, kiểu dữ liệu, định dạng request/response được gom vào adapter.
- **Hỗ trợ kiểm thử tốt hơn:** client có thể được test bằng mock của Target, không cần khởi tạo service thật bên ngoài.
- **Giảm rủi ro khi tích hợp:** thay vì rải logic gọi API bên thứ ba khắp hệ thống, ta cô lập nó trong adapter.

2.1.25. Nhược điểm

- **Tăng số lượng class:** mỗi interface hoặc service không tương thích thường cần thêm một adapter riêng.
- **Có thể làm kiến trúc rườm rà:** nếu dùng adapter cho những khác biệt quá nhỏ, code có thể bị over-engineering.
- **Adapter có thể phình to:** nếu Adaptee quá khác biệt, adapter phải xử lý nhiều mapping phức tạp.
- **Không giải quyết khác biệt nghiệp vụ sâu:** Adapter chủ yếu xử lý khác biệt interface. Nếu hai hệ thống có quy tắc nghiệp vụ mâu thuẫn, cần giải pháp thiết kế rộng hơn.
- **Có thể che giấu lỗi thiết kế:** nếu trong cùng một hệ thống nội bộ mà phải tạo quá nhiều adapter, đó có thể là dấu hiệu các module đang thiếu chuẩn interface chung.

2.1.26. Lưu ý khi cài đặt

Khi triển khai Adapter Pattern, cần lưu ý một số điểm sau:

- Nên để client phụ thuộc vào interface Target, không phụ thuộc trực tiếp vào adapter cụ thể.
- Adapter chỉ nên chịu trách nhiệm chuyển đổi interface và dữ liệu, không nên chứa nhiều business rule.
- Nếu tích hợp API bên thứ ba, nên gom toàn bộ logic chuyển đổi request/response, exception, mã lỗi vào adapter để tránh rò rỉ chi tiết bên ngoài vào domain nội bộ.
- Với C#, Java, TypeScript, object adapter thường linh hoạt hơn class adapter.
- Nên đặt tên adapter theo ý nghĩa chuyển đổi, ví dụ StripePaymentAdapter, LegacyUserAdapter, XmlOrderAdapter.
- Nếu có nhiều adapter cho cùng một interface, có thể kết hợp với Dependency Injection để chọn implementation phù hợp.
- Nếu cần bọc cả một subsystem phức tạp, cần cân nhắc Facade thay vì Adapter.

2.1.27. So sánh với các pattern liên quan

2.1.28. Adapter và Facade

Tiêu chí	Adapter	Facade
Mục đích chính	Chuyển đổi interface không tương thích thành interface client mong muốn.	Đơn giản hóa cách sử dụng một hệ thống con phức tạp.
Vấn đề giải quyết	Client không dùng được class có sẵn vì khác interface.	Client không muốn làm việc trực tiếp với nhiều class hoặc nhiều bước phức tạp.
Có thay đổi interface không?	Có. Adapter biến interface của Adaptee thành Target.	Có thể có, nhưng mục tiêu chính là gom và đơn giản hóa thao tác.

Thường bọc	Một class hoặc một service không tương thích.	Nhiều class hoặc cả một subsystem.
Ví dụ	StripePaymentAdapter biến StripeService thành IPaymentGateway.	OrderFacade gọi Inventory, Payment, Shipping để hoàn tất đơn hàng.

Tóm lại, Adapter tập trung vào **tương thích interface**, còn Facade tập trung vào **đơn giản hóa cách sử dụng hệ thống phức tạp**.

2.1.29. Adapter và Decorator

Tiêu chí	Adapter	Decorator
Mục đích chính	Làm interface không tương thích trở nên tương thích.	Thêm chức năng mới cho object mà không sửa object gốc.
Interface sau khi bọc	Thường khác với interface của Adaptee.	Thường giống interface của object gốc.
Có thay đổi hành vi không?	Có thể có chuyển đổi lời gọi và dữ liệu.	Có, bằng cách bổ sung hành vi trước/sau khi gọi object gốc.
Tập trung vào	Tương thích interface.	Mở rộng chức năng.
Ví dụ	StripePaymentAdapter biến StripeService thành IPaymentGateway.	LoggingPaymentGateway thêm logging trước/sau khi thanh toán.

Decorator thường giữ nguyên interface để client vẫn dùng object như cũ nhưng có thêm chức năng. Adapter thường đổi interface để client có thể dùng được object vốn không tương thích.

2.1.30. Adapter và Bridge

Tiêu chí	Adapter	Bridge
Nhóm pattern	Structural.	Structural.

Mục đích	Kết nối các interface không tương thích.	Tách abstraction khỏi implementation để cả hai phát triển độc lập.
Thời điểm dùng	Thường dùng sau khi đã có class hoặc service không tương thích.	Thường được thiết kế từ đầu để hệ thống dễ mở rộng.
Quan hệ chính	Adapter bọc hoặc giữ tham chiếu đến Adaptee.	Abstraction giữ tham chiếu đến Implementor.
Vấn đề chính	“Tôi có class này nhưng interface không khớp.”	“Tôi muốn abstraction và implementation thay đổi độc lập.”

Adapter thường mang tính chữa cháy hoặc tích hợp, còn Bridge thường là một quyết định thiết kế có chủ đích từ sớm.

2.1.31. Adapter và Proxy

Tiêu chí	Adapter	Proxy
Mục đích chính	Chuyển đổi interface.	Kiểm soát truy cập đến object thật.
Interface	Thường khác interface của object được bọc.	Thường giống interface của object thật.
Vai trò	Làm cho object không tương thích dùng được.	Thêm lớp đại diện để lazy loading, caching, authorization, remote access.
Ví dụ	Bọc API thanh toán bên ngoài thành IPaymentGateway.	Bọc service thật để kiểm tra quyền trước khi gọi.

Proxy không nhằm mục tiêu đổi interface. Nó thường giữ nguyên interface và kiểm soát việc truy cập đến object thật.

2.1.32. Ví dụ trong hệ thống thực tế

Adapter Pattern xuất hiện rất nhiều trong các hệ thống thực tế:

- **Database driver:** ứng dụng làm việc thông qua interface chung, còn driver cụ thể chuyển lời gọi sang MySQL, PostgreSQL, SQL Server hoặc Oracle.
- **Cổng thanh toán:** hệ thống định nghĩa IPaymentGateway, sau đó tạo adapter cho Stripe, PayPal, MoMo, VNPay hoặc ZaloPay.
- **Dịch vụ gửi thông báo:** client gọi INotificationSender.Send(), adapter chuyển sang API riêng của email provider, SMS provider hoặc push notification provider.
- **Chuyển đổi dữ liệu:** adapter chuyển XML từ hệ thống cũ thành DTO hoặc JSON cho module mới.
- **Tích hợp legacy system:** một hệ thống mới có thể gọi chức năng cũ thông qua adapter thay vì sửa toàn bộ legacy code.
- **Phản cứng và thiết bị:** driver hoặc lớp giao tiếp thiết bị cũng có thể đóng vai trò adapter giữa hệ điều hành và phần cứng.

2.1.33. Kết luận

Adapter Pattern là một mẫu thiết kế quan trọng khi hệ thống cần tích hợp các thành phần không tương thích nhưng vẫn muốn giữ client code ổn định. Thay vì sửa client hoặc sửa class cũ, ta tạo một lớp adapter để chuyển đổi interface, dữ liệu và kết quả trả về.

Pattern này đặc biệt hữu ích trong các tình huống tích hợp thư viện bên thứ ba, hệ thống legacy, nhiều vendor hoặc nhiều API bên ngoài. Tuy nhiên, cần dùng đúng mục đích: Adapter nên là

lớp chuyển đổi mỏng, tập trung vào tương thích interface, không nên trở thành nơi chứa quá nhiều business logic.

Có thể ghi nhớ Adapter bằng câu ngắn gọn:

Adapter giúp một class có interface không phù hợp trở nên dùng được trong hệ thống hiện tại mà không cần sửa client và không cần sửa class gốc.

2.2. Bridge (Đạt)

2.2.1. Tên và phân loại

Bridge Pattern trong tiếng Việt thường được gọi là **mẫu Cầu nối**.

Bridge thuộc nhóm **Structural Pattern** vì nó tập trung vào cách tổ chức cấu trúc lớp/đối tượng. Ý tưởng chính là tách phần trừu tượng (abstraction) khỏi phần triển khai chi tiết (implementation), sau đó nối hai phần bằng composition.

Tên gọi “Bridge” thể hiện đúng bản chất: tạo một chiếc cầu giữa hai hệ phân cấp độc lập để cả hai có thể phát triển riêng mà vẫn phối hợp được.

2.2.2. Vấn đề cần giải quyết

Trong nhiều hệ thống, một tính năng có ít nhất hai chiều thay đổi độc lập.

Ví dụ với hệ thống thông báo:

- Chiều 1: loại thông báo (BasicNotification, UrgentNotification, DigestNotification, ...).
- Chiều 2: kênh gửi (EmailSender, SmsSender, PushSender, ...).

Nếu không dùng Bridge, ta thường tạo class cho mọi tổ hợp:

BasicEmailNotification

BasicSmsNotification

UrgentEmailNotification

UrgentSmsNotification

...

Khi có m loại thông báo và n kênh gửi, số class có thể tăng gần $m * n$.

Cách làm này dẫn tới:

- Số lượng class bùng nổ khi mở rộng.
- Logic bị lặp lại ở nhiều class tổ hợp.
- Thêm biến thể mới ở một chiều phải sửa nhiều nơi.
- Coupling cao, bảo trì và test khó hơn.

Vấn đề cốt lõi là: **làm sao tách hai chiều thay đổi độc lập để mở rộng linh hoạt mà không nhân chéo số lượng class?**

2.2.3. Mục đích và ý định

Mục đích chính của Bridge là **decouple abstraction khỏi implementation để chúng có thể thay đổi độc lập**.

Nói ngắn gọn:

- Abstraction mô tả “muốn làm gì”.
- Implementor mô tả “làm bằng cách nào”.
- Abstraction ủy quyền phân chi tiết cho implementor thông qua một interface chung.

2.2.4. Định nghĩa

Bridge Pattern là một structural design pattern tách abstraction và implementation thành hai hệ phân cấp riêng biệt, cho phép cả hai biến thiên độc lập và kết hợp linh hoạt thông qua object composition.

Các thành phần ở mức định nghĩa:

- Abstraction: giao diện mức cao mà client sử dụng.
- RefinedAbstraction: các biến thể cụ thể của abstraction.

- Implementor: hợp đồng cho phân cài đặt chi tiết.
- ConcreteImplementor: các cài đặt cụ thể.

2.2.5. Ý tưởng cốt lõi

Thay vì kế thừa chéo theo nhiều tổ hợp, Bridge tách thành hai cây class:

Abstraction tree: Notification -> Basic/Urgent/...

Implementation tree: INotificationSender -> Email/SMS/...

Mỗi object ở phía abstraction giữ tham chiếu đến implementor, rồi gọi implementor khi cần thao tác chi tiết.

Có thể hình dung luồng dùng như sau:

Client chọn loại Notification

+

Client chọn kênh Sender

-> Inject sender vào notification

-> Notification gọi sender để gửi

2.2.6. Thành phần chính

2.2.7. Abstraction

Abstraction định nghĩa thao tác mức cao và giữ tham chiếu đến Implementor.

Trong demo Bridge hiện tại, lớp này là Notification.

2.2.8. RefinedAbstraction

RefinedAbstraction mở rộng hoặc tinh chỉnh hành vi của abstraction.

Ví dụ:

- BasicNotification: gửi nguyên văn nội dung.
- UrgentNotification: thêm tiền tố [URGENT] trước khi gửi.

2.2.9. Implementor

Implementor là interface định nghĩa các thao tác chi tiết.

Ví dụ: INotificationSender với phương thức Send(string message).

2.2.10. ConcreteImplementor

Các cài đặt cụ thể của Implementor.

Ví dụ:

- EmailSender
- SmsSender

2.2.11. Client

Client kết hợp abstraction và implementor theo nhu cầu runtime.

```
Notification notification = new UrgentNotification(new SmsSender());
```

```
notification.Send("Server is down!");
```

2.2.12. Cấu trúc UML

Class diagram tổng quát bằng PlantUML:

```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
class Client
```

```
abstract class Abstraction {
```

```
    -implementor: Implementor
```

```
    +operation()
```

```
}
```

```
class RefinedAbstractionA {  
    +operation()  
}
```

```
class RefinedAbstractionB {  
    +operation()  
}
```

```
interface Implementor {  
    +operationImpl()  
}
```

```
class ConcreteImplementorA {  
    +operationImpl()  
}
```

```
class ConcreteImplementorB {  
    +operationImpl()  
}
```

Client --> Abstraction

Abstraction o--> Implementor

RefinedAbstractionA --|> Abstraction

```

RefinedAbstractionB --|> Abstraction
ConcreteImplementorA ..|> Implementor
ConcreteImplementorB ..|> Implementor

@enduml

```

2.2.13. Luồng hoạt động

Luồng cơ bản của Bridge:

1. Client tạo một ConcreteImplementor.
2. Client inject implementor vào constructor của RefinedAbstraction.
3. Client gọi operation trên abstraction.
4. Abstraction xử lý logic mức cao.
5. Abstraction ủy quyền phần thực thi chi tiết cho implementor.
6. Implementor thực thi thao tác cụ thể.

Sequence diagram minh họa:

```

@startuml
actor Client
participant "UrgentNotification" as N
participant "SmsSender" as S

Client -> N : Send("Server is down!")
N -> N : prepend "[URGENT]"
N -> S : Send("[URGENT] Server is down!")
S -> S : output SMS

```

S --> Client : done

@enduml

2.2.14. Các biến thể triển khai

2.2.15. Bridge cơ bản

Abstraction gọi implementor theo một chiều rõ ràng. Đây là dạng phổ biến nhất.

2.2.16. Bridge hai chiều mở rộng mạnh

Cả abstraction lẫn implementor đều có nhiều nhánh con và cùng phát triển độc lập.

Ví dụ:

- Abstraction: Basic, Urgent, Scheduled.
- Implementor: Email, SMS, Push.

2.2.17. Chọn implementor theo runtime

Implementor có thể được cấu hình theo môi trường hoặc theo cấu hình người dùng:

- Development: ConsoleSender.
- Production: EmailSender/SmsSender.

2.2.18. Ví dụ minh họa bằng C#

Ví dụ sau đồng bộ với code trong BridgeDemo của nhóm.

2.2.19. Implementor

```
public interface INotificationSender
```

```
{  
  
    void Send(string message);  
  
}
```

```
public class EmailSender : INotificationSender
```

```
{  
  
    public void Send(string message)
```

```

    {
        Console.WriteLine($"[Email] Sending: {message}");
    }
}

```

```

public class SmsSender : INotificationSender
{
    public void Send(string message)
    {
        Console.WriteLine($"[SMS] Sending: {message}");
    }
}

```

2.2.20. Abstraction và RefinedAbstraction

```

public abstract class Notification
{
    protected INotificationSender sender;

    protected Notification(INotificationSender sender)
    {
        this.sender = sender;
    }

    public abstract void Send(string message);
}

```

```
}
```

```
public class BasicNotification : Notification
{
    public BasicNotification(INotificationSender sender)
        : base(sender) { }

    public override void Send(string message)
    {
        sender.Send(message);
    }
}
```

```
public class UrgentNotification : Notification
{
    public UrgentNotification(INotificationSender sender)
        : base(sender) { }

    public override void Send(string message)
    {
        sender.Send("[URGENT] " + message);
    }
}
```

2.2.21. Client sử dụng Bridge

```
Notification notification1 = new BasicNotification(new EmailSender());  
  
notification1.Send("Hello World");
```

```
Console.WriteLine();
```

```
Notification notification2 = new UrgentNotification(new SmsSender());  
  
notification2.Send("Server is down!");
```

```
Console.WriteLine();
```

```
Notification notification3 = new UrgentNotification(new EmailSender());  
  
notification3.Send("Database error!");
```

Ở ví dụ này:

- Loại notification và kênh gửi được chọn độc lập.
- Có thể mix linh hoạt các tổ hợp ngay tại runtime.
- Việc mở rộng hai phía không phụ thuộc chặt vào nhau.

2.2.22. Class diagram cho ví dụ C#

```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
abstract class Notification {  
  
    #sender: INotificationSender  
  
    +Send(message: string)
```

```
}
```

```
class BasicNotification {  
    +Send(message: string)  
}
```

```
class UrgentNotification {  
    +Send(message: string)  
}
```

```
interface INotificationSender {  
    +Send(message: string)  
}
```

```
class EmailSender {  
    +Send(message: string)  
}
```

```
class SmsSender {  
    +Send(message: string)  
}
```

```
Notification <|-- BasicNotification
```



```

Notification <|-- UrgentNotification

Notification o--> INotificationSender

INotificationSender <|.. EmailSender

INotificationSender <|.. SmsSender

@enduml

```

2.2.23. Đánh giá

2.2.24. Ưu điểm

Giảm class explosion. Tránh tạo class cho mọi tổ hợp biến thể.

Loose coupling. Abstraction chỉ phụ thuộc vào interface implementor.

Dễ mở rộng. Thêm biến thể mới ở từng phía với ít tác động.

Tuân thủ SRP/OCP tốt hơn. Trách nhiệm tách rõ và dễ mở rộng không phá code cũ.

Linh hoạt runtime. Có thể thay implementor theo cấu hình hoặc ngữ cảnh.

2.2.25. Nhược điểm

Tăng số lớp và mức trừu tượng. Với bài toán nhỏ có thể gây over-engineering.

Khó nắm bắt với người mới. Cần hiểu rõ vai trò từng thành phần để tránh thiết kế sai.

Thêm tầng gián tiếp. Có thể làm trace luồng gọi phức tạp hơn.

2.2.26. Khi nào nên dùng

Nên dùng Bridge khi:

- Có từ hai chiều thay đổi độc lập.
- Dự đoán còn mở rộng ở cả hai chiều trong tương lai.
- Muốn tránh inheritance tổ hợp gây bùng nổ class.
- Cần hoán đổi implementation ở runtime.

Ví dụ phù hợp:

- Notification type x Delivery channel.

- Shape x Render engine.
- RemoteControl x Device.
- Report x Export backend.

2.2.27. Khi nào không nên dùng

Không nên dùng Bridge khi:

- Bài toán chỉ có một chiều thay đổi rõ ràng.
- Số biến thể ít và ổn định lâu dài.
- Yêu cầu ưu tiên tối đa sự đơn giản hơn tính mở rộng.

2.2.28. Lưu ý khi cài đặt

- Xác định đúng hai chiều biến thiên trước khi tách Bridge.
- Ưu tiên composition thay vì kế thừa theo tổ hợp.
- Giữ interface implementor gọn, tập trung primitive operation.
- Không để abstraction phụ thuộc concrete implementor.
- Có thể kết hợp Dependency Injection để quản lý implementor.
- Nên có test riêng cho refined abstraction và từng concrete implementor.

2.2.29. Ví dụ thực tế

2.2.30. UI toolkit đa nền tảng

Abstraction là widget (Button, TextBox), implementation là backend render theo nền tảng (Win32, X11, Cocoa).

2.2.31. Device driver

Abstraction là lệnh nghiệp vụ cấp cao, implementation là driver cụ thể của từng hãng.

2.2.32. Logging platform

Abstraction là logger mức cao, implementation là đích ghi (Console, File, Elastic, Cloud).

2.2.33. Notification platform

Abstraction là loại thông báo, implementation là provider gửi (SMTP, SMS Gateway, Push Service).

2.2.34. So sánh với các pattern khác

2.2.35. Bridge và Adapter

Tiêu chí	Bridge	Adapter
Mục đích	Tách abstraction và implementation	Chuyển đổi interface không tương thích
Thời điểm dùng	Thiết kế từ đầu để mở rộng	Tích hợp lớp/hệ thống có sẵn
Trọng tâm	Khả năng mở rộng cấu trúc	Khả năng tương thích

2.2.36. Bridge và Strategy

Tiêu chí	Bridge	Strategy
Mục đích	Tách hai hierarchy độc lập	Thay đổi thuật toán
Số chiều	Thường 2 chiều	Thường 1 chiều
Vai trò	Kiến trúc cấu trúc lớp	Chiến lược hành vi

2.2.37. Bridge và Abstract Factory

Abstract Factory có thể phối hợp với Bridge để tạo đúng cặp abstraction + implementor theo từng môi trường hoặc cấu hình.

2.2.38. Quan hệ với nguyên lý thiết kế

Bridge hỗ trợ mạnh các nguyên lý:

Single Responsibility Principle. Mỗi hierarchy xử lý một chiều thay đổi.

Open/Closed Principle. Thêm abstraction/implementor mới mà không cần sửa sâu mã cũ.

Dependency Inversion Principle. Abstraction phụ thuộc vào interface implementor.

Ngoài ra, Bridge thể hiện rõ tư tưởng “favor composition over inheritance”.

2.2.39. Tóm tắt

Bridge Pattern là lựa chọn phù hợp khi hệ thống có hai chiều biến thiên độc lập và có nguy cơ bùng nổ class nếu kế thừa theo tổ hợp.

Bằng cách tách abstraction và implementation thành hai hệ phân cấp riêng, rồi nối chúng bằng composition, Bridge giúp hệ thống dễ mở rộng, dễ bảo trì và linh hoạt hơn khi thay đổi.

Có thể ghi nhớ ngắn gọn:

Bridge = Tách phân "muôn làm gì" và "làm bằng cách nào"
đề hai phía phát triển độc lập nhưng vẫn phối hợp được.

2.3. Composite (Phức)

2.3.1. Tên và Phân loại

Composite - Mẫu cấu trúc (Structural Pattern)

2.3.2. Mục đích, ý định

Soạn các đối tượng thành cấu trúc cây để biểu diễn các phần-toàn thể. Cho phép client xử lý các đối tượng riêng lẻ và tổ hợp của chúng một cách thống nhất.

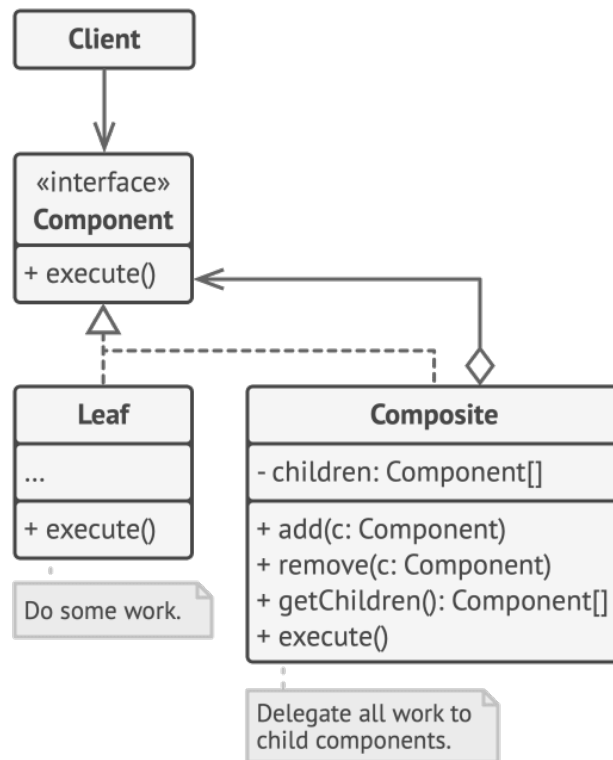
2.3.3. Motivation

Trong các ứng dụng có cấu trúc cây (file system, UI components), cần xử lý cả phần tử đơn lẻ và tập hợp phần tử. Composite cho phép cách tiếp cận thống nhất.

2.3.4. Khả năng ứng dụng

- Biểu diễn các cấu trúc part-whole dưới dạng cây
- Muốn client không cần phân biệt đối tượng riêng lẻ hay tổ hợp
- Cần hỗ trợ các phép toán trên cây đối tượng

2.3.5. Cấu trúc



Hình 2.15: Cấu trúc của mẫu Composite

Component là giao diện chung. Leaf là phần tử lá (không có con). Composite là phần tử chứa các component khác.

2.3.6. Các thành viên

- Component: giao diện chung cho phần tử và tổ hợp
- Leaf: phần tử lá, không có con
- Composite: chứa danh sách component và cho phép thêm/xóa
- Client: làm việc qua giao diện Component

2.3.7. Sự cộng tác

Client làm việc qua giao diện Component. Composite ủy nhiệm các thao tác cho các component con.

2.3.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Tạo cấu trúc phân cấp sạch; dễ thêm thành phần mới. **Nhược điểm:** Khó hạn chế các loại component; có thể quá tổng quát.

2.3.9. Chú ý liên quan đến việc cài đặt

- Thiết kế giao diện Component cẩn thận
- Quản lý danh sách con trong Composite

2.3.10. Mã nguồn minh họa

File system là ví dụ: Directory chứa File và Directory con.

2.3.11. Ví dụ về các hệ thống thực tế

- File system (folder, file)
- UI framework (panel, button, textbox)
- Document structure (chapters, sections, paragraphs)
- Biểu diễn cây tổ chức công ty

2.3.12. Các mẫu liên quan

Thường dùng với Iterator, Visitor. Tương tự Iterator nhưng cho cấu trúc.

2.4. Decorator (Phức)

2.4.1. Tên và Phân loại

Decorator (Trang trí) — Structural

2.4.2. Mục đích, ý định

Gắn thêm trách nhiệm (hành vi) cho đối tượng một cách linh hoạt trong lúc chạy, thay vì tạo nhiều lớp con cố định.

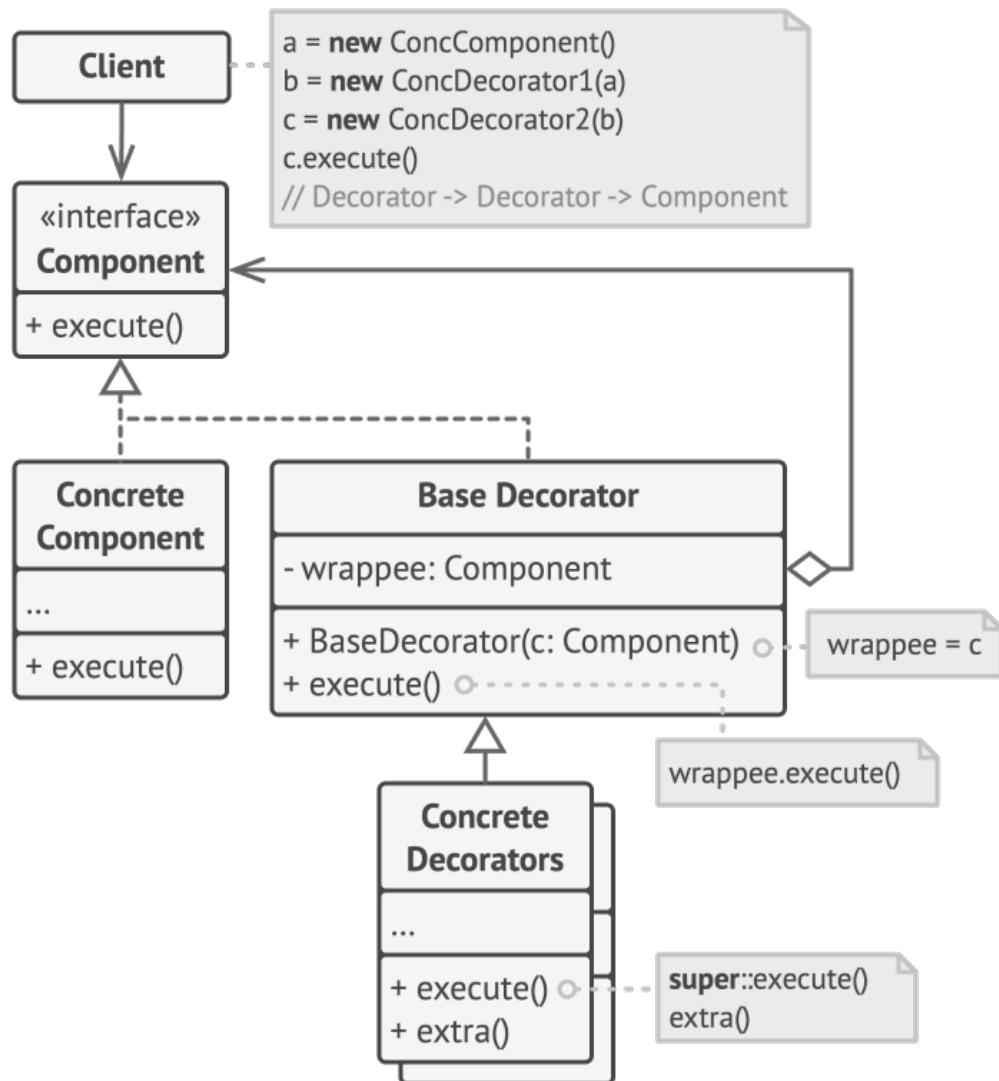
2.4.3. Motivation

Khi cần thêm nhiều biến thể tính năng (ví dụ: thông báo + log + mã hóa + nén) theo tổ hợp, kế thừa sẽ bùng nổ số lớp. Decorator cho phép “bọc” đối tượng bằng các lớp bổ sung để ghép tính năng linh hoạt.

2.4.4. Khả năng ứng dụng

- Cần mở rộng hành vi cho đối tượng tại runtime.
- Tránh tạo nhiều lớp con chỉ để kết hợp tính năng.
- Muốn áp dụng/loại bỏ tính năng theo cấu hình.

2.4.5. Cấu trúc



Hình 2.16: Cấu trúc của mẫu Decorator

Component định nghĩa giao diện chung. ConcreteComponent là đối tượng gốc. Decorator giữ tham chiếu đến Component và chuyển tiếp lời gọi. ConcreteDecorator mở rộng bằng hành vi trước/sau khi ủy quyền.

2.4.6. Các thành viên

- Component: giao diện chung cho đối tượng và lớp bọc.
- ConcreteComponent: triển khai hành vi cốt lõi.
- Decorator: lớp trừu tượng giữ tham chiếu Component, ủy quyền lời gọi.
- ConcreteDecorator: thêm hành vi cụ thể.

2.4.7. Sự cộng tác

Client thao tác qua kiểu Component. Các Decorator có thể xếp chuỗi, mỗi lớp bọc thêm hành vi rồi chuyển tiếp đến đối tượng bên trong.

2.4.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu: linh hoạt, tuân thủ Open/Closed, tránh bùng nổ lớp con. Nhược: nhiều lớp nhỏ, khó debug chuỗi bọc, thứ tự bọc ảnh hưởng kết quả.

2.4.9. Chú ý liên quan đến việc cài đặt

- Đảm bảo Decorator và Component cùng giao diện.
- Tránh lộ kiểu cụ thể để không phá vỡ khả năng thay thế.
- Quản lý thứ tự bọc nếu hành vi phụ thuộc nhau.

2.4.10. Một số so sánh nếu có

So với kế thừa: Decorator linh hoạt hơn khi cần tổ hợp tính năng. So với Proxy: Proxy kiểm soát truy cập, Decorator tập trung mở rộng hành vi. So với Adapter: Adapter đổi giao diện, Decorator giữ nguyên giao diện.

2.4.11. Mã nguồn minh họa

Ví dụ khái niệm: bọc MessageSender bằng LoggingDecorator rồi EncryptDecorator để thêm log và mã hóa mà không đổi đối tượng gốc.

2.4.12. Ví dụ về các hệ thống thực tế

- Java I/O: BufferedInputStream, DataInputStream bọc InputStream.
- UI frameworks: thêm border, shadow, scroll bằng lớp bọc.

- Middleware pipeline: thêm kiểm tra, ghi log, nén.

2.4.13. Các mẫu liên quan

Composite (cấu trúc cây), Proxy (kiểm soát truy cập), Adapter (đổi giao diện). Có thể phối hợp với Factory Method để tạo chuỗi bọc.

2.5. Facade (Phát)

2.5.1. Tên và phân loại

Facade Pattern có thể hiểu là **mẫu mặt tiền**. Từ “facade” mang nghĩa là mặt ngoài hoặc lớp giao diện phía trước của một công trình. Trong thiết kế phần mềm, Facade đóng vai trò tương tự: nó đứng phía trước một hệ thống con phức tạp và cung cấp cho client một cách sử dụng đơn giản hơn.

Facade thuộc nhóm **Structural Pattern** vì nó tập trung vào cách tổ chức và ghép nối các lớp/object trong hệ thống. Pattern này không thay đổi nghiệp vụ chính của các subsystem, mà tạo thêm một lớp trung gian để client không phải tương tác trực tiếp với quá nhiều class bên trong. Nói ngắn gọn, Facade là một lớp “cửa vào” cấp cao cho một nhóm class/service phức tạp.

2.5.2. Vấn đề cần giải quyết

Trong nhiều hệ thống, một chức năng nhìn từ bên ngoài có vẻ đơn giản nhưng bên trong lại cần phối hợp nhiều class, service hoặc subsystem khác nhau.

Ví dụ trong chức năng đặt hàng của một hệ thống thương mại điện tử, client có thể phải gọi nhiều thành phần:

- InventoryService để kiểm tra và trừ tồn kho.
- PaymentService để xử lý thanh toán.
- OrderService để tạo đơn hàng.
- ShippingService để tạo yêu cầu giao hàng.
- EmailService để gửi email xác nhận.

- NotificationService để gửi thông báo cho người dùng.

Nếu client phải tự gọi trực tiếp từng service, code client sẽ trở nên phức tạp:

```
inventory.CheckStock(productId, quantity);
```

```
payment.Pay(userId, amount);
```

```
order.CreateOrder(userId, items);
```

```
shipping.CreateShipment(orderId);
```

```
email.SendConfirmation(userEmail, orderId);
```

Cách thiết kế này tạo ra nhiều vấn đề:

- Client biết quá nhiều chi tiết nội bộ của subsystem.
- Client phải biết gọi service nào trước, service nào sau.
- Khi subsystem thay đổi, nhiều nơi trong client có thể phải sửa theo.
- Logic điều phối bị lặp lại ở nhiều màn hình, controller hoặc service khác nhau.
- Code client khó đọc vì bị trộn giữa mục tiêu nghiệp vụ cấp cao và các thao tác kỹ thuật cấp thấp.
- Hệ thống dễ bị coupling chặt giữa tầng bên ngoài và các class bên trong.

Vấn đề cốt lõi là: **làm sao cung cấp một interface đơn giản, thống nhất để client sử dụng một hệ thống phức tạp bên trong mà không cần biết toàn bộ chi tiết triển khai?**

Facade Pattern giải quyết vấn đề này bằng cách đặt một lớp trung gian ở phía trước subsystem.

Client chỉ gọi Facade, còn Facade chịu trách nhiệm điều phối các subsystem cần thiết.

2.5.3. Mục đích và ý định

Mục đích chính của Facade là **đơn giản hóa cách sử dụng một subsystem phức tạp.**

Facade không nhất thiết phải thay thế toàn bộ subsystem. Các class bên trong vẫn tồn tại và vẫn có thể được dùng trực tiếp nếu cần. Tuy nhiên, với các use case phổ biến, Facade cung cấp một API cấp cao giúp client làm việc dễ hơn.

Facade thường được dùng để:

- Cung cấp một điểm truy cập đơn giản cho một nhóm class phức tạp.
- Che giấu thứ tự gọi nhiều subsystem.
- Giảm phụ thuộc giữa client và subsystem.
- Gom logic điều phối nhiều service vào một nơi rõ ràng.
- Làm cho code client ngắn gọn và dễ đọc hơn.
- Tạo ranh giới giữa các layer trong kiến trúc nhiều tầng.

Ví dụ, thay vì controller phải gọi `InventoryService`, `PaymentService`, `OrderService`, `ShippingService` và `EmailService`, controller chỉ cần gọi:

```
checkoutFacade.Checkout(request);
```

2.5.4. Định nghĩa

Facade Pattern là một structural design pattern cung cấp một interface đơn giản, thống nhất cho một tập hợp interface phức tạp trong một subsystem.

Định nghĩa ngắn gọn:

- Client cần thực hiện một chức năng cấp cao.
- Subsystem Classes là các class/service thực hiện logic thật sự.
- Facade đứng trước subsystem và cung cấp các phương thức đơn giản cho client.
- Client gọi Facade thay vì gọi trực tiếp nhiều subsystem.
- Facade điều phối thứ tự gọi các subsystem và có thể tổng hợp kết quả trả về.

Nói đơn giản, Facade là “mặt tiền” của một hệ thống phức tạp. Client không cần biết bên trong có bao nhiêu class, các class đó liên hệ thế nào, hoặc phải gọi theo thứ tự nào. Client chỉ cần gọi một phương thức cấp cao từ Facade.

2.5.5. Ý tưởng cốt lõi

Ý tưởng chính của Facade Pattern là **gom nhiều thao tác phức tạp của nhiều class con vào một class cấp cao**, để client chỉ cần làm việc với một điểm truy cập đơn giản.

Facade thường thực hiện các việc sau:

1. Nhận request từ client.
2. Kiểm tra hoặc chuẩn hóa dữ liệu đầu vào nếu cần.
3. Gọi các subsystem cần thiết.
4. Điều phối thứ tự xử lý giữa các subsystem.
5. Ẩn chi tiết phức tạp bên trong.
6. Tổng hợp hoặc chuyển đổi kết quả nếu cần.
7. Trả kết quả đơn giản cho client.

Có thể hình dung luồng xử lý như sau:

```
Client -> Facade -> Subsystem A
                        -> Subsystem B
                        -> Subsystem C
                        -> Subsystem D
```

Client chỉ nhìn thấy Facade. Các subsystem phía sau có thể phức tạp, có nhiều dependency hoặc nhiều bước xử lý, nhưng những chi tiết đó được đóng gói bên trong Facade.

2.5.6. Thành phần chính

Một cấu trúc Facade Pattern điển hình gồm ba nhóm thành phần chính.

2.5.7. Client

Client là đối tượng sử dụng chức năng cấp cao. Client có thể là controller, UI, API endpoint, service khác hoặc một module bên ngoài.

Client không nên phải biết quá nhiều chi tiết nội bộ của subsystem. Thay vào đó, client chỉ cần gọi các phương thức rõ nghĩa trên Facade.

Ví dụ:

```
await checkoutFacade.CheckoutAsync(request);
```

Ở đây client không cần biết bên trong checkout gồm những bước nào.

2.5.8. Facade

Facade là lớp trung gian cung cấp interface đơn giản cho client.

Nhiệm vụ của Facade là:

- Giữ tham chiếu đến các subsystem cần thiết.
- Cung cấp các method cấp cao như Checkout(), PlaceOrder(), GenerateReport().
- Gọi các subsystem theo đúng thứ tự.
- Ẩn chi tiết xử lý phức tạp.
- Có thể chuyển đổi dữ liệu đầu vào/đầu ra để client dễ dùng hơn.

Facade không nên chứa toàn bộ business logic chi tiết. Nó chủ yếu là nơi **orchestrate** hoặc **coordinate** các subsystem.

2.5.9. Subsystem Classes

Subsystem Classes là các class/service thật sự xử lý logic bên trong.

Ví dụ trong hệ thống đặt hàng:

- InventoryService xử lý tồn kho.
- PaymentService xử lý thanh toán.
- OrderService xử lý đơn hàng.

- EmailService xử lý gửi email.

Các subsystem không nhất thiết phải biết Facade tồn tại. Chúng chỉ tập trung vào trách nhiệm riêng của mình.

2.5.10. Additional Facade

Trong hệ thống lớn, có thể có nhiều Facade khác nhau cho nhiều nhóm use case khác nhau.

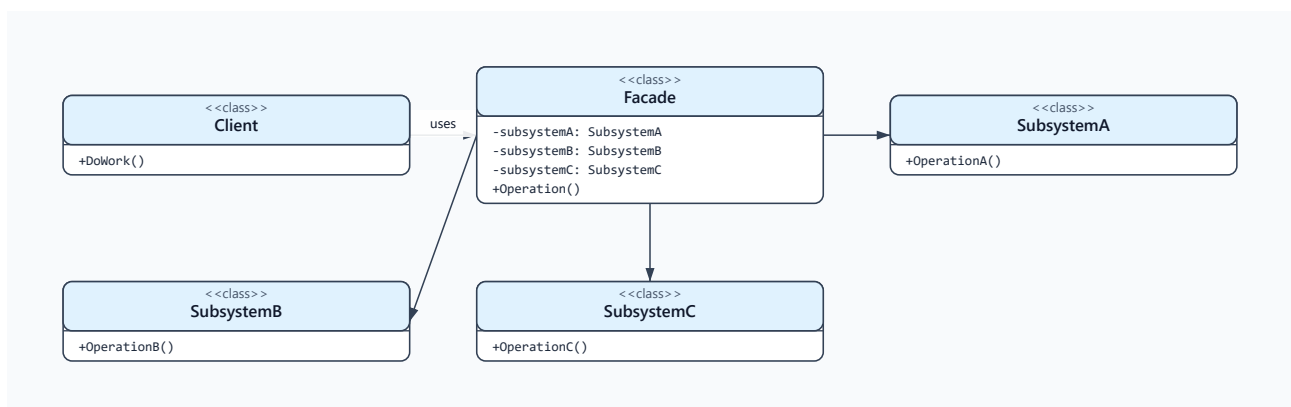
Ví dụ:

- CheckoutFacade cho luồng thanh toán.
- ReportFacade cho luồng xuất báo cáo.
- UserManagementFacade cho luồng quản lý người dùng.

Việc chia nhỏ Facade giúp tránh tình trạng một Facade quá lớn và trở thành God Class.

2.5.11. Cấu trúc UML tổng quát

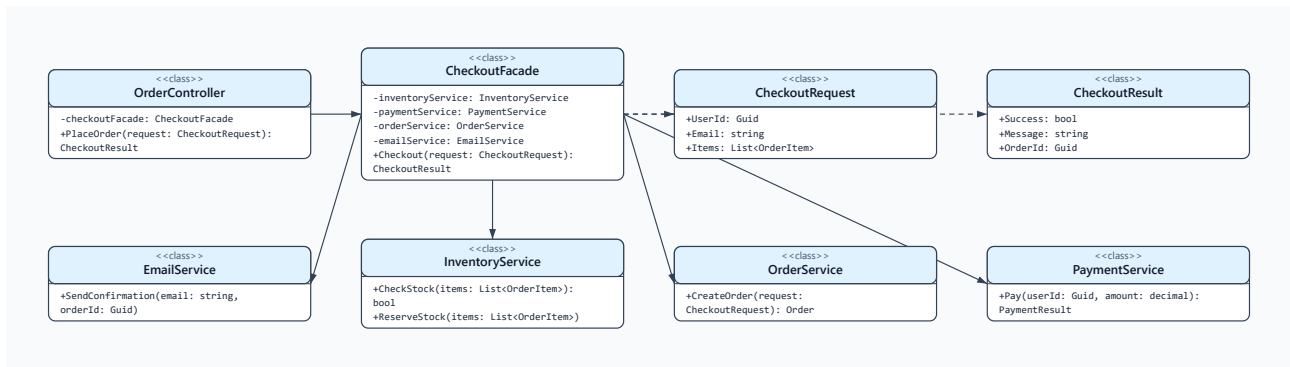
Dưới đây là class diagram tổng quát bằng PlantUML:



Hình 2.17: Cấu trúc UML tổng quát của Facade Pattern

2.5.12. UML minh họa: Checkout Facade

Ví dụ dưới đây mô phỏng chức năng checkout trong hệ thống bán hàng. `OrderController` chỉ gọi `CheckoutFacade`, còn Facade điều phối nhiều service bên trong.



Hình 2.18: Class diagram của ví dụ Checkout Facade

2.5.13. Luồng hoạt động

Luồng hoạt động cơ bản của Facade Pattern gồm các bước:

1. Client cần thực hiện một chức năng cấp cao.
2. Client gọi một phương thức đơn giản trên Facade.
3. Facade nhận request và xác định các subsystem cần dùng.
4. Facade gọi các subsystem theo đúng thứ tự.
5. Các subsystem xử lý công việc chuyên biệt của chúng.
6. Facade có thể tổng hợp kết quả hoặc xử lý lỗi.
7. Client nhận kết quả mà không cần biết chi tiết bên trong.

Với ví dụ checkout:

OrderController

```
-> CheckoutFacade.Checkout()

    -> InventoryService.CheckStock()

    -> InventoryService.ReserveStock()

    -> PaymentService.Pay()

    -> OrderService.CreateOrder()
```

```
-> EmailService.SendConfirmation()
```

```
<- CheckoutResult
```

Điểm quan trọng là controller không cần chứa logic điều phối các service. Nếu sau này cần thêm CouponService, ShippingService hoặc NotificationService, ta có thể sửa trong Facade thay vì sửa nhiều controller khác nhau.

2.5.14. Ví dụ code C#

2.5.15. Subsystem classes

```
public sealed class InventoryService
{
    public bool CheckStock(IEnumerable<OrderItem> items)
    {
        Console.WriteLine("Checking inventory...");
        return items.All(item => item.Quantity > 0);
    }

    public void ReserveStock(IEnumerable<OrderItem> items)
    {
        Console.WriteLine("Reserving stock...");
    }
}

public sealed class PaymentService
{

```



```

    public PaymentResult Pay(Guid userId, decimal amount)
    {
        Console.WriteLine($"Processing payment: {amount:N0} VND");
        return new PaymentResult(true, "Payment successful");
    }
}

public sealed class OrderService
{
    public Order CreateOrder(CheckoutRequest request)
    {
        Console.WriteLine("Creating order...");
        return new Order(Guid.NewGuid(), request.UserId,
request.TotalAmount);
    }
}

public sealed class EmailService
{
    public void SendConfirmation(string email, Guid orderId)
    {
        Console.WriteLine($"Sending confirmation email to {email} for order
{orderId}");
    }
}

```

```
    }  
}
```

2.5.16. Model đơn giản

```
public sealed record OrderItem(Guid ProductId, int Quantity, decimal  
UnitPrice);
```

```
public sealed record CheckoutRequest(  
    Guid UserId,  
    string Email,  
    IReadOnlyList<OrderItem> Items)  
{  
    public decimal TotalAmount => Items.Sum(item => item.Quantity *  
item.UnitPrice);  
}
```

```
public sealed record PaymentResult(bool Success, string Message);
```

```
public sealed record Order(Guid Id, Guid UserId, decimal TotalAmount);
```

```
public sealed record CheckoutResult(bool Success, string Message, Guid?  
OrderId = null);
```

2.5.17. Facade

```
public sealed class CheckoutFacade  
{
```

```

private readonly InventoryService _inventoryService;

private readonly PaymentService _paymentService;

private readonly OrderService _orderService;

private readonly EmailService _emailService;


public CheckoutFacade(
    InventoryService inventoryService,
    PaymentService paymentService,
    OrderService orderService,
    EmailService emailService)
{
    _inventoryService = inventoryService;
    _paymentService = paymentService;
    _orderService = orderService;
    _emailService = emailService;
}


public CheckoutResult Checkout(CheckoutRequest request)
{
    if (!_inventoryService.CheckStock(request.Items))
    {
        return new CheckoutResult(false, "Not enough stock");
    }
}

```

```

_inventoryService.ReserveStock(request.Items);

PaymentResult paymentResult = _paymentService.Pay(
    request.UserId,
    request.TotalAmount);

if (!paymentResult.Success)
{
    return new CheckoutResult(false, paymentResult.Message);
}

Order order = _orderService.CreateOrder(request);
_emailService.SendConfirmation(request.Email, order.Id);

return new CheckoutResult(true, "Checkout successful", order.Id);
}
}

```

2.5.18. Client

```

public sealed class OrderController
{
    private readonly CheckoutFacade _checkoutFacade;

```

```

public OrderController(CheckoutFacade checkoutFacade)
{
    _checkoutFacade = checkoutFacade;
}

public CheckoutResult PlaceOrder(CheckoutRequest request)
{
    return _checkoutFacade.Checkout(request);
}
}

```

2.5.19. Chạy thử

```

var facade = new CheckoutFacade(
    new InventoryService(),
    new PaymentService(),
    new OrderService(),
    new EmailService());

var controller = new OrderController(facade);

var request = new CheckoutRequest(
    Guid.NewGuid(),
    "customer@example.com",
    new List<OrderItem>

```

```

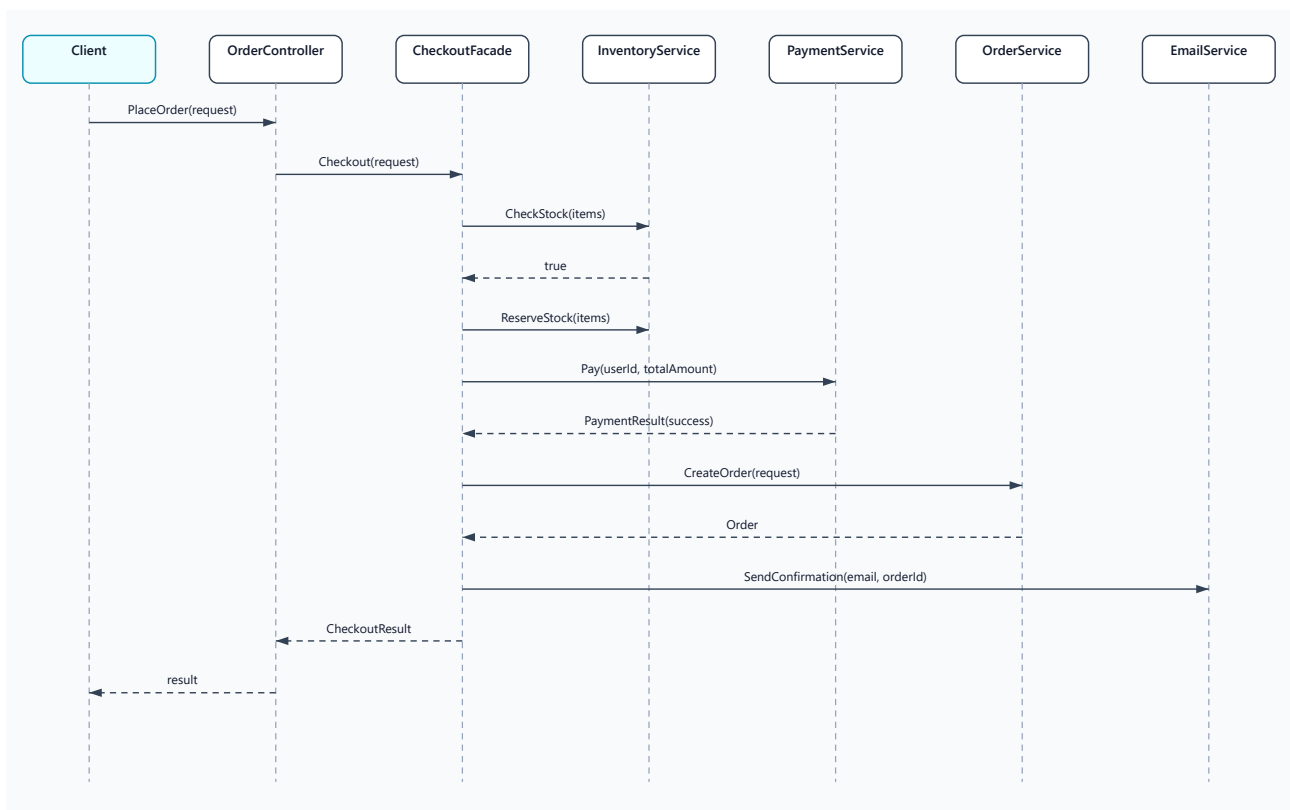
{
    new(Guid.NewGuid(), 2, 50000),
    new(Guid.NewGuid(), 1, 120000)
});

```

```
CheckoutResult result = controller.PlaceOrder(request);
```

```
Console.WriteLine(result.Message);
```

Client chỉ gọi PlaceOrder() hoặc Checkout(), không cần tự điều phối tồn kho, thanh toán, tạo đơn và gửi email.



Hình 2.19: Luồng chạy code ví dụ CheckoutFacade

2.5.20. Đánh giá

2.5.21. Ưu điểm

Giảm độ phức tạp cho client. Client không cần biết nhiều class/service bên trong. Nó chỉ cần gọi một API cấp cao.

Che giấu chi tiết xử lý nội bộ. Thứ tự gọi subsystem, cách xử lý lỗi, cách tổng hợp kết quả được đặt bên trong Facade.

Giảm coupling giữa client và subsystem. Khi subsystem thay đổi, nếu interface của Facade vẫn giữ nguyên thì client ít bị ảnh hưởng.

Phù hợp với kiến trúc nhiều layer. Facade có thể đóng vai trò service cấp cao ở tầng application, giúp controller hoặc UI không chứa quá nhiều logic điều phối.

Dễ bảo trì hơn khi subsystem thay đổi. Ví dụ thêm CouponService hoặc đổi EmailService thành NotificationService, ta có thể cập nhật trong Facade.

Code client dễ đọc hơn. Thay vì đọc nhiều lời gọi rời rạc, client thể hiện rõ ý định nghiệp vụ: `Checkout()`, `GenerateReport()`, `RegisterUser()`.

2.5.22. Nhược điểm

Facade có thể trở thành God Class. Nếu gom quá nhiều chức năng không liên quan vào một Facade, class này sẽ phình to và khó bảo trì.

Có thể che giấu quá nhiều chi tiết quan trọng. Một số client cần kiểm soát chi tiết hơn, nhưng Facade lại chỉ cung cấp API cấp cao.

Không thay thế được thiết kế subsystem tốt. Nếu subsystem bên trong được thiết kế kém, Facade chỉ che bớt sự phức tạp ở bên ngoài chứ không làm hệ thống thật sự tốt hơn.

Có thể làm giảm tính linh hoạt. Client muốn gọi subsystem theo cách đặc biệt có thể bị giới hạn bởi API của Facade.

Dễ bị lạm dụng. Không phải mọi nhóm class đều cần Facade. Nếu subsystem đơn giản, thêm Facade có thể chỉ làm tăng số lượng class không cần thiết.

2.5.23. Khi nào nên dùng

Nên dùng Facade Pattern khi:

- Client phải gọi quá nhiều class/service để thực hiện một use case.
- Có một subsystem phức tạp cần được ẩn sau một API đơn giản.
- Muốn giảm phụ thuộc giữa tầng bên ngoài và các class bên trong.
- Muốn gom logic điều phối nhiều service vào một nơi rõ ràng.
- Muốn tạo một boundary giữa các layer hoặc module.
- Có nhiều client cùng lặp lại một chuỗi thao tác giống nhau.
- Muốn cung cấp API đơn giản cho thư viện hoặc framework nội bộ.

Ví dụ phù hợp:

- CheckoutFacade trong hệ thống bán hàng.
- ReportFacade để gom nhiều bước truy vấn, tính toán và xuất file.
- AuthenticationFacade để gom login, token, refresh token và audit log.
- FileProcessingFacade để gom đọc file, parse, validate và import dữ liệu.

2.5.24. Khi nào không nên dùng

Không nên dùng Facade Pattern khi:

- Subsystem rất đơn giản, client chỉ cần gọi một hoặc hai method.
- Client thật sự cần kiểm soát chi tiết từng subsystem.
- Facade chỉ chuyển tiếp method một cách máy móc mà không đơn giản hóa gì.
- Facade gom quá nhiều chức năng không liên quan.
- Vấn đề chính là interface không tương thích; trường hợp đó nên cân nhắc Adapter.

- Vấn đề chính là thêm hành vi mới cho object; trường hợp đó nên cân nhắc Decorator.

Một dấu hiệu lạm dụng Facade là class có tên rất chung như SystemFacade, AppFacade, ManagerFacade nhưng chứa hàng chục method không liên quan.

2.5.25. So sánh Facade và Adapter

Cả Facade và Adapter đều thuộc nhóm Structural Pattern, đều có thể bọc một hoặc nhiều object bên trong. Tuy nhiên mục đích của chúng khác nhau.

Tiêu chí	Facade	Adapter
Mục đích chính	Đơn giản hóa cách sử dụng một subsystem phức tạp.	Làm hai interface không tương thích có thể làm việc với nhau.
Vấn đề giải quyết	Client phải gọi quá nhiều class hoặc quá nhiều bước.	Client không dùng được class có sẵn vì interface không khớp.
Số lượng class thường bọc	Thường bọc nhiều class/subsystem.	Thường bọc một class hoặc một service không tương thích.
Có tập trung vào tương thích interface không?	Không phải trọng tâm chính.	Có, đây là trọng tâm chính.
Client dùng vì	Muốn API đơn giản hơn.	Muốn dùng lại class không tương thích.
Ví dụ	CheckoutFacade.Checkout() gọi nhiều service.	StripeAdapter.Pay() gọi StripeService.

Tóm lại: Facade giải quyết vấn đề **phức tạp khi sử dụng subsystem**, còn Adapter giải quyết vấn đề **không tương thích interface**.

2.5.26. So sánh Facade và Decorator

Facade và Decorator đều có thể được xem là một lớp bọc bên ngoài object khác. Tuy nhiên ý định thiết kế khác nhau.

Tiêu chí	Facade	Decorator
Mục đích chính	Đơn giản hóa cách dùng subsystem.	Thêm chức năng mới cho object mà không sửa class gốc.
Interface sau khi bọc	Thường là interface cấp cao và đơn giản hơn.	Thường giữ cùng interface với object gốc.
Đối tượng được bọc	Nhiều class/service bên trong.	Một object cụ thể hoặc nhiều wrapper lồng nhau.
Tập trung vào	Đơn giản hóa và che giấu phức tạp.	Mở rộng hành vi.
Ví dụ	CheckoutFacade gọi Payment, Inventory, Email.	LoggingPaymentGateway thêm logging trước/sau khi thanh toán.

Tóm lại: Facade làm hệ thống dễ dùng hơn, còn Decorator làm object có thêm hành vi mới.

2.5.27. So sánh Facade và Mediator

Facade và Mediator đôi khi dễ bị nhầm vì cả hai đều đứng giữa nhiều object. Khác biệt nằm ở quan hệ và mục tiêu.

Tiêu chí	Facade	Mediator
Mục đích chính	Cung cấp interface đơn giản cho subsystem.	Điều phối giao tiếp giữa nhiều object.
Quan hệ với các object	Facade đứng trước subsystem.	Mediator đứng giữa các object ngang hàng.
Client có biết subsystem không?	Thường không cần biết.	Các object thường biết mediator.
Trọng tâm	Đơn giản hóa cách sử dụng.	Giảm phụ thuộc nhiều-nhiều giữa các object.

Ví dụ	CheckoutFacade gọi Payment, Inventory, Email.	ChatRoomMediator điều phối các user gửi tin nhắn.
-------	---	---

Tóm lại: Facade là cửa vào đơn giản cho một subsystem, còn Mediator là trung tâm điều phối tương tác giữa nhiều object ngang hàng.

2.5.28. So sánh Facade và Proxy

Facade và Proxy đều có thể đứng trước object khác, nhưng mục tiêu không giống nhau.

Tiêu chí	Facade	Proxy
Mục đích chính	Đơn giản hóa một hệ thống phức tạp.	Kiểm soát quyền truy cập đến object thật.
Interface	Có thể cung cấp interface mới, cấp cao hơn.	Thường giữ cùng interface với real subject.
Đối tượng phía sau	Nhiều subsystem.	Một object thật hoặc tài nguyên từ xa.
Tập trung vào	Đơn giản hóa sử dụng.	Lazy loading, access control, caching, remote access.
Ví dụ	ReportFacade.GenerateMonthly	ImageProxy.Display() tải ảnh khi cần.

2.5.29. Liên hệ với kiến trúc phần mềm

Trong các dự án thực tế, Facade thường xuất hiện dưới các tên như:

- ApplicationService
- UseCaseService
- Orchestrator
- Coordinator
- Gateway

- Manager, nếu dùng đúng phạm vi

Ví dụ trong Clean Architecture hoặc DDD, tầng Application thường chứa các service điều phối use case. Những service này có thể mang tinh thần của Facade vì chúng cung cấp API cấp cao cho controller và điều phối domain service, repository, external service.

Tuy nhiên, không nên đánh đồng mọi service với Facade. Một class chỉ nên được xem là Facade khi nó thật sự đơn giản hóa việc sử dụng một nhóm subsystem phức tạp.

2.5.30. Lưu ý khi triển khai

Để dùng Facade hiệu quả, cần chú ý một số điểm:

- Đặt tên method theo use case cấp cao, ví dụ `Checkout()`, `RegisterUser()`, `GenerateReport()`.
- Không để Facade chứa quá nhiều logic chi tiết. Logic chuyên biệt nên nằm trong subsystem.
- Không gom nhiều use case không liên quan vào một Facade.
- Có thể chia Facade theo module hoặc bounded context.
- Không bắt buộc client chỉ được dùng Facade. Với nhu cầu đặc biệt, client nội bộ vẫn có thể dùng subsystem trực tiếp.
- Kiểm soát kích thước Facade để tránh God Class.
- Dùng dependency injection để truyền subsystem vào Facade thay vì tự khởi tạo cứng bên trong.

2.5.31. Kết luận

Facade Pattern là một mẫu thiết kế hữu ích khi hệ thống có nhiều class/service bên trong và client cần một cách sử dụng đơn giản hơn. Pattern này giúp giảm độ phức tạp cho client, che giấu chi tiết xử lý nội bộ, giảm coupling và làm code cấp cao dễ đọc hơn.

Điểm mạnh nhất của Facade là biến một quy trình phức tạp thành một lời gọi đơn giản. Tuy nhiên, Facade không sửa được thiết kế kém của subsystem và cũng không nên bị lạm dụng thành một God Class chứa mọi thứ.

Có thể ghi nhớ Facade bằng câu sau:

Facade không làm subsystem biến mất; nó chỉ tạo một cửa vào đơn giản để client không phải đi qua quá nhiều cánh cửa nhỏ bên trong.

2.6. Flyweight (Phát)

2.6.1. Tên và phân loại

Flyweight Pattern có thể hiểu là **mẫu hạng ruồi** hoặc **mẫu đối tượng nhẹ**. Tên gọi này nhấn mạnh mục tiêu chính của pattern: làm cho hệ thống sử dụng các object nhẹ hơn bằng cách chia sẻ những dữ liệu giống nhau giữa nhiều object.

Flyweight thuộc nhóm **Structural Pattern** vì nó tập trung vào cách tổ chức cấu trúc object để giảm chi phí bộ nhớ. Pattern này không thay đổi trực tiếp thuật toán nghiệp vụ, mà thay đổi cách lưu trữ và chia sẻ trạng thái giữa các object.

Nói ngắn gọn, Flyweight là pattern dùng để **tái sử dụng các object chứa dữ liệu dùng chung** thay vì tạo ra nhiều object riêng biệt nhưng chứa dữ liệu lặp lại giống nhau.

2.6.2. Vấn đề cần giải quyết

Trong một số hệ thống, ta cần tạo ra rất nhiều object giống nhau hoặc gần giống nhau. Nếu mỗi object đều lưu toàn bộ dữ liệu của riêng nó, kể cả những dữ liệu bị lặp lại, hệ thống sẽ tiêu tốn rất nhiều bộ nhớ.

Ví dụ:

- Game có hàng ngàn viên đạn, cây cối, quân lính hoặc particle effect.
- Trình soạn thảo văn bản có thể hiển thị hàng triệu ký tự.

- Bản đồ có rất nhiều icon như nhà hàng, cây xăng, khách sạn, trạm xe buýt.
- App giao hàng có thể hiển thị nhiều marker shipper, marker cửa hàng và marker đơn hàng trên bản đồ.
- Hệ thống vẽ đồ họa có rất nhiều hình tròn, hình vuông hoặc texture giống nhau.

Giả sử một game cần vẽ 100.000 cây trong rừng. Mỗi cây có các thông tin:

- Tọa độ x, y .
- Chiều cao hoặc kích thước.
- Loại cây: thông, sồi, anh đào.
- Màu sắc.
- Texture/hình ảnh.
- Dữ liệu mesh/model.

Trong đó, x, y , chiều cao có thể khác nhau theo từng cây. Nhưng type, color, texture, mesh của cùng một loại cây thường giống nhau. Nếu mỗi object Tree đều lưu lại toàn bộ texture/model, bộ nhớ sẽ bị lãng phí nghiêm trọng.

Vấn đề cốt lõi là: **làm sao biểu diễn số lượng lớn object mà không phải lưu lặp lại những phần dữ liệu giống nhau trong từng object?**

Flyweight Pattern giải quyết vấn đề này bằng cách tách dữ liệu thành hai phần:

- Dữ liệu dùng chung, được lưu trong object Flyweight.
- Dữ liệu riêng theo từng context, được lưu bên ngoài hoặc truyền vào khi xử lý.

2.6.3. Mục đích và ý định

Mục đích chính của Flyweight là **giảm lượng bộ nhớ sử dụng khi hệ thống phải làm việc với số lượng lớn object có nhiều phân trạng thái giống nhau.**

Flyweight thường được dùng để:

- Giảm số lượng object nặng cần tạo ra.
- Chia sẻ dữ liệu giống nhau giữa nhiều context.
- Tăng hiệu quả khi xử lý số lượng lớn object.
- Tách rõ dữ liệu dùng chung và dữ liệu riêng.
- Tránh lưu lặp lại dữ liệu lớn như texture, font, icon, metadata, style hoặc configuration.

Flyweight không có nghĩa là toàn bộ object đều chỉ có một instance. Thay vào đó, hệ thống có thể có nhiều Flyweight, mỗi Flyweight đại diện cho một nhóm dữ liệu dùng chung.

Ví dụ trong bài toán rừng cây:

- `TreeType("Oak", "green", "oak.png")` là một Flyweight.
- 10.000 cây sồi khác nhau có thể cùng tham chiếu đến cùng một `TreeType`.
- Mỗi cây riêng chỉ cần lưu `x`, `y`, `height` và tham chiếu đến `TreeType`.

2.6.4. Định nghĩa

Flyweight Pattern là một structural design pattern giúp giảm lượng bộ nhớ sử dụng bằng cách chia sẻ những phần trạng thái giống nhau giữa nhiều object, thay vì lưu trữ lặp lại trong từng object.

Định nghĩa ngắn gọn:

- Flyweight chứa phần dữ liệu có thể chia sẻ.
- Context chứa phần dữ liệu riêng của từng object.
- FlyweightFactory quản lý cache để tái sử dụng Flyweight đã tồn tại.
- Client yêu cầu Factory cung cấp Flyweight phù hợp và sử dụng nó trong từng context cụ thể.

Nói đơn giản, Flyweight biến câu hỏi:

Mỗi object có cần giữ toàn bộ dữ liệu không?

thành:

Phân nào giống nhau thì dùng chung, phân nào khác nhau thì lưu riêng.

2.6.5. Ý tưởng cốt lõi

Ý tưởng chính của Flyweight Pattern là: **không lưu dữ liệu giống nhau trong từng object.**

Thay vào đó, hãy lưu dữ liệu giống nhau vào một object dùng chung, còn dữ liệu riêng thì truyền từ bên ngoài hoặc lưu trong object context.

Pattern này dựa trên hai khái niệm quan trọng:

2.6.6. Intrinsic State

Intrinsic state là trạng thái bên trong, có thể dùng chung giữa nhiều object.

Đặc điểm:

- Không phụ thuộc vào từng context cụ thể.
- Thường không thay đổi hoặc ít thay đổi.
- Có thể được chia sẻ an toàn.
- Được lưu trong Flyweight.

Ví dụ với cây trong game:

- Tên loại cây.
- Màu lá mặc định.
- Texture.
- Sprite.
- Mesh/model 3D.

2.6.7. Extrinsic State

Extrinsic state là trạng thái bên ngoài, khác nhau theo từng context cụ thể.

Đặc điểm:

- Phụ thuộc vào từng object cụ thể.

- Không nên lưu trong Flyweight dùng chung.
- Thường được lưu trong Context hoặc truyền vào method khi xử lý.

Ví dụ với cây trong game:

- Vị trí x, y.
- Chiều cao riêng.
- Góc xoay.
- Trạng thái bị cháy, bị chặt, bị chọn.

Nếu lưu nhầm extrinsic state vào Flyweight, các object dùng chung Flyweight có thể ảnh hưởng lẫn nhau. Đây là lỗi thiết kế rất quan trọng cần tránh.

2.6.8. Thành phần chính

Các thành phần chính trong Flyweight Pattern gồm:

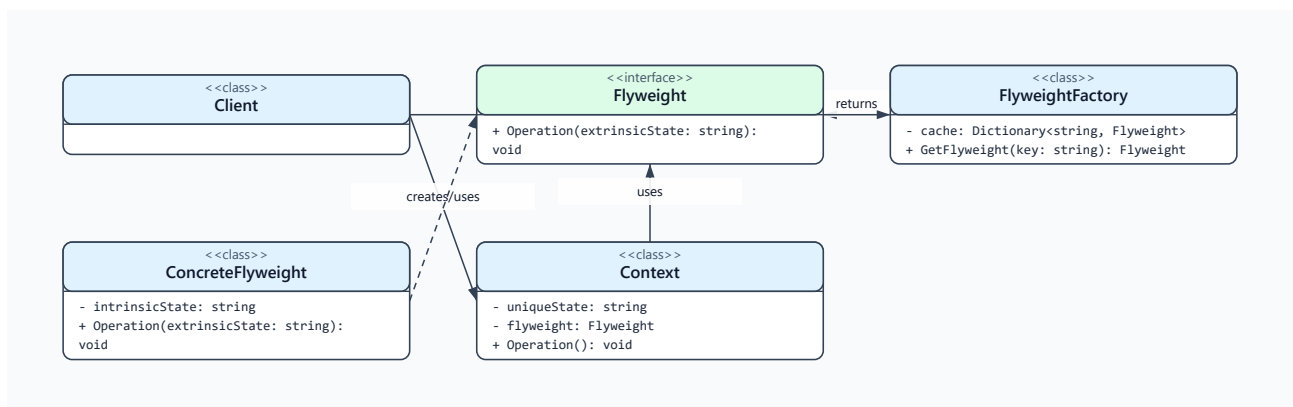
Thành phần	Vai trò
Flyweight	Interface hoặc class khai báo các thao tác dùng chung. Nó thường nhận thêm extrinsic state từ bên ngoài khi thực thi.
ConcreteFlyweight	Object thật sự được chia sẻ. Nó lưu intrinsic state, ví dụ TreeType, CharacterStyle, MapIconType.
FlyweightFactory	Quản lý cache các Flyweight. Khi client yêu cầu một loại Flyweight, Factory trả về object đã tồn tại nếu có, hoặc tạo mới rồi lưu lại.
Context	Object chứa extrinsic state riêng và tham chiếu đến Flyweight. Ví dụ Tree chứa x, y và tham chiếu đến TreeType.

Client	Sử dụng Context hoặc yêu cầu Factory cung cấp Flyweight phù hợp. Client không nên tự tạo Flyweight một cách tùy tiện nếu muốn đảm bảo tái sử dụng.
--------	--

Trong triển khai thực tế, có thể không cần tách Flyweight interface nếu hệ thống đơn giản. Tuy nhiên, với hệ thống có nhiều loại Flyweight, interface giúp code dễ mở rộng và dễ thay thế hơn.

2.6.9. UML tổng quát bằng PlantUML

Sơ đồ tổng quát:



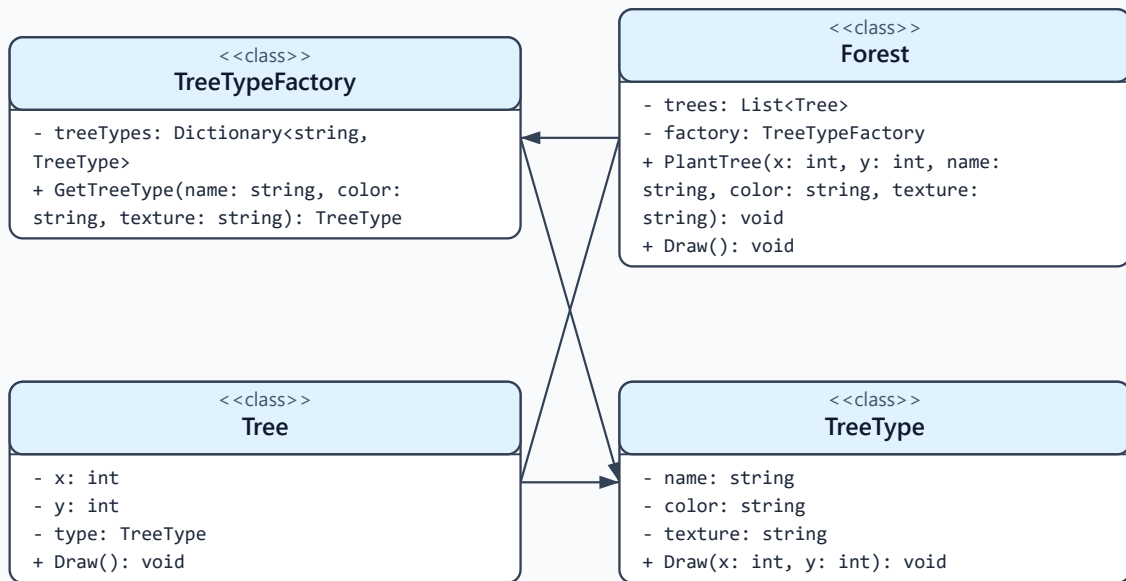
Hình 2.20: Cấu trúc UML tổng quát của Flyweight Pattern

Ý nghĩa sơ đồ:

- Client không cần tạo trực tiếp ConcreteFlyweight.
- FlyweightFactory giữ cache để tránh tạo trùng lặp.
- Context giữ dữ liệu riêng và tham chiếu đến Flyweight dùng chung.
- Flyweight xử lý logic dựa trên intrinsic state của nó và extrinsic state được truyền vào.

2.6.10. UML ví dụ: rừng cây

Ví dụ phổ biến nhất của Flyweight là hệ thống vẽ rừng cây. Nhiều cây có vị trí khác nhau nhưng có thể dùng chung loại cây.



Hình 2.21: Class diagram của ví dụ rừng cây

Trong ví dụ này:

- **TreeType** là Flyweight vì chứa dữ liệu dùng chung.
- **Tree** là Context vì chứa vị trí riêng.
- **TreeTypeFactory** tái sử dụng các **TreeType** đã tồn tại.
- **Forest** đóng vai trò client/use case quản lý nhiều cây.

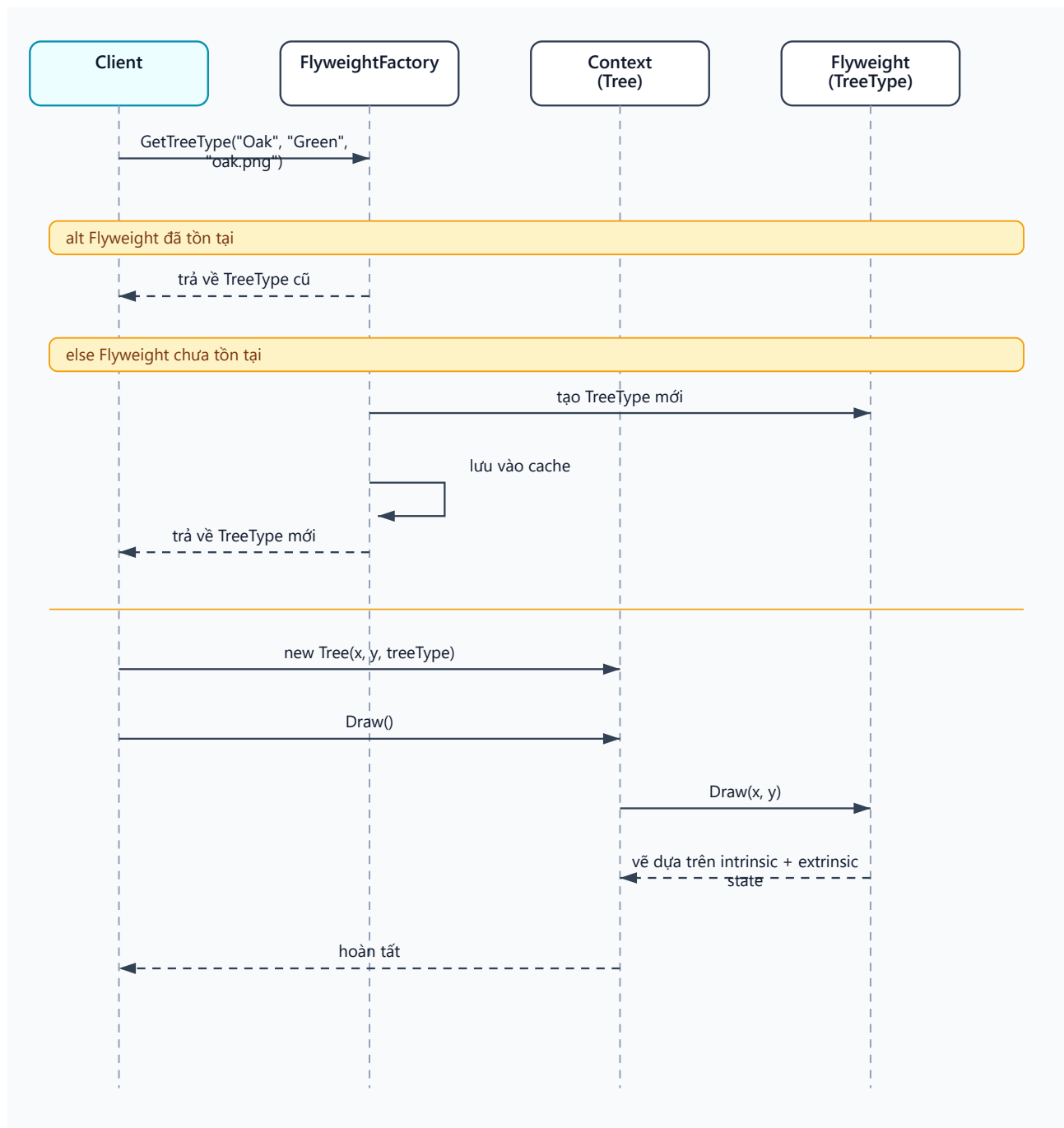
2.6.11. Luồng hoạt động

Luồng hoạt động cơ bản của Flyweight Pattern:

1. Client cần tạo hoặc sử dụng một object mới trong một context cụ thể.
2. Client xác định phần dữ liệu dùng chung, ví dụ loại cây, màu, texture.
3. Client gửi thông tin định danh phần dùng chung cho FlyweightFactory.
4. Factory kiểm tra cache xem Flyweight tương ứng đã tồn tại chưa.
5. Nếu đã tồn tại, Factory trả về object cũ.

6. Nếu chưa tồn tại, Factory tạo Flyweight mới, lưu vào cache rồi trả về.
7. Context lưu phần trạng thái riêng và tham chiếu đến Flyweight.
8. Khi cần xử lý, Context gọi Flyweight và truyền thêm trạng thái riêng nếu cần.

Sơ đồ tuần tự:



Hình 2.22: Luồng tạo và tái sử dụng Flyweight qua Factory

Điểm quan trọng là Factory không tạo mới Flyweight mỗi lần. Nó chỉ tạo mới khi chưa có object dùng chung tương ứng.

2.6.12. Ví dụ minh họa bằng C#

Bài toán: một hệ thống vẽ rừng cây. Mỗi cây có vị trí riêng, nhưng nhiều cây cùng loại có thể dùng chung thông tin tên, màu và texture.

```
using System;
```

```
using System.Collections.Generic;
```

```
// Flyweight: chứa intrinsic state dùng chung
```

```
public class TreeType
```

```
{
```

```
    public string Name { get; }
```

```
    public string Color { get; }
```

```
    public string Texture { get; }
```

```
    public TreeType(string name, string color, string texture)
```

```
{
```

```
        Name = name;
```

```
        Color = color;
```

```
        Texture = texture;
```

```
}
```

```
// x, y là extrinsic state được truyền từ bên ngoài
```

```

public void Draw(int x, int y)
{
    Console.WriteLine(
        $"Draw {Name} tree at ({x}, {y}) with color {Color} and texture
{Texture}"
    );
}
}

```

// FlyweightFactory: quản lý cache Flyweight

```

public class TreeTypeFactory
{
    private readonly Dictionary<string, TreeType> _treeTypes = new();

    public TreeType GetTreeType(string name, string color, string texture)
    {
        string key = $"{name}_{color}_{texture}";

        if (!_treeTypes.ContainsKey(key))
        {
            Console.WriteLine($"Create new TreeType: {key}");
            _treeTypes[key] = new TreeType(name, color, texture);
        }
    }
}

```

```

        else
        {
            Console.WriteLine($"Reuse existing TreeType: {key}");
        }

        return _treeTypes[key];
    }

    public int Count => _treeTypes.Count;
}

```

// Context: chứa extrinsic state riêng của từng cây

```

public class Tree
{
    private readonly int _x;
    private readonly int _y;
    private readonly TreeType _type;

    public Tree(int x, int y, TreeType type)
    {
        _x = x;
        _y = y;
        _type = type;
    }
}

```

```

    }

    public void Draw()
    {
        _type.Draw(_x, _y);
    }
}

// Client/use case
public class Forest
{
    private readonly List<Tree> _trees = new();
    private readonly TreeTypeFactory _factory = new();

    public void PlantTree(int x, int y, string name, string color, string
texture)
    {
        TreeType type = _factory.GetTreeType(name, color, texture);
        Tree tree = new Tree(x, y, type);
        _trees.Add(tree);
    }

    public void Draw()

```



```

{
    foreach (Tree tree in _trees)
    {
        tree.Draw();
    }
}

public int TreeCount => _trees.Count;
public int TreeTypeCount => _factory.Count;
}

public class Program
{
    public static void Main()
    {
        Forest forest = new Forest();

        forest.PlantTree(10, 20, "Oak", "Green", "oak.png");
        forest.PlantTree(30, 40, "Oak", "Green", "oak.png");
        forest.PlantTree(50, 60, "Pine", "DarkGreen", "pine.png");
        forest.PlantTree(70, 80, "Oak", "Green", "oak.png");

        forest.Draw();
    }
}

```

```
        Console.WriteLine($"Total trees: {forest.TreeCount}");

        Console.WriteLine($"Total tree types: {forest.TreeTypeCount}");

    }

}
```

Kết quả mong đợi:

Create new TreeType: Oak_Green_oak.png

Reuse existing TreeType: Oak_Green_oak.png

Create new TreeType: Pine_DarkGreen_pine.png

Reuse existing TreeType: Oak_Green_oak.png

Draw Oak tree at (10, 20) with color Green and texture oak.png

Draw Oak tree at (30, 40) with color Green and texture oak.png

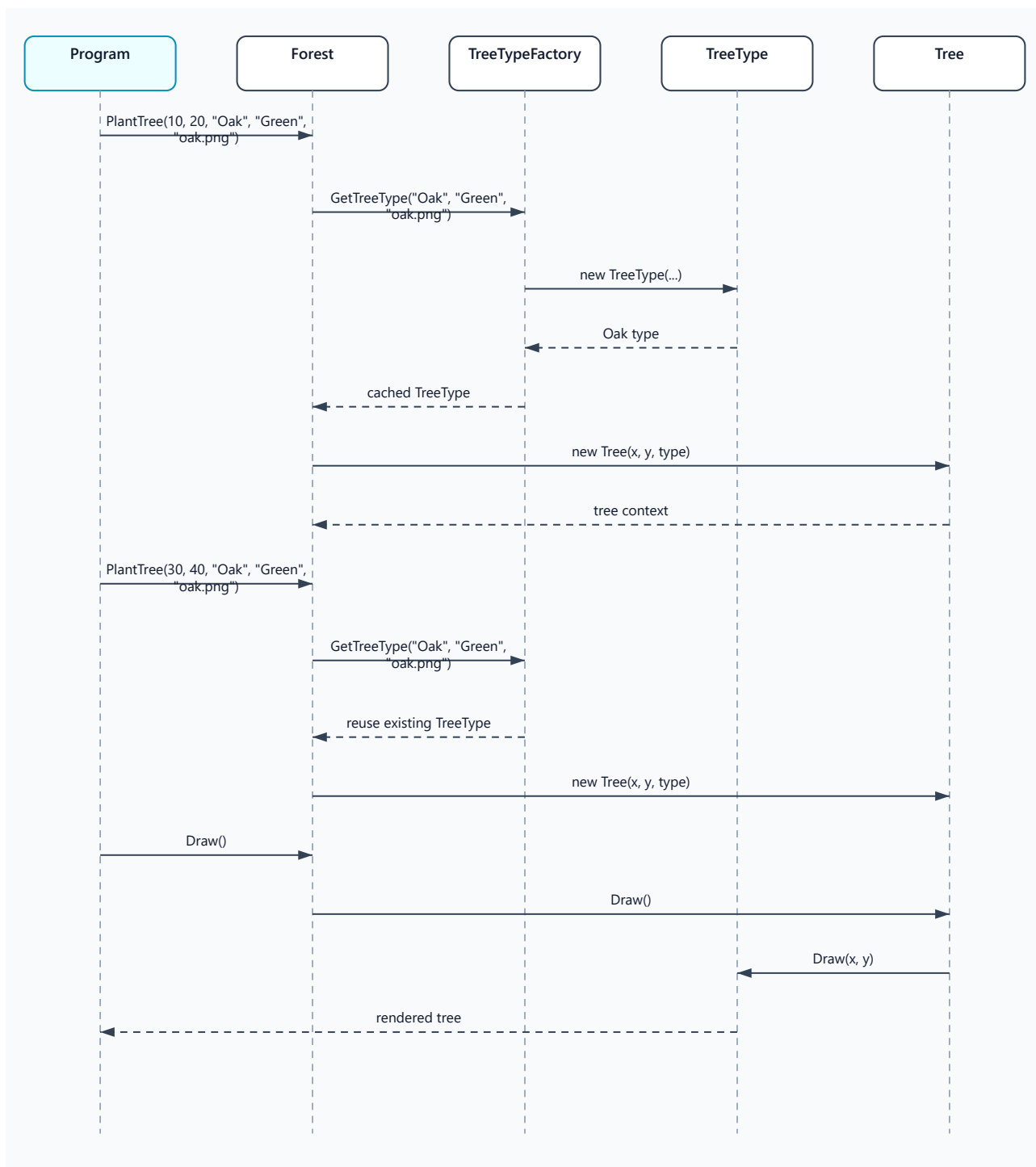
Draw Pine tree at (50, 60) with color DarkGreen and texture pine.png

Draw Oak tree at (70, 80) with color Green and texture oak.png

Total trees: 4

Total tree types: 2

Dù có 4 cây, hệ thống chỉ tạo 2 TreeType. Nếu số lượng cây tăng lên hàng chục nghìn, lợi ích tiết kiệm bộ nhớ sẽ rõ ràng hơn.



Hình 2.23: Luồng chạy code ví dụ Forest, Tree và TreeTypeFactory

2.6.13. Phân tích ví dụ

Trong đoạn code trên:

- TreeType là Flyweight, chứa intrinsic state: Name, Color, Texture.

- Tree là Context, chứa extrinsic state: `_x, _y`.
- TreeTypeFactory là Factory/cache, đảm bảo không tạo trùng TreeType.
- Forest là client/use case, quản lý danh sách cây và yêu cầu Factory cung cấp loại cây.

Điểm quan trọng nằm ở method:

```
public TreeType GetTreeType(string name, string color, string texture)
```

Method này dùng key để xác định một Flyweight đã tồn tại hay chưa. Nếu đã tồn tại, nó trả lại object cũ. Nếu chưa tồn tại, nó tạo mới và lưu vào cache.

Nhờ đó, các object Tree không cần lưu texture riêng. Chúng chỉ cần giữ vị trí riêng và tham chiếu đến TreeType dùng chung.

2.6.14. Khi nào nên dùng

Nên dùng Flyweight khi:

- Hệ thống cần tạo rất nhiều object.
- Nhiều object có phần dữ liệu giống nhau.
- Dữ liệu giống nhau chiếm nhiều bộ nhớ hoặc tốn chi phí khởi tạo.
- Phần dữ liệu dùng chung có thể tách khỏi dữ liệu riêng.
- Object dùng chung ít thay đổi hoặc có thể được xem là immutable.
- Chi phí lookup trong Factory nhỏ hơn lợi ích tiết kiệm bộ nhớ.

Một số tình huống phù hợp:

- Render ký tự trong trình soạn thảo văn bản.
- Icon/marker trên bản đồ.
- Texture, sprite, particle trong game.
- Các loại sản phẩm/category/tag dùng lặp lại trong nhiều bản ghi.
- Hệ thống vẽ đồ họa có nhiều shape cùng style.

- Cache metadata hoặc configuration nhỏ dùng chung cho nhiều entity.

2.6.15. Khi nào không nên dùng

Không nên dùng Flyweight khi:

- Số lượng object ít, không tạo áp lực bộ nhớ.
- Các object hầu như không có dữ liệu lặp lại.
- Dữ liệu riêng và dữ liệu dùng chung khó tách rõ ràng.
- Object dùng chung thường xuyên thay đổi theo từng context.
- Việc dùng Factory/cache làm code phức tạp hơn lợi ích nhận được.
- Performance bottleneck không nằm ở bộ nhớ mà nằm ở I/O, database hoặc network.

Nếu hệ thống chỉ có vài chục object đơn giản, dùng Flyweight có thể là over-engineering.

Pattern này chỉ thật sự có giá trị khi số lượng object lớn và dữ liệu trùng lặp đáng kể.

2.6.16. Ưu điểm và nhược điểm

Ưu điểm	Nhược điểm
Tiết kiệm bộ nhớ vì dữ liệu giống nhau được chia sẻ.	Tăng độ phức tạp thiết kế do phải tách intrinsic/extrinsic state.
Giảm số lượng object nặng cần tạo ra.	Khó debug hơn vì dữ liệu của một object có thể nằm ở nhiều nơi.
Tăng hiệu quả khi xử lý số lượng lớn object.	Không hiệu quả nếu object không có nhiều dữ liệu lặp lại.
Tách rõ dữ liệu dùng chung và dữ liệu riêng.	Có thể tốn thêm chi phí lookup trong Factory/cache.
Hỗ trợ cache và tái sử dụng object tốt hơn.	Nếu Flyweight bị mutable, các context dùng chung có thể ảnh hưởng lẫn nhau.

2.6.17. Lưu ý thiết kế

Khi triển khai Flyweight, cần chú ý một số điểm:

2.6.18. Flyweight nên immutable

Flyweight chứa dữ liệu dùng chung, vì vậy nó nên được thiết kế bất biến hoặc hạn chế thay đổi. Nếu một context thay đổi Flyweight, tất cả context khác đang dùng chung Flyweight đó có thể bị ảnh hưởng.

Ví dụ, nếu `TreeType.Color` bị đổi từ Green sang Red, toàn bộ cây đang dùng `TreeType` đó sẽ bị đổi màu.

2.6.19. Key của Factory phải đủ chính xác

Factory thường dùng key để xác định Flyweight. Key phải bao gồm đầy đủ các thuộc tính tạo nên intrinsic state.

Ví dụ:

```
string key = $"{name}_{color}_{texture}";
```

Nếu key thiếu texture, hai loại cây có cùng tên và màu nhưng khác texture có thể bị nhầm thành một Flyweight.

2.6.20. Không nhầm với cache thông thường

Flyweight có liên quan đến cache, nhưng không phải mọi cache đều là Flyweight. Flyweight tập trung vào việc chia sẻ intrinsic state giữa nhiều context. Cache thông thường có thể chỉ nhằm tăng tốc truy cập dữ liệu.

2.6.21. Không lưu extrinsic state trong Flyweight

Đây là lỗi phổ biến nhất. Nếu `TreeType` lưu x, y, nó không còn là dữ liệu dùng chung nữa.

Khi đó các cây dùng chung `TreeType` sẽ ghi đè vị trí lẫn nhau.

2.6.22. So sánh Flyweight và Singleton

Tiêu chí	Flyweight	Singleton
----------	-----------	-----------

Mục đích chính	Chia sẻ nhiều object nhỏ có trạng thái giống nhau để tiết kiệm bộ nhớ.	Đảm bảo một class chỉ có một instance duy nhất.
Số lượng instance	Có thể có nhiều instance, mỗi instance đại diện một loại state dùng chung.	Chỉ một instance duy nhất cho class đó.
Trọng tâm	Tiết kiệm bộ nhớ bằng cách chia sẻ intrinsic state.	Kiểm soát số lượng instance và điểm truy cập toàn cục.
Factory/cache	Thường có Factory/cache để tái sử dụng Flyweight.	Không bắt buộc có Factory/cache.
Ví dụ	TreeType cho Oak, Pine, Cherry.	ConfigurationManager, Logger, AppSettings.

Điểm dễ nhầm là cả hai đều có yếu tố dùng chung object. Tuy nhiên, Singleton chỉ nói về một instance duy nhất của một class, còn Flyweight có thể có nhiều instance dùng chung, mỗi instance đại diện cho một nhóm dữ liệu giống nhau.

2.6.23. So sánh Flyweight và Prototype

Tiêu chí	Flyweight	Prototype
Mục đích chính	Chia sẻ object để giảm bộ nhớ.	Clone object để tạo object mới nhanh hơn hoặc linh hoạt hơn.
Object được tạo ra	Dùng lại object có sẵn.	Tạo bản sao mới từ object mẫu.
Trạng thái	Tách intrinsic state và extrinsic state.	Mỗi clone thường có state riêng.
Có chia sẻ object không?	Có, đây là trọng tâm chính.	Không nhất thiết; clone thường là object độc lập.

Ví dụ	Dùng chung TreeType cho nhiều cây.	Clone một Enemy mẫu thành nhiều enemy mới.
-------	------------------------------------	--

Prototype hữu ích khi chi phí tạo object từ đầu cao và ta muốn clone từ mẫu. Flyweight hữu ích khi ta muốn nhiều context cùng dùng chung một phần dữ liệu.

2.6.24. So sánh Flyweight và Object Pool

Tiêu chí	Flyweight	Object Pool
Mục đích chính	Chia sẻ dữ liệu giống nhau để tiết kiệm bộ nhớ.	Tái sử dụng object tạm thời để tránh tạo/hủy nhiều lần.
Object dùng đồng thời	Một Flyweight có thể được nhiều context dùng cùng lúc.	Một object trong pool thường được một client mượn tại một thời điểm.
State	Flyweight thường chứa state dùng chung, ít thay đổi.	Object trong pool thường có state thay đổi và cần reset trước khi trả lại pool.
Vòng đời	Flyweight thường tồn tại lâu trong cache.	Object được mượn, dùng, reset, rồi trả lại pool.
Ví dụ	Dùng chung TreeType cho nhiều cây.	Pool connection database, pool bullet object trong game.

Object Pool giải quyết vấn đề chi phí tạo/hủy object. Flyweight giải quyết vấn đề lưu lặp lại dữ liệu giống nhau. Hai pattern có thể cùng xuất hiện trong game, nhưng mục tiêu của chúng khác nhau.

2.6.25. So sánh Flyweight và Facade

Tiêu chí	Flyweight	Facade
----------	-----------	--------

Nhóm pattern	Structural.	Structural.
Mục đích chính	Giảm bộ nhớ bằng cách chia sẻ trạng thái dùng chung.	Đơn giản hóa cách sử dụng subsystem phức tạp.
Vấn đề giải quyết	Có quá nhiều object chứa dữ liệu lặp lại.	Client phải gọi quá nhiều class/service bên trong.
Trọng tâm	Tối ưu cấu trúc dữ liệu/object.	Đơn giản hóa interface cho client.
Ví dụ	Nhiều marker bản đồ dùng chung một MarkerIconType.	CheckoutFacade gọi Payment, Inventory, Shipping.

Cả hai đều là Structural Pattern, nhưng Flyweight thiên về tối ưu bộ nhớ, còn Facade thiên về giảm độ phức tạp khi client sử dụng subsystem.

2.6.26. Ví dụ áp dụng trong app giao hàng

Trong app giao hàng, bản đồ có thể hiển thị rất nhiều marker:

- Marker shipper.
- Marker nhà hàng.
- Marker khách hàng.
- Marker đơn hàng đang xử lý.

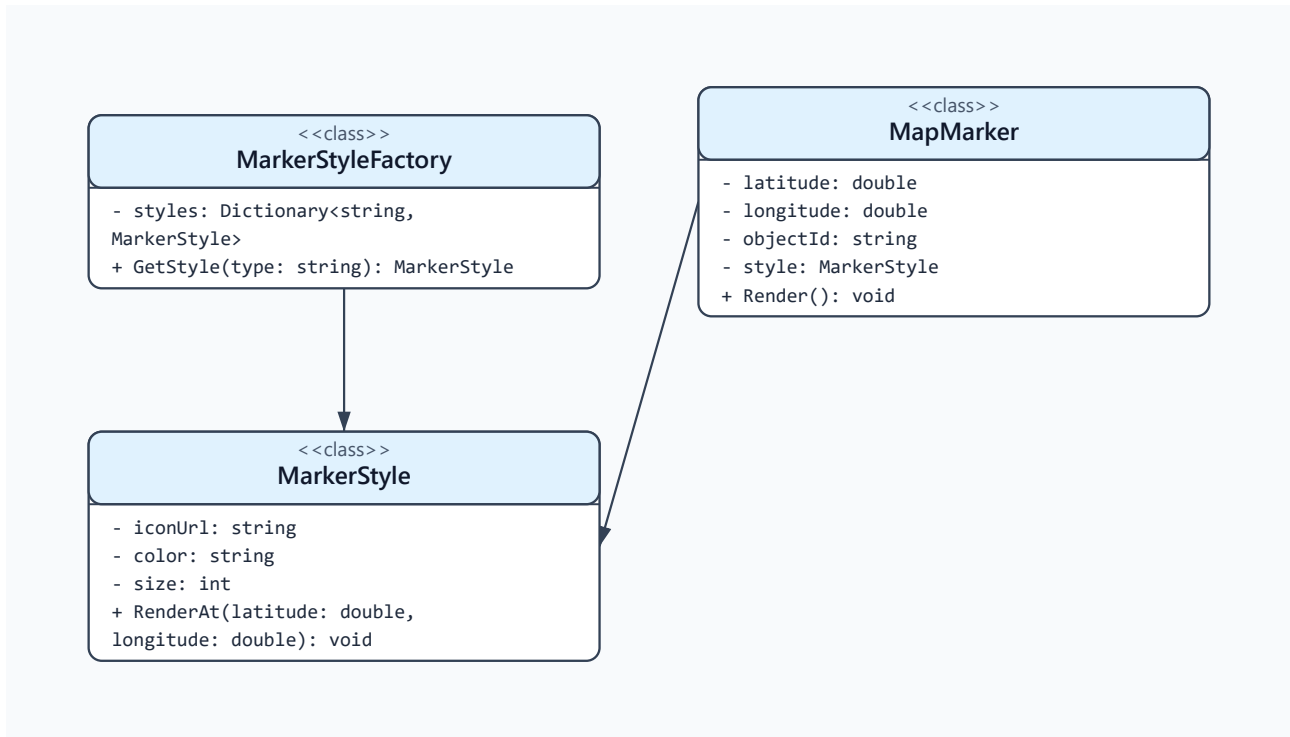
Nếu mỗi marker đều lưu riêng icon, màu, style, animation config, hệ thống có thể lãng phí bộ nhớ, đặc biệt khi bản đồ hiển thị nhiều đối tượng.

Có thể áp dụng Flyweight như sau:

- MarkerStyle là Flyweight: chứa icon, màu, kích thước, loại marker.
- MapMarker là Context: chứa vị trí latitude/longitude, id đối tượng, trạng thái realtime.
- MarkerStyleFactory quản lý các style dùng chung như ShipperAvailable, ShipperBusy, Restaurant, Customer, Order.

Khi cần hiển thị 500 shipper online, không cần tạo 500 icon/style riêng. Mỗi marker chỉ giữ vị trí và trạng thái riêng, còn style được dùng chung.

Sơ đồ ý tưởng:



Hình 2.24: Class diagram của ví dụ marker bản đồ

Cách thiết kế này giúp bản đồ nhẹ hơn và tránh lặp lại style/icon cho từng marker.

2.6.27. Tổng kết

Flyweight Pattern phù hợp khi hệ thống phải xử lý số lượng lớn object có nhiều dữ liệu giống nhau. Pattern này giúp giảm bộ nhớ bằng cách tách dữ liệu thành intrinsic state và extrinsic state.

Điểm mạnh nhất của Flyweight là khả năng chia sẻ object dùng chung thông qua Factory/cache. Tuy nhiên, pattern này cũng làm thiết kế phức tạp hơn và đòi hỏi phân tách trạng thái cẩn thận.

Có thể ghi nhớ Flyweight bằng câu sau:

Nhiều object khác nhau không nhất thiết phải lưu toàn bộ dữ liệu khác nhau.

Phần nào giống nhau thì chia sẻ, phần nào riêng thì để bên ngoài.

2.7. Interpreter (Phức)

2.7.1. Tên và Phân loại

Interpreter - Mẫu hành vi (Behavioral Pattern)

2.7.2. Mục đích, ý định

Xác định cách biểu diễn một ngôn ngữ cụ thể và xây dựng trình thông dịch để xử lý các câu trong ngôn ngữ đó.

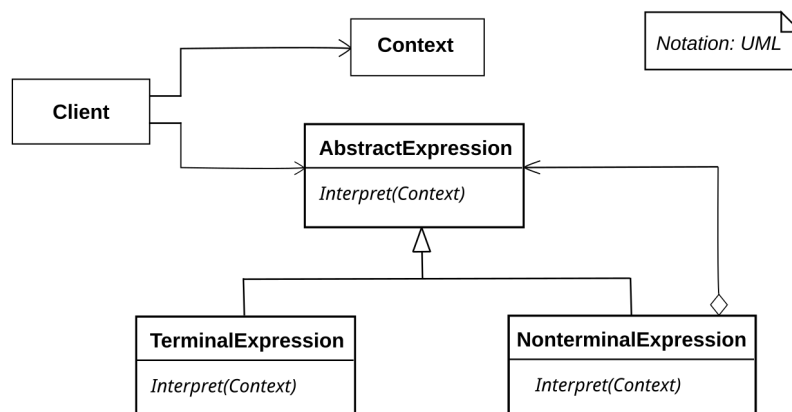
2.7.3. Motivation

Cần xử lý một ngôn ngữ hay ký pháp đặc biệt. Interpreter cho phép xây dựng một cấu trúc biểu diễn văn pháp và thực hiện các phép tính.

2.7.4. Khả năng ứng dụng

- Ngôn ngữ cần được định nghĩa có thể được biểu diễn dưới dạng cây cú pháp
- Hiệu suất không phải vấn đề chính
- Độ phức tạp của ngôn ngữ vừa phải
- Cần linh hoạt thêm câu lệnh mới

2.7.5. Cấu trúc



Hình 2.25: Cấu trúc của mẫu Interpreter

AbstractExpression khai báo phương thức interpret(). ConcreteExpression triển khai phương thức này. Các expression tổ hợp thành cây cú pháp.

2.7.6. Các thành viên

- AbstractExpression: giao diện cơ sở cho các biểu thức
- ConcreteExpression: triển khai interpret() cho terminal/nonterminal
- Context: lưu trữ thông tin toàn cục
- Client: xây dựng cây cú pháp và gọi interpret

2.7.7. Sự cộng tác

Client xây dựng cây biểu thức. Mỗi nút gọi interpret() đệ quy trên các nút con.

2.7.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Dễ thêm phép tính mới; biểu diễn ngôn ngữ linh hoạt. **Nhược điểm:** Khó thêm quy tắc ngữ pháp; hiệu suất thấp với ngôn ngữ phức tạp.

2.7.9. Chú ý liên quan đến việc cài đặt

- Xác định rõ ngữ pháp của ngôn ngữ
- Triển khai parser để xây dựng cây
- Quản lý Context cẩn thận

2.7.10. Mã nguồn minh họa

Biểu thức số học, ký pháp logic, hay các mini-language.

2.7.11. Ví dụ về các hệ thống thực tế

- Biểu thức số học trong máy tính
- SQL query interpreter
- Expression evaluator
- Regular expression engine

2.7.12. Các mẫu liên quan

Liên quan đến Composite (cây cú pháp), Visitor (xử lý cây).

2.8. Proxy (Đặt)

2.8.1. Tên và Phân loại

Proxy Pattern là một mẫu thiết kế thuộc nhóm **Structural Pattern** (nhóm cấu trúc).

Mẫu này tạo ra một đối tượng đại diện (proxy) có cùng giao diện với đối tượng thật (real subject), từ đó kiểm soát cách truy cập đến đối tượng thật.

2.8.2. Mục đích, ý định

Mục đích chính của Proxy là cung cấp một lớp trung gian để thêm kiểm soát trước/sau khi chuyển tiếp lời gọi tới đối tượng thật.

Ý định của pattern:

- Kiểm soát quyền truy cập vào đối tượng thật.
- Trì hoãn khởi tạo đối tượng nặng tài nguyên (lazy initialization).
- Tối ưu hiệu năng thông qua cache.
- Ghi log, đo thời gian, audit mà không sửa logic lõi.
- Ẩn truy cập từ xa (remote object) như truy cập cục bộ.

2.8.3. Motivation

Giả sử hệ thống có dịch vụ đọc ảnh độ phân giải cao từ kho lưu trữ từ xa. Việc tải ảnh tốn thời gian và băng thông, nhưng không phải lúc nào client cũng thật sự cần dữ liệu ảnh đầy đủ.

Nếu client gọi trực tiếp đối tượng thật:

- Mỗi lần truy cập đều tải dữ liệu nặng.
- Khó kiểm soát quyền truy cập theo vai trò người dùng.
- Logic log và cache bị lặp lại ở nhiều nơi.

Proxy giải quyết bằng cách đặt một lớp đại diện:

- Kiểm tra quyền trước khi truy cập ảnh thật.
- Cache kết quả sau lần tải đầu.

- Chỉ khởi tạo đối tượng thật khi thật sự cần.

Client vẫn dùng cùng một interface, nên có thể thay real subject bằng proxy mà không đổi mã gọi.

2.8.4. Khả năng ứng dụng

Nên dùng Proxy khi:

- Đối tượng thật nặng, khởi tạo tốn tài nguyên.
- Cần kiểm soát truy cập theo quyền.
- Cần thêm cache hoặc giới hạn tần suất gọi.
- Cần theo dõi/audit lời gọi mà không sửa đối tượng thật.
- Cần truy cập đối tượng ở máy khác nhưng muốn API cục bộ.

Tình huống thực tế:

- Ảnh/video lớn chỉ tải khi người dùng mở chi tiết.
- API client có lớp proxy để retry/throttle.
- ORM dùng proxy để lazy load quan hệ.
- Reverse proxy (Nginx) kiểm soát request trước backend.

2.8.5. Cấu trúc

Thành phần chính của Proxy Pattern:

- Subject: interface chung cho RealSubject và Proxy.
- RealSubject: đối tượng thực hiện nghiệp vụ thật.
- Proxy: giữ tham chiếu đến RealSubject, thêm logic kiểm soát.
- Client: làm việc thông qua Subject.

Sơ đồ lớp tổng quát:

classDiagram

```
class Subject {
```

```

<<interface>>

+Request()

}

```

```

class RealSubject {

+Request()

}

```

```

class Proxy {

- realSubject: RealSubject

+Request()

-CheckAccess() bool

-LogAccess()

}

```

```

Subject <|.. RealSubject

```

```

Subject <|.. Proxy

```

```

Proxy --> RealSubject : controls access

```

```

Client --> Subject

```

UML tổng quát (PlantUML):

```

@startuml

```

```

skinparam classAttributeIconSize 0

```

```

interface Subject {
    +Request()
}

class RealSubject {
    +Request()
}

class Proxy {
    -realSubject: RealSubject
    +Request()
    -CheckAccess(): bool
    -LogAccess(): void
}

```

```

class Client

```

```

Subject <|.. RealSubject

```

```

Subject <|.. Proxy

```

```

Proxy --> RealSubject

```

```

Client --> Subject

```

```

@enduml

```

Sequence cộng tác cơ bản:

sequenceDiagram

participant C as Client

participant P as Proxy

participant R as RealSubject

C->>P: Request()

P->>P: CheckAccess()

alt Access Granted

P->>R: Request()

R-->>P: result

P->>P: LogAccess()

P-->>C: result

else Access Denied

P-->>C: throw/deny

end

2.8.6. Các thành viên

Subject

- Định nghĩa giao diện mà cả proxy và real subject cùng tuân theo.
- Giúp client không phụ thuộc vào cài đặt cụ thể.

RealSubject

- Chứa nghiệp vụ cốt lõi.
- Thực thi xử lý chính khi có request hợp lệ.

Proxy

- Giữ tham chiếu tới RealSubject.
- Có thể trì hoãn khởi tạo real subject.
- Chặn/lọc request trước khi chuyển tiếp.
- Bổ sung cross-cutting concerns: logging, caching, auth, metrics.

Client

- Chỉ làm việc qua Subject.
- Không cần biết phía sau là proxy hay real subject.

2.8.7. Sự cộng tác

Luồng hoạt động điển hình:

1. Client gọi Request () thông qua interface Subject.
2. Proxy nhận request và thực hiện bước tiền xử lý (check quyền, check cache, validate input).
3. Nếu chưa có RealSubject, proxy có thể tạo mới (lazy load).
4. Proxy chuyển tiếp request sang RealSubject.
5. RealSubject xử lý nghiệp vụ và trả kết quả.
6. Proxy hậu xử lý (log, metrics, cache) rồi trả về cho client.

2.8.8. Các hệ quả mang lại, Ưu nhược điểm

2.8.9. Ưu điểm

- Kiểm soát truy cập tốt mà không chạm vào logic lõi.
- Tối ưu hiệu năng với lazy load và cache.
- Tăng khả năng mở rộng cho các concern phụ trợ (audit, tracing).
- Tuân thủ OCP: thêm proxy mới không sửa subject cũ.
- Client ít bị ảnh hưởng khi thay đổi cơ chế truy cập.

2.8.10. Nhược điểm

- Tăng số lớp và độ phức tạp cấu trúc.

- Có thêm một mức gián tiếp, có thể gây overhead nhỏ.
- Dễ tạo chuỗi proxy quá dài nếu lạm dụng.
- Khó debug hơn nếu nhiều logic ẩn trong proxy.

2.8.11. Chú ý liên quan đến việc cài đặt

- Interface của proxy phải giống real subject để thay thế minh bạch.
- Nếu proxy giữ trạng thái (cache/session), cần chú ý thread safety.
- Tránh để proxy chứa quá nhiều nghiệp vụ chính.
- Với remote proxy, cần xử lý timeout, retry, circuit breaker.
- Cần chính sách invalidate cache rõ ràng.
- Quy định rõ lỗi phát sinh ở proxy hay real subject để logging nhất quán.

2.8.12. Một số so sánh nếu có

2.8.13. Proxy và Decorator

- Cùng bọc object và cùng interface.
- Proxy tập trung **kiểm soát truy cập** hoặc **tối ưu truy cập**.
- Decorator tập trung **mở rộng hành vi** theo kiểu lắp ghép linh hoạt.

2.8.14. Proxy và Adapter

- Proxy giữ nguyên giao diện đối tượng thật.
- Adapter chuyển đổi từ giao diện này sang giao diện khác.

2.8.15. Proxy và Facade

- Facade cung cấp giao diện đơn giản cho cả subsystem.
- Proxy đại diện cho **một đối tượng cụ thể** để kiểm soát truy cập.

2.8.16. Proxy và Mediator

- Proxy điều tiết truy cập tới một đối tượng.
- Mediator điều phối tương tác giữa nhiều đối tượng đồng cấp.

2.8.17. Proxy và Flyweight

- Flyweight tối ưu bộ nhớ bằng chia sẻ trạng thái dùng chung cho nhiều object nhỏ.
- Proxy tối ưu hoặc kiểm soát **đường truy cập** đến object.

2.8.18. Proxy và Observer

- Observer lan truyền sự kiện một-nhiều qua cơ chế subscribe.
- Proxy không phát sự kiện; proxy chặn/chuyển tiếp lời gọi một cách minh bạch.

2.8.19. Các biến thể Proxy thường dùng

2.8.20. Virtual Proxy

- Mục tiêu: trì hoãn tạo object nặng.
- Dùng khi chi phí khởi tạo cao nhưng tần suất sử dụng thấp.
- Rủi ro: độ trễ cao ở lần truy cập đầu tiên (cold start).

2.8.21. Protection Proxy

- Mục tiêu: kiểm soát quyền truy cập.
- Dùng khi cần RBAC/ABAC cho từng thao tác.
- Rủi ro: logic phân quyền đặt sai chỗ có thể gây lộ quyền.

2.8.22. Remote Proxy

- Mục tiêu: đại diện đối tượng từ xa như local object.
- Dùng với RPC/gRPC/REST client wrapper.
- Rủi ro: timeout, mạng lỗi, serialization mismatch.

2.8.23. Caching (Smart) Proxy

- Mục tiêu: giảm độ trễ và giảm số lần gọi đến real subject.
- Dùng cho dữ liệu đọc nhiều, ít thay đổi.
- Rủi ro: cache stale nếu không có chiến lược invalidation rõ ràng.

2.8.24. Checklist triển khai thực tế

- Xác định rõ loại proxy cần dùng: virtual, protection, remote hay caching.

- Thiết kế Subject ổn định để client không phụ thuộc vào triển khai.
- Đảm bảo proxy và real subject tuân thủ cùng hợp đồng đầu vào/đầu ra.
- Nếu có cache: định nghĩa key, TTL, và điều kiện xóa cache.
- Nếu remote: chuẩn hóa timeout, retry, backoff và circuit breaker.
- Bổ sung observability: log tương quan, metrics latency, error rate.
- Viết test cho cả trường hợp pass-through lẫn trường hợp bị chặn tại proxy.
- Đánh giá overhead do proxy để tránh chain proxy quá sâu.

2.8.25. Lỗi thường gặp và cách tránh

- Đưa nghiệp vụ lỗi vào proxy: Proxy chỉ nên xử lý control concerns; nghiệp vụ chính để ở real subject.
- Quên đồng bộ khi proxy có trạng thái dùng chung: Dùng lock/thread-safe collection hoặc thiết kế stateless.
- Cache không invalidation: Định nghĩa chiến lược invalidation ngay từ đầu thay vì xử lý vá lỗi.
- Bắt mọi exception rồi nuốt lỗi: Cần phân biệt lỗi business và lỗi kỹ thuật, log có cấu trúc.
- Lộ khác biệt hành vi giữa proxy và real subject: Đảm bảo tính thay thế (LSP), cùng semantics cho cùng request.

2.8.26. Mã nguồn minh họa

Ví dụ C# dưới đây minh họa **Protection + Virtual Proxy**:

```
using System;
```

```
namespace ProxyDemo
```

```
{
```

```

public interface IImage
{
    void Display();
}

// RealSubject: tải ảnh nặng và hiển thị.
public class HighResolutionImage : IImage
{
    private readonly string _fileName;

    public HighResolutionImage(string fileName)
    {
        _fileName = fileName;
        LoadFromDisk();
    }

    private void LoadFromDisk()
    {
        Console.WriteLine($"Loading image '{_fileName}' from disk...");
    }

    public void Display()
    {

```

```

        Console.WriteLine($"Displaying '{_fileName}'.");
    }
}

// Proxy: kiểm tra quyền và trì hoãn tạo real object.
public class ImageProxy : IImage
{
    private readonly string _fileName;
    private readonly string _role;
    private HighResolutionImage _realImage;

    public ImageProxy(string fileName, string role)
    {
        _fileName = fileName;
        _role = role;
    }

    public void Display()
    {
        if (!HasAccess())
        {
            Console.WriteLine("Access denied: user is not allowed to view this
image.");

```

```

        return;
    }

    if (_realImage == null)
    {
        _realImage = new HighResolutionImage(_fileName);
    }

    Console.WriteLine("Proxy: access granted, forwarding request.");
    _realImage.Display();
}

private bool HasAccess()
{
    return _role == "Admin" || _role == "Editor";
}
}

public class Program
{
    public static void Main()
    {
        IImage guestImage = new ImageProxy("architecture.png", "Guest");
    }
}

```



```

        guestImage.Display();

        Console.WriteLine();

        IImage adminImage = new ImageProxy("architecture.png", "Admin");
        adminImage.Display();

        adminImage.Display(); // Lần 2 không cần load lại từ disk.
    }
}
}

```

Kết quả dự kiến:

Access denied: user is not allowed to view this image.

Loading image 'architecture.png' from disk...

Proxy: access granted, forwarding request.

Displaying 'architecture.png'.

Proxy: access granted, forwarding request.

Displaying 'architecture.png'.

Nhận xét:

- Người dùng Guest bị chặn tại proxy.
- Đối tượng ảnh thật chỉ tạo khi lần đầu truy cập hợp lệ.
- Lần gọi thứ hai không tải lại ảnh, thể hiện lợi ích lazy initialization.

2.8.27. Ví dụ về các hệ thống thực tế

- ORM như Hibernate/Entity Framework dùng proxy để lazy loading entity liên quan.
- AOP proxy trong Spring/.NET để chèn transaction, logging, security.
- Reverse proxy (Nginx, HAProxy) đứng trước service để route, auth, cache.
- HTTP client wrapper thêm retry/circuit breaker cho gọi API.
- CDN edge proxy cache nội dung tĩnh trước khi truy cập origin.

2.8.28. Các mẫu liên quan

- **Decorator**: cùng dạng wrapper, khác mục tiêu thiết kế.
- **Adapter**: chuyển giao diện, thường xuất hiện trước/sau proxy trong integration layer.
- **Facade**: có thể dùng cùng proxy để vừa đơn giản hóa vừa kiểm soát truy cập.
- **Bridge**: tách abstraction-implementation; proxy có thể bọc phía implementation.
- **Chain of Responsibility**: có thể kết hợp nhiều proxy/filter dạng pipeline xử lý request.

Tóm lại, Proxy Pattern đặc biệt hữu ích khi hệ thống cần kiểm soát truy cập, tối ưu hiệu năng hoặc ẩn độ phức tạp giao tiếp từ xa mà vẫn giữ API ổn định cho client. Nếu dùng đúng ranh giới trách nhiệm, Proxy giúp mở rộng hệ thống sạch và an toàn hơn mà không làm rối nghiệp vụ cốt lõi.

Chương 3. Các mẫu hành vi

3.1. Chain of Responsibility (Phát)

3.1.1. Tên và phân loại

Chain of Responsibility Pattern thường được viết tắt là **CoR**. Trong tiếng Việt, có thể hiểu là **mẫu chuỗi trách nhiệm**.

Chain of Responsibility thuộc nhóm **Behavioral Pattern** vì trọng tâm của nó nằm ở cách các object phối hợp để xử lý một request. Pattern này không tập trung vào việc tổ chức cấu trúc lớp như Adapter hay Facade, mà tập trung vào **luồng xử lý hành vi** giữa nhiều đối tượng.

Ý nghĩa của tên gọi “Chain of Responsibility” là: trách nhiệm xử lý request không bị gán cứng vào một object duy nhất, mà được đặt vào một chuỗi các handler. Mỗi handler trong chuỗi có quyền xử lý request, từ chối request, hoặc chuyển tiếp request cho handler kế tiếp.

3.1.2. Vấn đề cần giải quyết

Trong nhiều hệ thống thực tế, một request trước khi được xử lý chính thức thường phải đi qua nhiều bước khác nhau. Ví dụ trong một web API, một request có thể cần đi qua các bước:

- Ghi log request để phục vụ audit hoặc debug.
- Kiểm tra authentication để xác định người dùng đã đăng nhập hay chưa.
- Kiểm tra authorization để xác định người dùng có quyền thực hiện thao tác hay không.
- Validate dữ liệu đầu vào.
- Kiểm tra rate limit, cache, transaction hoặc các điều kiện nghiệp vụ phụ.
- Cuối cùng mới thực hiện business logic chính.

Nếu viết toàn bộ logic này trong một hàm duy nhất, code rất dễ biến thành một khối điều kiện lồng nhau:

```

if (authenticated)
{
    if (authorized)
    {
        if (valid)
        {
            Process();
        }
    }
}

```

Cách viết này có một số vấn đề lớn:

- Hàm xử lý chính bị phình to vì chứa quá nhiều trách nhiệm.
- Mỗi lần thêm một bước xử lý mới, ta phải sửa code cũ.
- Thứ tự xử lý bị gắn cứng vào một nơi, khó thay đổi linh hoạt.
- Các bước như logging, authentication, validation khó tái sử dụng ở nhiều flow khác.
- Code khó test vì nhiều điều kiện bị lồng vào nhau.
- Debug phức tạp khi request có nhiều nhánh xử lý.

Vấn đề cốt lõi là: làm sao để một request có thể đi qua nhiều bước xử lý khác nhau mà client không cần biết chính xác object nào sẽ xử lý, thứ tự xử lý có thể thay đổi, và mỗi bước vẫn giữ trách nhiệm riêng biệt?

Chain of Responsibility giải quyết vấn đề này bằng cách tách từng bước xử lý thành một handler độc lập, sau đó nối các handler lại thành một chuỗi.

3.1.3. Mục đích và ý định

Mục đích chính của Chain of Responsibility là **tránh việc sender phụ thuộc trực tiếp vào receiver cụ thể** bằng cách cho nhiều object trong một chuỗi có cơ hội xử lý request.

Client chỉ gửi request vào đầu chuỗi. Sau đó, request có thể:

- Được một handler xử lý rồi dừng lại.
- Được một handler xử lý một phần rồi chuyển tiếp.
- Đi qua nhiều handler cho đến handler cuối cùng.
- Bị chặn lại nếu một handler phát hiện request không hợp lệ.
- Không được handler nào xử lý nếu chain không có handler phù hợp.

Pattern này đặc biệt phù hợp với các pipeline xử lý như middleware, validation pipeline, approval workflow, logging pipeline, authentication flow và event processing.

3.1.4. Định nghĩa

Chain of Responsibility Pattern là một behavioral design pattern cho phép truyền một request qua một chuỗi các handler. Mỗi handler quyết định sẽ xử lý request, chuyển tiếp request cho handler kế tiếp, hoặc dừng chuỗi xử lý.

Định nghĩa ngắn gọn:

- Client gửi request vào chain.
- Handler định nghĩa interface chung cho các object xử lý request.
- ConcreteHandler là các handler cụ thể, mỗi handler phụ trách một bước hoặc một điều kiện xử lý.
- Mỗi handler thường giữ tham chiếu đến handler kế tiếp.
- Sender không cần biết handler cụ thể nào sẽ xử lý request.

Điểm quan trọng của pattern này là **tách sender khỏi receiver**. Client không gọi trực tiếp AuthenticationHandler, AuthorizationHandler, ValidationHandler hay BusinessHandler. Client chỉ cần biết điểm đầu của chain.

3.1.5. Ý tưởng cốt lõi

Ý tưởng chính của Chain of Responsibility là biến một quy trình xử lý nhiều bước thành một chuỗi các object độc lập.

Mỗi handler thường thực hiện ba việc:

1. Nhận request.
2. Xử lý phần trách nhiệm của nó.
3. Quyết định chuyển request cho handler kế tiếp hay dừng chain.

Có thể hình dung request đi qua chain như sau:

Request -> Logging -> Authentication -> Authorization -> Validation ->
Business Logic

Mỗi handler chỉ biết handler kế tiếp, không cần biết toàn bộ chain phía sau. Điều này giúp chain có thể được thay đổi linh hoạt: thêm handler mới, bỏ handler cũ, đổi thứ tự handler, hoặc dùng chain khác cho từng loại request.

Ví dụ:

- Với API public, chain có thể chỉ gồm LoggingHandler -> RateLimitHandler -> BusinessHandler.
- Với API cần đăng nhập, chain có thể là LoggingHandler -> AuthenticationHandler -> AuthorizationHandler -> ValidationHandler -> BusinessHandler.
- Với luồng phê duyệt đơn nghỉ phép, chain có thể là TeamLeadApproval -> ManagerApproval -> DirectorApproval.

3.1.6. Thành phần chính

Một cấu trúc Chain of Responsibility điển hình gồm ba thành phần chính.

3.1.7. Handler

Handler là interface hoặc abstract class chung cho các handler trong chain. Nó thường định nghĩa:

- Phương thức để thiết lập handler kế tiếp, ví dụ `SetNext()`.
- Phương thức xử lý request, ví dụ `Handle()`.

Ví dụ:

```
public interface IHandler
{
    IHandler SetNext(IHandler next);

    Task HandleAsync(RequestContext context);
}
```

Handler giúp client và các handler cụ thể làm việc thông qua một hợp đồng chung, không phụ thuộc vào class cụ thể.

3.1.8. BaseHandler

Trong nhiều implementation, ta thường tạo thêm một abstract class `BaseHandler` để chứa logic chuyên tiếp dùng chung. Thành phần này không bắt buộc, nhưng rất hữu ích vì tránh lặp code ở các handler cụ thể.

Ví dụ:

```
public abstract class BaseHandler : IHandler
{
    private IHandler? _next;
```

```

public IHandler SetNext(IHandler next)
{
    _next = next;
    return next;
}

public virtual async Task HandleAsync(RequestContext context)
{
    if (_next != null)
    {
        await _next.HandleAsync(context);
    }
}
}

```

Với cách này, các concrete handler chỉ cần tập trung vào logic riêng, còn việc gọi handler kế tiếp có thể dùng lại từ BaseHandler.

3.1.9. Concrete Handler

ConcreteHandler là các handler cụ thể trong chain. Mỗi handler nên đảm nhận một trách nhiệm rõ ràng.

Ví dụ trong web API:

- LoggingHandler: ghi log request.
- AuthenticationHandler: kiểm tra người dùng đã đăng nhập chưa.
- AuthorizationHandler: kiểm tra quyền truy cập.

- `ValidationHandler`: kiểm tra dữ liệu đầu vào.
- `BusinessHandler`: thực hiện nghiệp vụ chính.

Một handler có thể xử lý theo hai kiểu:

- **Handle and forward**: xử lý phần của mình rồi chuyển tiếp.
- **Handle or stop**: nếu điều kiện không đạt thì dừng chain.

3.1.10. Client

Client là nơi tạo chain và gửi request vào chain. Client có thể cấu hình thứ tự các handler:

logging

```
.SetNext(authentication)

.SetNext(authorization)

.SetNext(validation)

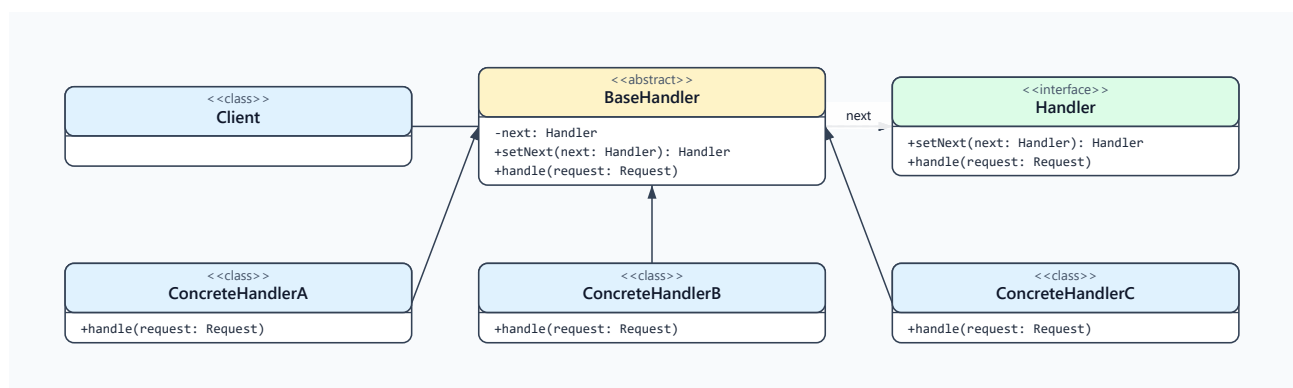
.SetNext(business);
```

```
await logging.HandleAsync(context);
```

Client không cần biết chi tiết bên trong từng handler. Nó chỉ cần biết handler đầu tiên của chain.

3.1.11. Cấu trúc UML

Dưới đây là class diagram tổng quát bằng PlantUML:



Hình 3.26: Cấu trúc UML tổng quát của Chain of Responsibility

Trong sơ đồ này:

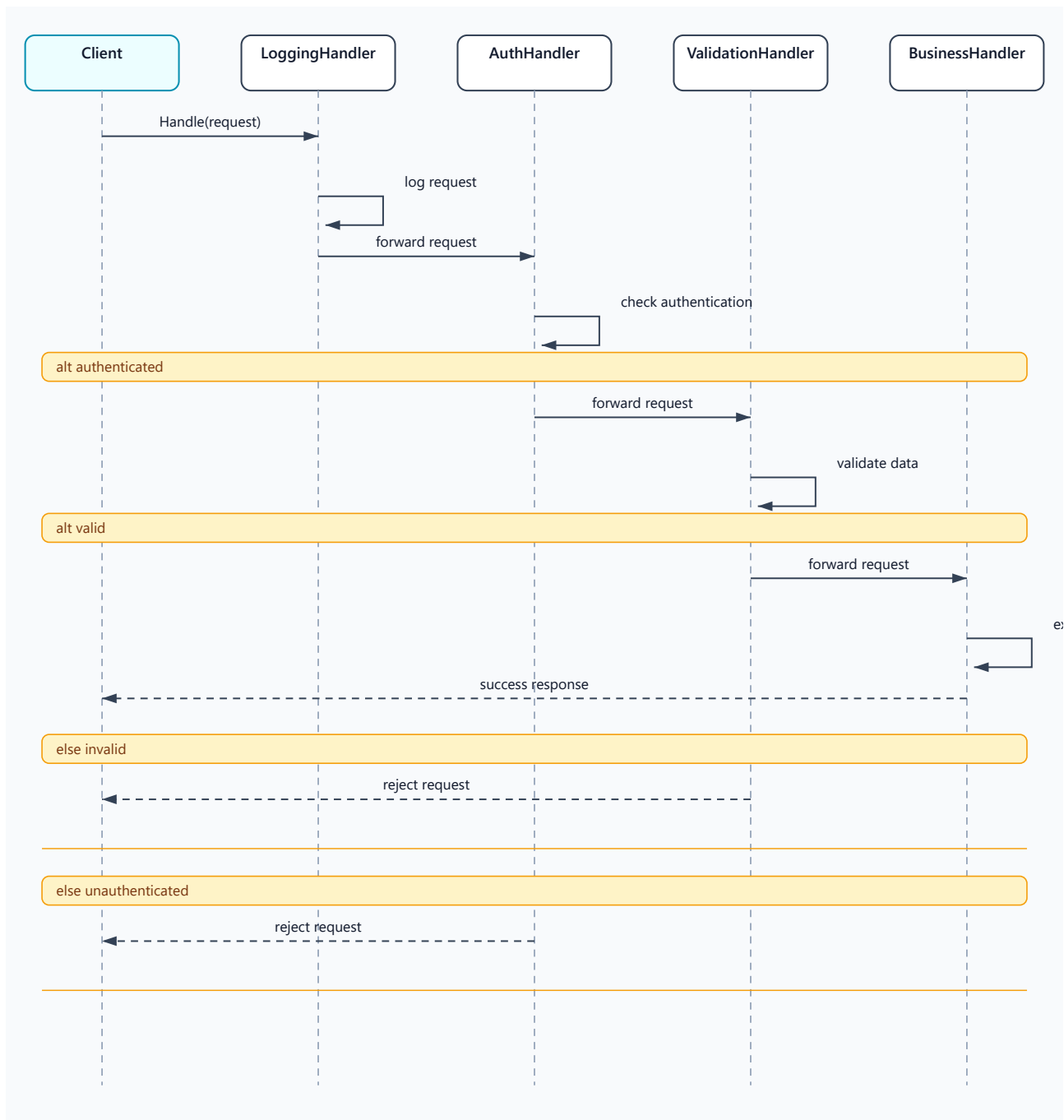
- Handler định nghĩa hợp đồng xử lý chung.
- BaseHandler giữ tham chiếu đến handler kế tiếp.
- ConcreteHandlerA/B/C kế thừa BaseHandler và override logic xử lý.
- Client chỉ gửi request vào handler đầu tiên.

3.1.12. Luồng hoạt động

Luồng hoạt động cơ bản của Chain of Responsibility gồm các bước:

1. Client tạo request.
2. Client gửi request cho handler đầu tiên trong chain.
3. Handler đầu tiên xử lý phần trách nhiệm của nó.
4. Nếu request không hợp lệ hoặc đã được xử lý xong, handler có thể dừng chain.
5. Nếu request vẫn cần tiếp tục, handler chuyển request cho handler kế tiếp.
6. Các handler tiếp theo lặp lại quá trình tương tự.
7. Chain kết thúc khi request được xử lý xong, bị reject, hoặc không còn handler kế tiếp.

Sequence diagram minh họa:



Hình 3.27: Luồng request qua các handler trong chain

Điểm quan trọng trong luồng này là mỗi handler có quyền quyết định chain có tiếp tục hay không.

3.1.13. Các biến thể xử lý

Chain of Responsibility có thể được triển khai theo nhiều biến thể khác nhau.

3.1.14. Một handler xử lý rồi dừng

Trong biến thể này, request đi qua chain cho đến khi gặp handler phù hợp. Handler đó xử lý request và không chuyển tiếp nữa.

Ví dụ: hệ thống hỗ trợ khách hàng tự động:

Request -> BasicSupport -> TechnicalSupport -> BillingSupport

Nếu TechnicalSupport xử lý được ticket, request dừng tại đó.

3.1.15. Nhiều handler cùng xử lý

Trong biến thể này, nhiều handler có thể cùng xử lý một request. Mỗi handler xử lý một phần rồi chuyển tiếp.

Ví dụ middleware trong web framework:

Logging -> Authentication -> Authorization -> Validation -> Controller

Logging ghi log, Authentication xác thực, Authorization kiểm tra quyền, Validation kiểm tra dữ liệu, sau đó Controller xử lý nghiệp vụ.

3.1.16. Handler có thể chặn request

Một số handler đóng vai trò kiểm soát. Nếu request không đạt điều kiện, handler dừng chain và trả lỗi.

Ví dụ:

- AuthenticationHandler dừng chain nếu user chưa đăng nhập.
- AuthorizationHandler dừng chain nếu user không có quyền.
- ValidationHandler dừng chain nếu dữ liệu sai.
- RateLimitHandler dừng chain nếu request vượt giới hạn.

3.1.17. Dynamic Chain

Trong một số hệ thống, chain có thể được cấu hình runtime thay vì hard-code. Ví dụ:

- Admin bật/tắt một rule validation.

- Hệ thống chọn chain khác nhau theo loại request.
- Pipeline được cấu hình bằng dependency injection.
- Workflow phê duyệt thay đổi theo số tiền hoặc phòng ban.

Dynamic chain giúp hệ thống linh hoạt, nhưng cũng làm việc trace và debug khó hơn.

3.1.18. Ví dụ minh họa bằng C#

Ví dụ dưới đây mô phỏng pipeline xử lý request gồm logging, authentication, authorization, validation và business logic.

3.1.19. Request context

```
public class RequestContext
{
    public string UserName { get; set; } = string.Empty;
    public bool IsAuthenticated { get; set; }
    public bool HasPermission { get; set; }
    public string Payload { get; set; } = string.Empty;
    public bool IsRejected { get; set; }
    public string? ErrorMessage { get; set; }
}
```

3.1.20. Handler interface và BaseHandler

```
public interface IRequestHandler
{
    IRequestHandler SetNext(IRequestHandler next);
    Task HandleAsync(RequestContext context);
}
```

```

public abstract class BaseRequestHandler : IRequestHandler
{
    private IRequestHandler? _next;

    public IRequestHandler SetNext(IRequestHandler next)
    {
        _next = next;
        return next;
    }

    public virtual async Task HandleAsync(RequestContext context)
    {
        if (_next != null && !context.IsRejected)
        {
            await _next.HandleAsync(context);
        }
    }
}

```

3.1.21. Các concrete handler

```

public class LoggingHandler : BaseRequestHandler
{
    public override async Task HandleAsync(RequestContext context)
    {

```

```

        Console.WriteLine($"[LOG] User: {context.UserName}, Payload:
{context.Payload}");

        await base.HandleAsync(context);
    }
}

```

```

public class AuthenticationHandler : BaseRequestHandler
{
    public override async Task HandleAsync(RequestContext context)
    {
        if (!context.IsAuthenticated)
        {
            context.IsRejected = true;
            context.ErrorMessage = "User is not authenticated.";
            return;
        }

        await base.HandleAsync(context);
    }
}

```

```

public class AuthorizationHandler : BaseRequestHandler
{

```

```

public override async Task HandleAsync(RequestContext context)
{
    if (!context.HasPermission)
    {
        context.IsRejected = true;
        context.ErrorMessage = "User does not have permission.";
        return;
    }

    await base.HandleAsync(context);
}
}

```

```

public class ValidationHandler : BaseRequestHandler
{
    public override async Task HandleAsync(RequestContext context)
    {
        if (string.IsNullOrEmpty(context.Payload))
        {
            context.IsRejected = true;
            context.ErrorMessage = "Payload must not be empty.";
            return;
        }
    }
}

```



```

        await base.HandleAsync(context);
    }
}

```

```

public class BusinessHandler : BaseRequestHandler
{
    public override Task HandleAsync(RequestContext context)
    {
        Console.WriteLine("Business logic executed successfully.");
        return Task.CompletedTask;
    }
}

```

3.1.22. Sử dụng chain

```

var logging = new LoggingHandler();
var authentication = new AuthenticationHandler();
var authorization = new AuthorizationHandler();
var validation = new ValidationHandler();
var business = new BusinessHandler();

```

```

logging
    .SetNext(authentication)
    .SetNext(authorization)

```

```

        .SetNext(validation)

        .SetNext(business);

var context = new RequestContext
{
    UserName = "phat",
    IsAuthenticated = true,
    HasPermission = true,
    Payload = "Create order"
};

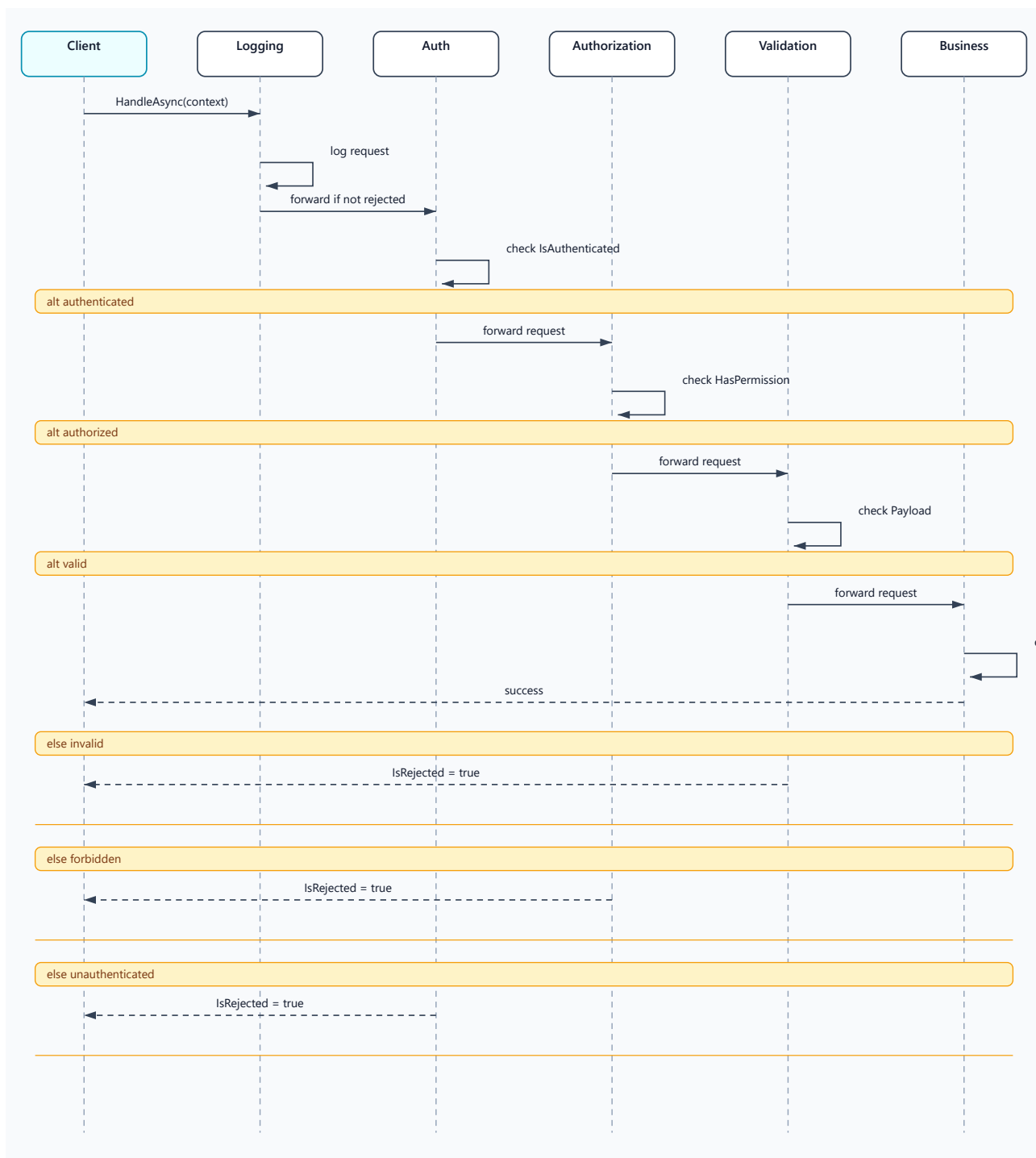
await logging.HandleAsync(context);

if (context.IsRejected)
{
    Console.WriteLine($"Request rejected: {context.ErrorMessage}");
}

```

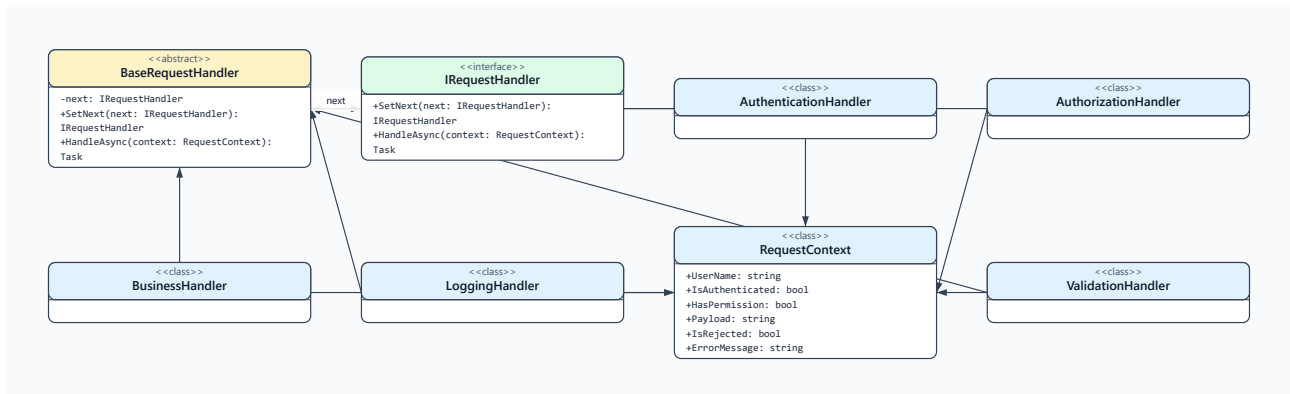
Ở ví dụ này:

- Client chỉ gọi `logging.HandleAsync(context)`.
- `LoggingHandler` luôn ghi log rồi chuyển tiếp.
- `AuthenticationHandler`, `AuthorizationHandler`, `ValidationHandler` có thể dừng chain nếu request không hợp lệ.
- `BusinessHandler` chỉ chạy khi request đã vượt qua các bước trước đó.



Hình 3.28: Luồng chạy code ví dụ request pipeline

3.1.23. Class diagram cho ví dụ C#



Hình 3.29: Class diagram của pipeline xử lý request

3.1.24. Đánh giá

3.1.25. Ưu điểm

Loose Coupling. Sender không cần biết object nào sẽ xử lý request. Client chỉ gửi request vào đầu chain, còn việc handler nào xử lý là chi tiết bên trong.

Dễ mở rộng. Khi cần thêm một bước xử lý mới, ta có thể tạo handler mới và gắn vào chain mà không cần sửa nhiều code cũ. Điều này phù hợp với Open/Closed Principle.

Single Responsibility Principle. Mỗi handler chỉ đảm nhận một nhiệm vụ rõ ràng, chẳng hạn logging, authentication, validation hoặc business logic.

Reusable Handlers. Handler có thể được tái sử dụng ở nhiều chain khác nhau. Ví dụ `LoggingHandler` hoặc `ValidationHandler` có thể dùng cho nhiều loại request.

Dynamic Chain. Thứ tự handler có thể thay đổi ở runtime hoặc thông qua cấu hình, giúp hệ thống linh hoạt hơn.

Dễ test từng bước. Vì mỗi handler độc lập, ta có thể unit test từng handler thay vì phải test một hàm lớn chứa nhiều điều kiện lồng nhau.

3.1.26. Nhược điểm

Debug khó hơn. Request đi qua nhiều object nên việc theo dõi flow có thể khó hơn so với một hàm xử lý trực tiếp.

Khó trace request trong dynamic chain. Nếu chain được cấu hình runtime, developer cần công cụ logging/tracing tốt để biết request đã đi qua những handler nào.

Không đảm bảo request được xử lý. Nếu chain không có handler phù hợp hoặc handler cuối không xử lý, request có thể kết thúc mà không có kết quả mong muốn.

Performance overhead. Nếu chain quá dài, request phải đi qua nhiều handler, gây thêm chi phí gọi hàm và xử lý trung gian.

Có thể bị lạm dụng. Với logic đơn giản, dùng Chain of Responsibility có thể làm hệ thống phức tạp không cần thiết.

3.1.27. Khi nào nên dùng

Nên dùng Chain of Responsibility khi:

- Một request cần đi qua nhiều bước xử lý độc lập.
- Có nhiều object có thể xử lý request, nhưng sender không nên biết object cụ thể nào xử lý.
- Thứ tự xử lý có thể thay đổi hoặc cần cấu hình linh hoạt.
- Cần tách các bước như logging, authentication, authorization, validation, business logic.
- Cần xây dựng middleware pipeline.
- Cần xây dựng approval workflow theo cấp bậc.
- Cần xử lý event qua nhiều bộ lọc hoặc nhiều rule.
- Muốn tránh code nhiều if/else lồng nhau trong một hàm lớn.

Ví dụ phù hợp:

- Middleware trong ASP.NET Core, Express.js, Spring Security.
- Pipeline validate request trong backend service.
- Workflow phê duyệt đơn hàng, đơn nghỉ phép, khoản vay.
- Logging pipeline.

- Event processing pipeline.
- Rule engine đơn giản.

3.1.28. Khi nào không nên dùng

Không nên dùng Chain of Responsibility khi:

- Logic xử lý rất đơn giản và chỉ có một hoặc hai bước cố định.
- Flow xử lý không có khả năng mở rộng trong tương lai.
- Cần biết chắc chắn một request luôn được xử lý bởi một object cụ thể.
- Việc trace/debug quan trọng hơn khả năng linh hoạt.
- Handler có phụ thuộc chặt vào nhau, khiến việc tách thành chain không tự nhiên.
- Thứ tự xử lý quá phức tạp và có nhiều nhánh rẽ, khi đó state machine hoặc workflow engine có thể phù hợp hơn.

3.1.29. Lưu ý khi cài đặt

Một số lưu ý khi triển khai Chain of Responsibility:

- Mỗi handler nên có một trách nhiệm rõ ràng.
- Không nên để handler biết quá nhiều về toàn bộ chain.
- Nên có logging hoặc tracing để biết request đã đi qua handler nào.
- Cần quyết định rõ handler sẽ “xử lý rồi dừng” hay “xử lý rồi chuyển tiếp”.
- Nên có handler cuối cùng để xử lý trường hợp không có handler nào phù hợp.
- Tránh chain quá dài nếu không cần thiết.
- Với hệ thống lớn, có thể kết hợp Dependency Injection để đăng ký handler.
- Với request có kết quả trả về, cần thiết kế response model rõ ràng để tránh handler trả kết quả không nhất quán.

3.1.30. Ví dụ thực tế

3.1.31. Middleware trong web framework

Đây là ví dụ rất phổ biến của Chain of Responsibility. Request đi qua nhiều middleware trước khi đến controller:

HTTP Request

- > Exception Handling Middleware
- > Logging Middleware
- > Authentication Middleware
- > Authorization Middleware
- > Validation Middleware
- > Controller / Endpoint

Mỗi middleware có thể xử lý request, thêm thông tin vào context, hoặc dừng request và trả response.

3.1.32. Approval workflow

Trong hệ thống phê duyệt chi phí:

Expense Request -> Team Lead -> Manager -> Director -> CFO

Nếu số tiền nhỏ, Team Lead có thể duyệt. Nếu số tiền lớn hơn, request được chuyển lên Manager hoặc Director. Sender không cần biết cấp nào sẽ duyệt cuối cùng.

3.1.33. Validation pipeline

Một request tạo đơn hàng có thể đi qua nhiều validator:

CreateOrderRequest

- > CustomerValidator
- > AddressValidator
- > PaymentValidator

-> InventoryValidator

-> ShippingValidator

Mỗi validator kiểm tra một phần riêng. Nếu có lỗi, chain dừng và trả lỗi cho client.

3.1.34. So sánh với các pattern khác

3.1.35. Chain of Responsibility và Decorator

Tiêu chí	Chain of Responsibility	Decorator
Mục đích	Xử lý request theo một chuỗi handler	Bọc object để thêm hành vi mới
Luồng xử lý	Có thể dừng chain ở giữa	Thường đi qua toàn bộ wrapper
Trọng tâm	Request flow	Object enhancement
Quan hệ	Handler biết handler kế tiếp	Decorator giữ object gốc hoặc decorator khác
Ví dụ	Authentication -> Authorization -> Validation	LoggingPaymentGateway bọc PaymentGateway để thêm logging

Điểm dễ nhầm là cả hai đều có thể tạo thành chuỗi object. Tuy nhiên, Decorator tập trung vào việc mở rộng hành vi của một object mà vẫn giữ interface giống object gốc, còn Chain of Responsibility tập trung vào việc truyền request qua nhiều handler và có thể dừng ở bất kỳ handler nào.

3.1.36. Chain of Responsibility và Observer

Tiêu chí	Chain of Responsibility	Observer
Mục đích	Truyền request qua chuỗi xử lý	Thông báo sự kiện đến nhiều observer
Cách lan truyền	Tuần tự, one-by-one	Broadcast, one-to-many

Có thể dừng không?	Có, handler có thể stop chain	Thường không, các observer độc lập nhận event
Trọng tâm	Request propagation	Event notification
Ví dụ	Middleware pipeline	UI event listener, domain event subscriber

Observer phù hợp khi một event xảy ra và nhiều object cần được thông báo. Chain of Responsibility phù hợp khi một request cần đi qua các bước xử lý tuần tự.

3.1.37. Chain of Responsibility và Command

Tiêu chí	Chain of Responsibility	Command
Mục đích	Route request qua nhiều handler	Đóng gói một hành động thành object
Số object xử lý	Có thể nhiều handler	Thường một command đại diện cho một thao tác
Trọng tâm	Flow xử lý	Operation/action
Kết quả	Request có thể bị xử lý, chuyển tiếp hoặc dừng	Command được execute, undo hoặc queue
Ví dụ	Validation chain	CopyCommand, SaveOrderCommand

Command biến một hành động thành object để có thể execute, undo, queue hoặc log. Chain of Responsibility không đóng gói một hành động duy nhất, mà tổ chức nhiều handler để xử lý một request.

3.1.38. Chain of Responsibility và Pipeline/Middleware

Pipeline hoặc middleware có thể xem là một ứng dụng thực tế rất phổ biến của Chain of Responsibility.

Điểm giống:

- Request đi qua nhiều bước xử lý.
- Mỗi bước có thể xử lý một phần request.
- Một bước có thể quyết định gọi bước tiếp theo hay dừng lại.

Điểm khác:

- Chain of Responsibility là pattern tổng quát.
- Middleware/Pipeline là cách triển khai cụ thể trong framework hoặc hệ thống backend.
- Middleware thường có context, response và cơ chế gọi `next()` rõ ràng hơn.

3.1.39. Quan hệ với nguyên lý thiết kế

Chain of Responsibility hỗ trợ một số nguyên lý thiết kế quan trọng:

Single Responsibility Principle. Mỗi handler chỉ phụ trách một phần xử lý.

Open/Closed Principle. Có thể thêm handler mới mà không cần sửa logic của handler cũ.

Dependency Inversion Principle. Client có thể phụ thuộc vào interface `Handler` thay vì phụ thuộc vào các concrete handler.

Tuy nhiên, pattern này cũng cần được dùng cẩn thận. Nếu lạm dụng, hệ thống có thể có quá nhiều class nhỏ, flow khó theo dõi, và việc debug trở nên phức tạp.

3.1.40. Tóm tắt

Chain of Responsibility là pattern phù hợp khi một request cần được xử lý qua nhiều bước, nhiều rule hoặc nhiều cấp trách nhiệm. Thay vì viết tất cả logic trong một hàm lớn với nhiều `if/else`, ta tách từng bước thành handler riêng và nối chúng lại thành chain.

Pattern này giúp giảm coupling giữa sender và receiver, tăng khả năng mở rộng, hỗ trợ tái sử dụng handler và làm code rõ trách nhiệm hơn. Đổi lại, nó có thể làm việc debug khó hơn, đặc biệt khi chain được cấu hình động hoặc có quá nhiều handler.

Có thể ghi nhớ ngắn gọn:

Chain of Responsibility = Request đi qua chuỗi handler,

mỗi handler xử lý một phần hoặc quyết định có chuyển tiếp hay không.

3.2. Command (Đặt)

3.2.1. Tên và Phân loại

Command Pattern là một mẫu thiết kế thuộc nhóm **Behavioral Pattern**.

Trong tiếng Việt, pattern này thường được gọi là **mẫu lệnh**. Ý tưởng trọng tâm là đóng gói một yêu cầu (request) thành một object độc lập, gọi là command. Khi đó, thao tác thực thi có thể được truyền đi, lưu lại, xếp hàng, hoãn tác hoặc thực hiện trễ (deferred execution).

3.2.2. Mục đích, ý định

Mục đích chính của Command là **biến một hành động thành object** để tách nơi phát sinh yêu cầu khỏi nơi xử lý yêu cầu.

Ý định cụ thể:

- Tách Invoker (nơi gọi lệnh) khỏi Receiver (nơi thực thi nghiệp vụ).
- Cho phép tham số hóa object bằng thao tác.
- Dễ hỗ trợ undo/redo.
- Dễ log, queue, replay thao tác.
- Dễ ghép macro command (một lệnh gồm nhiều lệnh con).

3.2.3. Motivation

Xét một ứng dụng editor có toolbar và shortcut bàn phím:

- Nút Save.
- Nút Copy/Paste.
- Nút Undo/Redo.
- Menu macro “Format + Save + Export”.

Nếu gắn trực tiếp xử lý nghiệp vụ vào từng nút, hệ thống nhanh chóng rối:

- UI phụ thuộc chặt vào class nghiệp vụ.
- Khó tái sử dụng hành động cho nhiều trigger khác nhau (button, hotkey, context menu).
- Khó thêm undo/redo vì không có đơn vị thao tác độc lập.
- Khó queue/retry thao tác trong môi trường bất đồng bộ.

Command giải quyết bằng cách gom mỗi thao tác thành một object có giao diện chung như `Execute()` (và có thể thêm `Undo()`). UI chỉ gọi command, còn logic thực thi nằm ở receiver.

3.2.4. Khả năng ứng dụng

Command phù hợp khi:

- Cần tách UI layer khỏi business logic.
- Cần hỗ trợ undo/redo.
- Cần queue command để xử lý async/background.
- Cần log và replay thao tác.
- Cần tạo macro từ nhiều thao tác nhỏ.
- Cần truyền thao tác qua network hoặc message bus.

Ví dụ điển hình:

- Text editor (Copy/Cut/Paste/Undo).
- CAD/design tools (move/rotate/resize with undo).
- Job scheduler (enqueue command chạy theo lịch).
- CQRS write model (command object đại diện ý định thay đổi trạng thái).

3.2.5. Cấu trúc

Command Pattern thường có cấu trúc sau:

- Command interface: định nghĩa `Execute()` (và có thể `Undo()`).
- ConcreteCommand: cài đặt lệnh cụ thể, giữ tham chiếu receiver và dữ liệu cần thiết.

- Receiver: chứa logic nghiệp vụ thật.
- Invoker: gọi command, có thể lưu history.
- Client: tạo command, gán receiver, cấu hình invoker.

UML tổng quát:

```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
interface Command {
```

```
    +Execute()
```

```
    +Undo()
```

```
}
```

```
class ConcreteCommandA {
```

```
    -receiver: Receiver
```

```
    +Execute()
```

```
    +Undo()
```

```
}
```

```
class ConcreteCommandB {
```

```
    -receiver: Receiver
```

```
    +Execute()
```

```
    +Undo()
```

```
}
```

```
class Receiver {  
    +ActionA()  
    +ActionB()  
}
```

```
class Invoker {  
    -history: List<Command>  
    +SetCommand(command: Command)  
    +Run()  
}
```

```
class Client
```

```
ConcreteCommandA ..|> Command
```

```
ConcreteCommandB ..|> Command
```

```
ConcreteCommandA --> Receiver
```

```
ConcreteCommandB --> Receiver
```

```
Invoker --> Command
```

```
Client --> Invoker
```

```
Client --> Receiver
```

```
Client --> ConcreteCommandA
```

Client --> ConcreteCommandB

@enduml

3.2.6. Các thành viên

Command

- Interface chung cho mọi lệnh.
- Tối thiểu có `Execute()`.
- Nếu cần undo/redo, thêm `Undo()`.

ConcreteCommand

- Đóng gói yêu cầu cụ thể.
- Chứa dữ liệu đầu vào cho thao tác.
- Gọi receiver để thực thi logic.

Receiver

- Nơi chứa nghiệp vụ thật.
- Không biết invoker hay UI cụ thể.
- Có thể được dùng bởi nhiều command.

Invoker

- Kích hoạt command.
- Có thể quản lý queue/history.
- Không cần biết chi tiết nghiệp vụ bên dưới.

Client

- Wiring toàn hệ thống.
- Khởi tạo receiver, command, invoker.
- Gắn command vào nút bấm/hotkey/scheduler.

3.2.7. Sự cộng tác

Luồng cộng tác chuẩn:

1. Client tạo receiver.
2. Client tạo concrete command và inject receiver.
3. Client gán command vào invoker.
4. Invoker gọi Execute() khi có trigger.
5. Command gọi receiver để xử lý nghiệp vụ.
6. Nếu có undo, invoker lưu history và gọi Undo() khi cần.

Sequence diagram:

```
@startuml
actor User
participant Invoker
participant ConcreteCommand
participant Receiver

User -> Invoker : click button / press shortcut
Invoker -> ConcreteCommand : Execute()
ConcreteCommand -> Receiver : Action(...)
Receiver --> ConcreteCommand : done
ConcreteCommand --> Invoker : done
@enduml
```

3.2.8. Các hệ quả mang lại, Ưu nhược điểm

3.2.9. Ưu điểm

- **Loose Coupling**: invoker không phụ thuộc receiver cụ thể.

- **Open/Closed:** thêm command mới mà ít sửa code cũ.
- **Undo/Redo friendly:** command là đơn vị thao tác độc lập.
- **Queue/Retry/Logging dễ dàng:** command object có thể lưu, serialize, replay.
- **Macro support:** gom nhiều command thành command lớn.

3.2.10. Nhược điểm

- Tăng số lượng class với mỗi thao tác mới.
- Có thể over-engineering cho bài toán nhỏ.
- Undo phức tạp khi thao tác có side-effect ngoài hệ thống (gửi mail, gọi API ngoài).
- Cần quản lý history và transaction boundary cẩn thận.

3.2.11. Chú ý liên quan đến việc cài đặt

- Mỗi command nên làm một việc rõ ràng (SRP).
- Quy định rõ idempotency nếu command có thể retry.
- Với undo, cần lưu đủ state trước khi execute.
- Tránh để command chứa logic nghiệp vụ quá lớn, nghiệp vụ nên ở receiver/domain service.
- Nếu cần chạy async, interface có thể là `Task ExecuteAsync()`.
- Với distributed system, nên có `CommandId`, `timestamp`, `correlation id` để trace.

3.2.12. Một số so sánh nếu có

3.2.13. Command và Strategy

- Command đóng gói **yêu cầu thực thi** thành object.
- Strategy đóng gói **thuật toán thay thế** cho một context.

Command thường gắn với trigger, queue, undo/redo; Strategy thường gắn với chọn cách xử lý tại runtime.

3.2.14. Command và Chain of Responsibility

- Command: một thao tác cụ thể được execute.
- Chain of Responsibility: request đi qua chuỗi handler có thể chặn hoặc chuyển tiếp.

Có thể kết hợp: mỗi handler trong chain thực thi một command.

3.2.15. Command và Mediator

- Command tập trung vào thao tác.
- Mediator tập trung điều phối giao tiếp giữa nhiều colleague object.

3.2.16. Command và Event

- Command diễn tả **ý định** (hãy làm gì đó), thường one handler chính.
- Event diễn tả **điều đã xảy ra**, có thể nhiều subscriber.

3.2.17. Mã nguồn minh họa

Ví dụ C# đơn giản có undo/redo cho tài khoản ngân hàng.

```
using System;
```

```
using System.Collections.Generic;
```

```
public interface ICommand
```

```
{
```

```
    void Execute();
```

```
    void Undo();
```

```
}
```

```
public class BankAccount
```

```
{
```

```
    public string Owner { get; }
```

```
public decimal Balance { get; private set; }
```

```
public BankAccount(string owner, decimal initialBalance)
```

```
{
```

```
    Owner = owner;
```

```
    Balance = initialBalance;
```

```
}
```

```
public void Deposit(decimal amount)
```

```
{
```

```
    Balance += amount;
```

```
    Console.WriteLine($"Deposit {amount}. Balance = {Balance}");
```

```
}
```

```
public void Withdraw(decimal amount)
```

```
{
```

```
    Balance -= amount;
```

```
    Console.WriteLine($"Withdraw {amount}. Balance = {Balance}");
```

```
}
```

```
}
```

```
public class DepositCommand : ICommand
```

```
{
```

```

private readonly BankAccount _account;

private readonly decimal _amount;

public DepositCommand(BankAccount account, decimal amount)
{
    _account = account;
    _amount = amount;
}

public void Execute() => _account.Deposit(_amount);
public void Undo() => _account.Withdraw(_amount);
}

public class WithdrawCommand : ICommand
{
    private readonly BankAccount _account;
    private readonly decimal _amount;

    public WithdrawCommand(BankAccount account, decimal amount)
    {
        _account = account;
        _amount = amount;
    }
}

```

```

    public void Execute() => _account.Withdraw(_amount);

    public void Undo() => _account.Deposit(_amount);
}

```

```

public class CommandInvoker
{
    private readonly Stack<ICommand> _undoStack = new();
    private readonly Stack<ICommand> _redoStack = new();

    public void Run(ICommand command)
    {
        command.Execute();

        _undoStack.Push(command);

        _redoStack.Clear();
    }

    public void Undo()
    {
        if (_undoStack.Count == 0) return;

        var cmd = _undoStack.Pop();

        cmd.Undo();
    }
}

```

```

        _redoStack.Push(cmd);
    }

    public void Redo()
    {
        if (_redoStack.Count == 0) return;

        var cmd = _redoStack.Pop();
        cmd.Execute();
        _undoStack.Push(cmd);
    }
}

public class Program
{
    public static void Main()
    {
        var account = new BankAccount("Phat", 1000m);
        var invoker = new CommandInvoker();

        invoker.Run(new DepositCommand(account, 200m));
        invoker.Run(new WithdrawCommand(account, 150m));
    }
}

```

```

    Console.WriteLine("Undo:");

    invoker.Undo();

    Console.WriteLine("Redo:");

    invoker.Redo();

}

}

```

Ví dụ trên minh họa:

- Mỗi thao tác gửi/rút tiền là một command object.
- Invoker quản lý lịch sử để undo/redo.
- UI hoặc API endpoint chỉ cần gọi invoker, không cần biết chi tiết tài khoản xử lý ra sao.

3.2.18. Ví dụ về các hệ thống thực tế

- **Text editors**: mỗi thao tác nhập/xóa/định dạng là command có undo.
- **IDE**: refactor action thường triển khai theo command + history.
- **Game engines**: input mapping (jump, shoot, interact) qua command.
- **Job queue systems**: mỗi job có thể xem như một command delayed execution.
- **CQRS architectures**: CreateOrderCommand, ApprovePaymentCommand, ...
- **Remote control IoT**: nút bấm ánh xạ tới command gửi tới thiết bị tương ứng.

3.2.19. Các mẫu liên quan

- **Memento**: thường đi cùng Command để lưu trạng thái undo phức tạp.
- **Composite**: dùng cho macro command (một command chứa danh sách command con).
- **Prototype**: clone command mẫu rồi chỉnh dữ liệu đầu vào.
- **Mediator**: invoker/command có thể gửi kết quả qua mediator để điều phối module khác.

- **Chain of Responsibility**: command sau khi tạo có thể đi qua chain kiểm tra/authorize trước khi execute.

Tóm lại, Command Pattern đặc biệt mạnh khi hệ thống cần tách trigger và xử lý, đồng thời hỗ trợ lịch sử thao tác, queue, retry và mở rộng hành vi theo hướng data-driven.

3.3. Iterator (Đạt)

3.3.1. Tên và Phân loại

Iterator Pattern là một mẫu thiết kế thuộc nhóm **Behavioral Pattern** (nhóm hành vi).

Mẫu này cung cấp cách duyệt tuần tự các phần tử trong một tập hợp mà không cần để lộ cấu trúc bên trong của tập hợp đó.

3.3.2. Mục đích, ý định

Mục đích chính của Iterator là tách logic duyệt dữ liệu ra khỏi đối tượng lưu trữ dữ liệu.

Ý định của pattern:

- Cho phép truy cập tuần tự các phần tử trong collection.
- Ẩn biểu diễn nội bộ của cấu trúc dữ liệu (array, list, tree, graph...).
- Cung cấp một giao diện duyệt thống nhất cho nhiều kiểu tập hợp khác nhau.
- Cho phép có nhiều cách duyệt trên cùng một collection (ví dụ: xuôi, ngược, lọc theo điều kiện).

3.3.3. Motivation

Giả sử ta có một lớp quản lý danh sách từ (WordsCollection) và cần duyệt dữ liệu theo nhiều cách:

- Duyệt từ đầu đến cuối.
- Duyệt từ cuối về đầu.

Nếu nhúng trực tiếp mọi vòng lặp vào WordsCollection, lớp này sẽ phải gánh quá nhiều trách nhiệm:

- Vừa quản lý dữ liệu, vừa quản lý chi tiết duyệt.
- Mỗi kiểu duyệt mới lại sửa class hiện có.
- Client phải biết nội bộ collection được lưu như thế nào.

Iterator giải quyết bằng cách tách phần duyệt thành các object riêng (AlphabeticalOrderIterator). Khi đó client chỉ làm việc với các hàm như `next()`, `valid()`, `current()`, không cần quan tâm collection lưu trữ dữ liệu ra sao.

3.3.4. Khả năng ứng dụng

Nên dùng Iterator khi:

- Cần duyệt một collection phức tạp nhưng muốn ẩn cấu trúc dữ liệu bên trong.
- Muốn hỗ trợ nhiều cách duyệt cho cùng một dữ liệu.
- Muốn thống nhất API duyệt giữa nhiều collection khác nhau.
- Muốn giảm coupling giữa client và collection.

Ví dụ phù hợp:

- Duyệt danh sách bản ghi trong ứng dụng quản lý.
- Duyệt node trong cây giao diện.
- Duyệt kết quả truy vấn theo trang (paging).
- Duyệt dữ liệu stream hoặc pipeline theo từng bước.

3.3.5. Cấu trúc

Iterator Pattern thường gồm các vai trò chính:

- Iterator: interface định nghĩa các thao tác duyệt (`current`, `next`, `valid`, `rewind`).
- ConcreteIterator: cài đặt cách duyệt cụ thể và giữ trạng thái vị trí hiện tại.
- Aggregator (hoặc Iterable): interface khai báo phương thức tạo iterator.
- ConcreteAggregator: collection cụ thể, cung cấp iterator tương ứng.

- Client: sử dụng iterator để duyệt dữ liệu.

Sơ đồ lớp tương ứng với demo trong project:

classDiagram

```
class Iterator {  
    <<interface>>  
    +current() Object  
    +next() Object  
    +key() Number  
    +valid() boolean  
    +rewind() void  
}
```

```
class AlphabeticalOrderIterator {  
    -collection WordsCollection  
    -position Number  
    -reverse boolean  
    +current() string  
    +next() string  
    +key() Number  
    +valid() boolean  
    +rewind() void  
}
```

```
class Aggregator {
    <<interface>>
    +getIterator() Iterator
}
```

```
class WordsCollection {
    -items string[]
    +addItem(item)
    +getItems() string[]
    +getCount() Number
    +getIterator() Iterator
    +getReverseIterator() Iterator
}
```

Iterator <|.. AlphabeticalOrderIterator

Aggregator <|.. WordsCollection

WordsCollection --> AlphabeticalOrderIterator : creates

AlphabeticalOrderIterator --> WordsCollection : traverses

3.3.6. Các thành viên

Iterator

- Định nghĩa giao diện duyệt thống nhất.
- Tách client khỏi cài đặt duyệt cụ thể.

AlphabeticalOrderIterator

- Là concrete iterator trong ví dụ.
- Giữ tham chiếu đến `WordsCollection`.
- Quản lý vị trí hiện tại bằng `position`.
- Hỗ trợ duyệt xuôi hoặc ngược thông qua `reverse`.

Aggregator

- Giao diện collection có thể tạo iterator.

WordsCollection

- Lưu dữ liệu gốc trong `items`.
- Cung cấp `getIterator()` để duyệt xuôi.
- Cung cấp `getReverseIterator()` để duyệt ngược.

Client

- Thêm dữ liệu vào collection.
- Lấy iterator phù hợp và duyệt bằng cùng một API.

3.3.7. Sự cộng tác

Luồng cộng tác điển hình:

1. Client tạo `WordsCollection` và thêm các phần tử.
2. Client gọi `getIterator()` hoặc `getReverseIterator()`.
3. `WordsCollection` tạo `AlphabeticalOrderIterator` tương ứng.
4. Client lặp theo mẫu `while (iterator.valid()) { iterator.next(); }`.
5. Iterator truy cập dữ liệu từ collection và tự cập nhật trạng thái vị trí.

Điểm quan trọng là client không cần truy cập trực tiếp `items` hay biết collection được cài đặt bằng kiểu dữ liệu nào.

3.3.8. Các hệ quả mang lại, Ưu nhược điểm

3.3.9. Ưu điểm

- Tuân thủ nguyên tắc trách nhiệm đơn (SRP): tách lưu trữ và duyệt.
- Tuân thủ nguyên tắc đóng/mở (OCP): thêm kiểu duyệt mới mà không phá code cũ.
- Giảm coupling giữa client và collection.
- Có thể tồn tại nhiều iterator đồng thời trên cùng một dữ liệu.
- Cải thiện khả năng kiểm thử phần duyệt độc lập.

3.3.10. Nhược điểm

- Tăng số lượng class/interface với hệ thống nhỏ.
- Có thêm overhead quản lý object iterator.
- Với collection đơn giản, dùng Iterator có thể bị xem là over-engineering.
- Cần lưu ý khi collection bị thay đổi trong lúc đang duyệt.

3.3.11. Chú ý liên quan đến việc cài đặt

- Xác định rõ iterator làm việc theo snapshot hay theo dữ liệu “live”.
- Quy ước rõ hành vi của next () khi hết phần tử.
- Tránh để client phụ thuộc vào index nội bộ của collection.
- Với dữ liệu lớn hoặc stream, cân nhắc lazy iteration để tiết kiệm bộ nhớ.
- Nếu có đa luồng, cần chính sách đồng bộ khi collection có thể bị sửa trong lúc duyệt.

3.3.12. Một số so sánh nếu có

3.3.13. Iterator và Visitor

- Iterator tập trung vào **cách duyệt** cấu trúc dữ liệu.
- Visitor tập trung vào **thao tác áp dụng** lên từng phần tử của cấu trúc.

3.3.14. Iterator và Composite

- Composite tổ chức dữ liệu theo cây phần-whole.

- Iterator thường được dùng để duyệt các node trong Composite mà không để lộ cấu trúc nội bộ.

3.3.15. Iterator và for/foreach trực tiếp

- for/foreach tiện cho trường hợp đơn giản.
- Iterator hữu ích hơn khi cần nhiều chiến lược duyệt hoặc cần che giấu cấu trúc dữ liệu phức tạp.

3.3.16. Mã nguồn minh họa

Đoạn mã sau sử dụng đúng ví dụ trong project:

```
function print(str) { console.log(str) }

class AlphabeticalOrderIterator {
  constructor(collection, reverse = false) {
    this.collection = collection;
    this.reverse = reverse;
    this.position = reverse ? collection.getCount() - 1 : 0;
  }

  rewind() {
    this.position = this.reverse ? this.collection.getCount() - 1 : 0;
  }

  current() {
    return this.collection.getItems()[this.position];
  }
}
```

```
key() {  
    return this.position;  
}
```

```
next() {  
    const item = this.collection.getItems()[this.position];  
    this.position += this.reverse ? -1 : 1;  
    return item;  
}
```

```
valid() {  
    return this.reverse  
        ? this.position >= 0  
        : this.position < this.collection.getCount();  
}  
}
```

```
class WordsCollection {  
    constructor() {  
        this.items = [];  
    }  
}
```

```

getItems() {
    return this.items;
}

getCount() {
    return this.items.length;
}

addItem(item) {
    this.items.push(item);
}

getIterator() {
    return new AlphabeticalOrderIterator(this);
}

getReverseIterator() {
    return new AlphabeticalOrderIterator(this, true);
}
}

const collection = new WordsCollection();
collection.addItem('First');

```



```
collection.addItem('Second');  
collection.addItem('Third');  
  
const iterator = collection.getIterator();  
print('Straight traversal:');  
while (iterator.valid()) {  
    print(iterator.next());  
}  
  
print('');  
print('Reverse traversal:');  
const reverseIterator = collection.getReverseIterator();  
while (reverseIterator.valid()) {  
    print(reverseIterator.next());  
}
```

Kết quả mong đợi:

Straight traversal:

First

Second

Third

Reverse traversal:

Third

Second

First

3.3.17. Ví dụ về các hệ thống thực tế

- Java Collections Framework: Iterator, ListIterator.
- .NET Collections: IEnumerable và IEnumerator.
- Python: giao thức iterator (`__iter__`, `__next__`).
- C++ STL: input/forward/bidirectional/random-access iterator.
- ORM frameworks: duyệt tập kết quả truy vấn theo cursor/iterator.

3.3.18. Các mẫu liên quan

- **Composite**: thường kết hợp với Iterator để duyệt cây đối tượng.
- **Factory Method**: có thể dùng để tạo iterator phù hợp cho từng loại collection.
- **Memento**: có thể kết hợp để lưu và phục hồi trạng thái vị trí duyệt.
- **Visitor**: Iterator cung cấp đường đi qua cấu trúc, Visitor định nghĩa thao tác trên từng phần tử.

3.4. Mediator (Phức)

3.4.1. Tên và Phân loại

Mediator - Mẫu hành vi (Behavioral Pattern)

3.4.2. Mục đích, ý định

Xác định một đối tượng trung gian để đơn giản hóa giao tiếp giữa các đối tượng. Giảm sự phụ thuộc lẫn nhau bằng cách cho các đối tượng giao tiếp qua mediator thay vì trực tiếp.

3.4.3. Motivation

Khi nhiều đối tượng cần giao tiếp phức tạp với nhau, chúng trở thành phụ thuộc chặt chẽ.

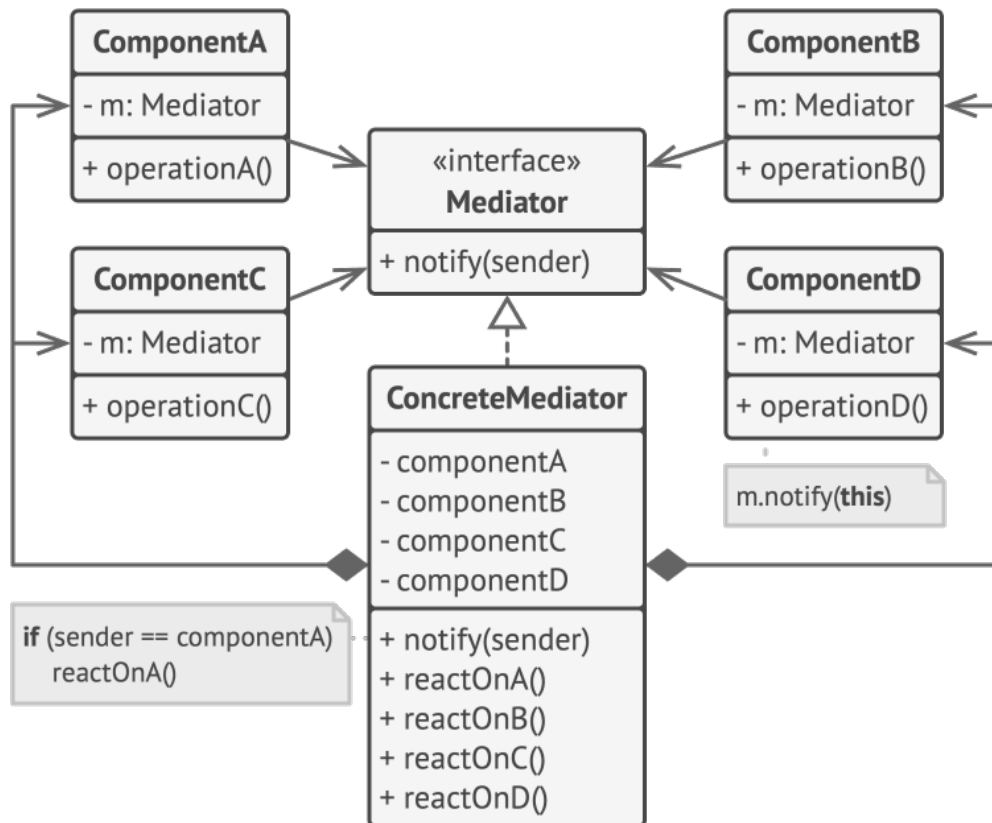
Mediator tập trung logic giao tiếp.

3.4.4. Khả năng ứng dụng

- Tập hợp các đối tượng giao tiếp theo cách không thể kiểm soát được

- Cần tái sử dụng đối tượng nhưng khó vì có nhiều tham chiếu tới các đối tượng khác
- Hành vi phân tán giữa các lớp cần tập trung

3.4.5. Cấu trúc



Hình 3.30: Cấu trúc của mẫu Mediator

Mediator định nghĩa giao diện để giao tiếp. **ConcreteMediator** triển khai và phối hợp các **Colleague**.

3.4.6. Các thành viên

- **Mediator**: giao diện xác định phương thức giao tiếp
- **ConcreteMediator**: triển khai giao tiếp và phối hợp **Colleague**
- **Colleague**: giao diện các đối tượng tham gia
- **ConcreteColleague**: giao tiếp qua mediator

3.4.7. Sự cộng tác

Colleague gửi yêu cầu tới **Mediator**. **Mediator** phối hợp phản hồi từ các **Colleague** khác.

3.4.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Giảm sự phụ thuộc giữa các đối tượng; tập trung logic giao tiếp. **Nhược điểm:**

Mediator có thể trở thành “God Object”; khó bảo trì nếu logic phức tạp.

3.4.9. Chú ý liên quan đến việc cài đặt

- Xác định tập hợp Colleague và cách giao tiếp
- Đảm bảo Mediator không vi phạm Single Responsibility

3.4.10. Mã nguồn minh họa

Các đối tượng dialog, event handlers trong UI framework.

3.4.11. Ví dụ về các hệ thống thực tế

- Dialog box: các control giao tiếp qua dialog
- Chat room: người dùng giao tiếp qua phòng
- Air traffic control: máy bay giao tiếp qua tower
- Event-driven systems

3.4.12. Các mẫu liên quan

Liên quan đến Observer (thông báo), Facade (đơn giản giao diện).

3.5. Memento (Phát)

3.5.1. Tên và phân loại

Memento Pattern có thể hiểu là **mẫu lưu dấu trạng thái** hoặc **mẫu ghi nhớ**. Tên gọi này nhấn mạnh mục tiêu chính của pattern: lưu lại một trạng thái tại một thời điểm cụ thể để sau này có thể khôi phục lại.

Memento thuộc nhóm **Behavioral Pattern** vì nó tập trung vào cách object quản lý hành vi liên quan đến việc lưu, phục hồi và quay lại trạng thái trước đó. Pattern này không chỉ nói về việc lưu dữ liệu, mà còn nói về cách lưu dữ liệu sao cho không phá vỡ tính đóng gói của object.

Nói ngắn gọn, Memento là pattern dùng để **lưu snapshot trạng thái của một object và khôi phục lại snapshot đó khi cần**, nhưng bên ngoài không được can thiệp trực tiếp vào cấu trúc nội bộ của object.

3.5.2. Vấn đề cần giải quyết

Trong nhiều hệ thống, người dùng hoặc chương trình cần quay lại trạng thái trước đó sau khi đã thực hiện một số thay đổi. Đây là nhu cầu rất phổ biến.

Ví dụ:

- Trình soạn thảo văn bản cần hỗ trợ **Undo/Redo**.
- Game cần lưu **checkpoint** hoặc **save slot**.
- Hệ thống cấu hình cần lưu snapshot trước khi thay đổi setting.
- Database hoặc nghiệp vụ cần rollback khi một thao tác thất bại.
- Form nhập liệu cần lưu trạng thái ban đầu trước khi người dùng chỉnh sửa.
- Công cụ thiết kế cần quay lại trạng thái trước đó của canvas.

Giả sử ta có một `TextEditor` đang chứa nội dung văn bản, vị trí con trỏ, vùng đang chọn, font, màu chữ và nhiều thông tin khác. Khi người dùng bấm `Ctrl + Z`, hệ thống cần khôi phục lại trạng thái trước đó.

Một cách làm đơn giản là để bên ngoài đọc toàn bộ state của `TextEditor`, lưu lại, rồi khi cần thì gán ngược lại. Tuy nhiên cách này có vấn đề lớn:

- Client phải biết quá nhiều chi tiết nội bộ của `TextEditor`.
- Object bị lộ state, làm giảm tính đóng gói.
- Khi cấu trúc state thay đổi, nhiều đoạn code bên ngoài cũng phải thay đổi.
- Dễ xảy ra lỗi nếu client lưu thiếu hoặc khôi phục sai state.

Vấn đề cốt lõi là: **làm sao quay lại trạng thái trước đó của object mà không làm lộ toàn bộ dữ liệu nội bộ của object?**

3.5.3. Định nghĩa

Memento Pattern là một design pattern cho phép lưu và khôi phục trạng thái trước đó của một object mà không phá vỡ tính đóng gói (**encapsulation**).

Pattern này tách trách nhiệm thành ba phần:

- Object chính tự biết cách tạo snapshot của chính nó.
- Snapshot được đóng gói trong một object riêng gọi là Memento.
- Một object quản lý lịch sử snapshot, nhưng không được phép sửa hoặc hiểu chi tiết bên trong snapshot.

Điểm quan trọng của Memento là: object bên ngoài có thể giữ snapshot, nhưng không nên truy cập trực tiếp vào dữ liệu nội bộ trong snapshot. Chỉ object tạo ra snapshot mới nên có quyền dùng snapshot đó để restore.

3.5.4. Ý tưởng cốt lõi

Ý tưởng chính của Memento Pattern là:

Trước khi thay đổi state quan trọng, object tạo ra một bản ghi nhớ trạng thái hiện tại.

Khi cần undo hoặc rollback, object dùng bản ghi nhớ đó để tự khôi phục lại state cũ.

Có thể hình dung qua ví dụ **save game**:

- Nhân vật game, level, máu, vị trí, inventory là trạng thái hiện tại.
- Save slot là Memento.
- Save manager là Caretaker, quản lý danh sách save slot.

Có thể hình dung qua ví dụ **Ctrl + Z trong text editor**:

- Văn bản hiện tại là state của Editor.
- Mỗi lần lưu lịch sử là một Memento.

- History hoặc UndoManager là Caretaker, quản lý stack các memento.

Điểm đặc biệt là Caretaker chỉ biết lưu và lấy memento. Nó không cần biết bên trong memento có text, cursor, selection hay format gì. Việc đóng gói này giúp bảo vệ object chính khỏi việc bị truy cập state tùy tiện từ bên ngoài.

3.5.5. Thành phần chính

Các thành phần chính của Memento Pattern gồm:

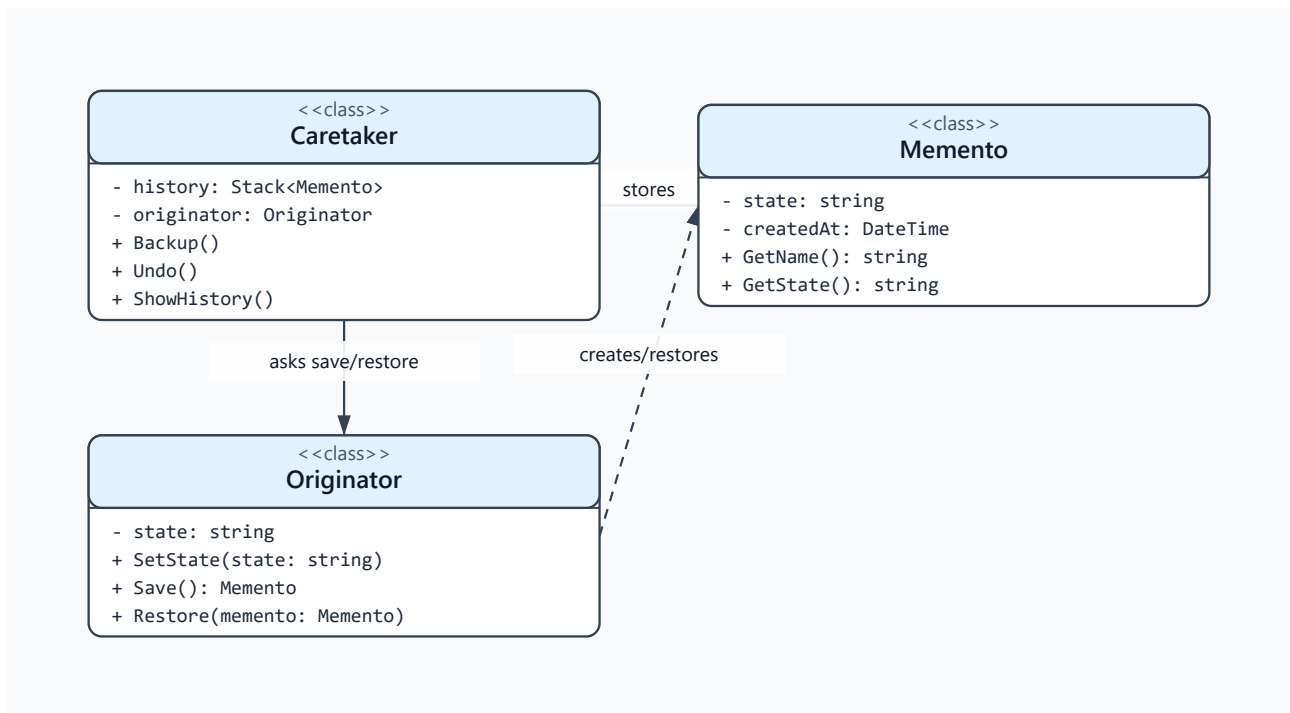
Thành phần	Vai trò
Originator	Object chính có state cần được lưu và khôi phục. Originator tự tạo memento và tự restore từ memento.
Memento	Object chứa snapshot trạng thái của Originator tại một thời điểm. Memento nên che giấu dữ liệu nội bộ khỏi bên ngoài.
Caretaker	Object quản lý lịch sử snapshot, ví dụ stack undo/redo hoặc danh sách checkpoint. Caretaker không xử lý logic state bên trong.
Client	Đối tượng kích hoạt thao tác save, undo, redo hoặc rollback thông qua Originator và Caretaker.

Vai trò cụ thể:

- Originator: biết rõ state nội bộ của mình, nên nó là nơi phù hợp nhất để tạo snapshot và khôi phục snapshot.
- Memento: đóng vai trò như một hộp lưu trạng thái. Bên ngoài chỉ nên xem nó như một bản ghi lịch sử, không nên sửa nội dung bên trong.
- Caretaker: lưu nhiều memento theo thứ tự thời gian. Với undo, thường dùng stack. Với save game, có thể dùng danh sách save slot hoặc file lưu trữ.

3.5.6. UML tổng quát

Sơ đồ UML tổng quát của Memento Pattern:



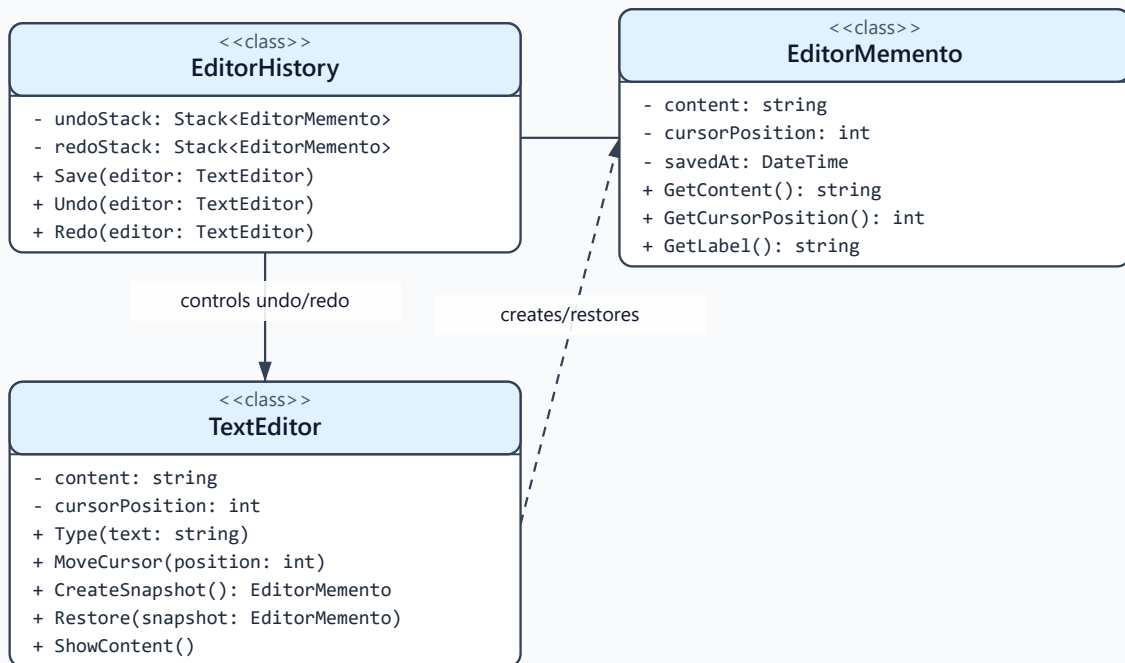
Hình 3.31: Cấu trúc UML tổng quát của Memento Pattern

Giải thích sơ đồ:

- **Originator** có state thật sự của object.
- Khi cần lưu, **Originator.Save()** tạo ra **Memento** chứa snapshot.
- **Caretaker.Backup()** gọi **Save()** và lưu memento vào history.
- Khi undo, **Caretaker.Undo()** lấy memento gần nhất và yêu cầu **Originator.Restore()**.

3.5.7. UML minh họa với Text Editor

Ví dụ cụ thể với trình soạn thảo văn bản:



Hình 3.32: Class diagram của ví dụ Text Editor

Trong ví dụ này:

- TextEditor là Originator.
- EditorMemento là snapshot của nội dung và vị trí con trỏ.
- EditorHistory là Caretaker, quản lý stack undo và redo.

3.5.8. Luồng hoạt động

Luồng hoạt động khi lưu trạng thái:

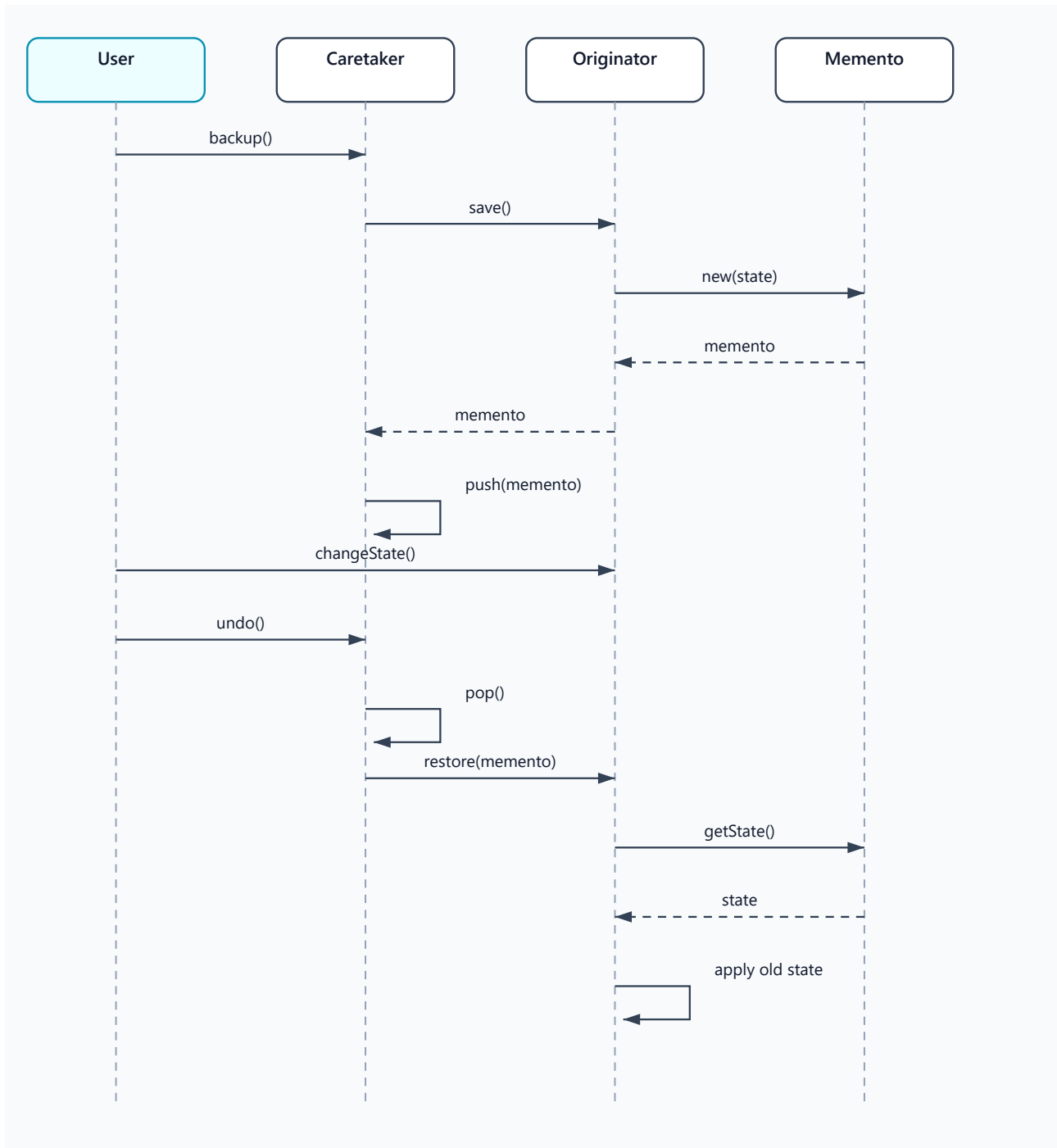
1. User hoặc client chuẩn bị thực hiện một thao tác có thể cần undo.
2. Client yêu cầu Caretaker lưu trạng thái hiện tại.
3. Caretaker gọi Originator.Save() hoặc CreateSnapshot().
4. Originator tạo Memento chứa state hiện tại.
5. Caretaker lưu Memento vào lịch sử.

6. User thực hiện thay đổi state.

Luồng hoạt động khi undo:

1. User bấm undo hoặc hệ thống cần rollback.
2. Caretaker lấy memento gần nhất từ lịch sử.
3. Caretaker truyền memento đó cho Originator.
4. Originator đọc state trong memento.
5. Originator khôi phục lại state cũ.
6. Client tiếp tục làm việc với object đã được restore.

Sequence diagram minh họa:



Hình 3.33: Luồng backup và undo bằng Memento

3.5.9. Ví dụ code C#

Ví dụ dưới đây mô phỏng một text editor đơn giản có chức năng undo.

```
using System;
```

```
using System.Collections.Generic;
```

```

public class EditorMemento
{
    private readonly string _content;
    private readonly int _cursorPosition;
    private readonly DateTime _savedAt;

    public EditorMemento(string content, int cursorPosition)
    {
        _content = content;
        _cursorPosition = cursorPosition;
        _savedAt = DateTime.Now;
    }

    public string GetContent()
    {
        return _content;
    }

    public int GetCursorPosition()
    {
        return _cursorPosition;
    }
}

```

```

public string GetLabel()
{
    return $"Saved at {_savedAt:HH:mm:ss}, length = {_content.Length}";
}
}

```

```

public class TextEditor
{
    private string _content = "";
    private int _cursorPosition = 0;

    public void Type(string text)
    {
        _content = _content.Insert(_cursorPosition, text);
        _cursorPosition += text.Length;
    }

    public void MoveCursor(int position)
    {
        if (position < 0 || position > _content.Length)
        {
            throw new ArgumentOutOfRangeException(nameof(position));
        }
    }
}

```

```

    }

    _cursorPosition = position;
}

public EditorMemento CreateSnapshot()
{
    return new EditorMemento(_content, _cursorPosition);
}

public void Restore(EditorMemento memento)
{
    _content = memento.GetContent();
    _cursorPosition = memento.GetCursorPosition();
}

public void Show()
{
    Console.WriteLine($"Content: '{_content}' | Cursor:
{_cursorPosition}");
}
}

```

```

public class EditorHistory
{
    private readonly Stack<EditorMemento> _undoStack = new();

    public void Save(TextEditor editor)
    {
        _undoStack.Push(editor.CreateSnapshot());
    }

    public void Undo(TextEditor editor)
    {
        if (_undoStack.Count == 0)
        {
            Console.WriteLine("Nothing to undo.");
            return;
        }

        EditorMemento snapshot = _undoStack.Pop();
        editor.Restore(snapshot);
        Console.WriteLine($"Restored: {snapshot.GetLabel()}");
    }
}

```

```
public class Program
{
    public static void Main()
    {
        TextEditor editor = new TextEditor();
        EditorHistory history = new EditorHistory();

        history.Save(editor);
        editor.Type("Hello");
        editor.Show();

        history.Save(editor);
        editor.Type(" World");
        editor.Show();

        history.Save(editor);
        editor.MoveCursor(5);
        editor.Type(", C#");
        editor.Show();

        history.Undo(editor);
        editor.Show();
    }
}
```



```
        history.Undo(editor);  
  
        editor.Show();  
    }  
}
```

Kết quả kỳ vọng:

Content: 'Hello' | Cursor: 5

Content: 'Hello World' | Cursor: 11

Content: 'Hello, C# World' | Cursor: 9

Restored: Saved at ..., length = 11

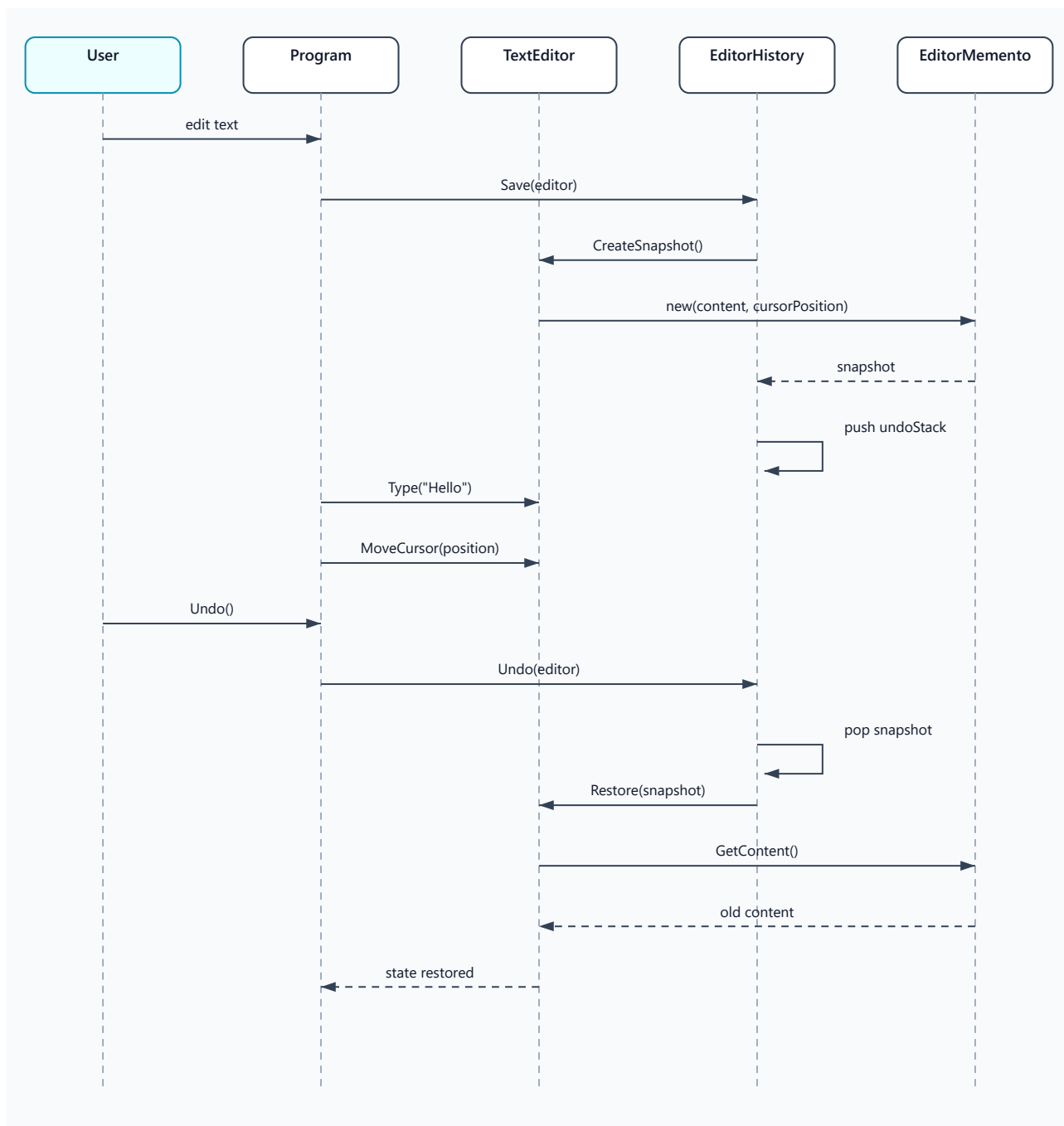
Content: 'Hello World' | Cursor: 11

Restored: Saved at ..., length = 5

Content: 'Hello' | Cursor: 5

Trong ví dụ này:

- `TextEditor` không để lộ trực tiếp field `_content` và `_cursorPosition` cho client chỉnh sửa tùy tiện.
- `EditorMemento` lưu snapshot của editor.
- `EditorHistory` chỉ quản lý các snapshot trong stack, không xử lý logic nội bộ của editor.



Hình 3.34: Luồng chạy code ví dụ TextEditor undo

3.5.10. Biến thể Undo/Redo

Để hỗ trợ cả undo và redo, có thể dùng hai stack:

- undoStack: lưu các trạng thái trước đó.
- redoStack: lưu các trạng thái bị undo để có thể redo lại.

Ý tưởng:

- Trước khi thay đổi, lưu snapshot vào undoStack.
- Khi undo, trạng thái hiện tại được đưa vào redoStack, sau đó restore snapshot từ undoStack.
- Khi redo, trạng thái hiện tại được đưa lại vào undoStack, sau đó restore snapshot từ redoStack.
- Khi người dùng thực hiện thay đổi mới sau undo, redoStack thường bị xóa vì nhánh lịch sử cũ không còn phù hợp.

Memento giúp làm undo/redo đơn giản hơn khi trạng thái object tương đối nhỏ và có thể snapshot trực tiếp.

3.5.11. Đánh giá

3.5.12. Ưu điểm

Ưu điểm	Giải thích
Bảo vệ encapsulation	Client không cần biết toàn bộ state nội bộ của Originator. Việc save/restore do Originator tự kiểm soát.
Dễ xây dựng undo/redo	Mỗi snapshot đại diện cho một trạng thái trước đó, rất phù hợp với undo trong editor, drawing tool hoặc form.
Tách logic lưu state	Caretaker quản lý lịch sử, Originator quản lý state thật. Mỗi thành phần có trách nhiệm rõ ràng.
Dễ rollback	Có thể lưu trạng thái trước thao tác rủi ro và khôi phục nếu thao tác thất bại.
Không làm phức tạp client	Client chỉ cần gọi save/undo, không phải tự copy từng field của object.

3.5.13. Nhược điểm

Nhược điểm	Giải thích
Tốn bộ nhớ	Nếu object lớn hoặc lưu snapshot quá thường xuyên, lịch sử memento có thể tiêu tốn nhiều RAM hoặc storage.
Deep copy phức tạp	Nếu state chứa nhiều object lồng nhau, collection hoặc reference phức tạp, việc snapshot đúng có thể khó.
Khó quản lý nhiều phiên bản	Cần cơ chế giới hạn số lượng snapshot, đặt tên checkpoint hoặc xóa lịch sử cũ.
Có thể che giấu chi phí	Nhìn bên ngoài thao tác save rất đơn giản, nhưng bên trong có thể copy lượng dữ liệu lớn.
Không phù hợp với state thay đổi liên tục	Nếu state thay đổi từng mili-giây và mỗi lần đều snapshot, hệ thống dễ bị overhead lớn.

3.5.14. Khi nào nên dùng?

Nên dùng Memento khi:

- Cần chức năng undo/redo.
- Cần rollback khi thao tác thất bại.
- Cần lưu checkpoint hoặc save slot.
- Cần snapshot state ở một thời điểm cụ thể.
- Muốn lưu trạng thái mà vẫn giữ encapsulation.
- State của object đủ nhỏ hoặc có thể snapshot với chi phí chấp nhận được.

Ví dụ phù hợp:

- Text editor.
- Drawing application.
- Game save/checkpoint.

- Form wizard nhiều bước.
- Cấu hình hệ thống trước khi apply thay đổi.
- Transaction nghiệp vụ cần rollback ở mức object.

3.5.15. Khi nào không nên dùng?

Không nên dùng Memento khi:

- State quá lớn và snapshot toàn bộ sẽ rất tốn bộ nhớ.
- State thay đổi liên tục với tần suất cao.
- Chi phí deep copy quá lớn.
- Chỉ cần lưu một vài field đơn giản, không cần pattern đầy đủ.
- Có thể rollback tốt hơn bằng log action hoặc event sourcing.
- Cần audit chi tiết từng thao tác hơn là chỉ lưu trạng thái cuối.

Trong các trường hợp state lớn, có thể cân nhắc:

- Lưu diff thay vì lưu full snapshot.
- Giới hạn số lượng snapshot.
- Nén snapshot.
- Lưu snapshot xuống disk/database.
- Kết hợp với Command Pattern để undo bằng hành động ngược.

3.5.16. So sánh với Command Pattern

Tiêu chí	Memento	Command
Mục đích chính	Lưu trạng thái để khôi phục lại sau.	Đóng gói hành động thành object.
Undo bằng gì?	Undo bằng snapshot state cũ.	Undo bằng <code>Unexecute()</code> hoặc hành động ngược.
Dữ liệu lưu	State của object tại một thời điểm.	Thông tin về thao tác đã thực hiện.

Mức độ đơn giản	Dễ hiểu nếu state nhỏ.	Linh hoạt hơn nhưng cần thiết kế action rõ.
Chi phí	Tốn bộ nhớ nếu snapshot lớn.	Tốn logic vì phải viết thao tác đảo ngược.
Phù hợp khi	Object state nhỏ, cần quay lại snapshot.	Action phức tạp, cần queue/log/macro/redo.

Ví dụ:

- Memento: lưu toàn bộ nội dung editor trước khi sửa.
- Command: lưu thao tác InsertTextCommand, khi undo thì xóa đoạn text vừa insert.

3.5.17. So sánh với State Pattern

Tiêu chí	Memento	State
Mục đích chính	Lưu và khôi phục trạng thái quá khứ.	Thay đổi hành vi của object theo trạng thái hiện tại.
Trọng tâm	Temporal history: lịch sử theo thời gian.	Runtime behavior: hành vi tại thời điểm hiện tại.
State được dùng để làm gì?	Để restore lại object.	Để quyết định object phản ứng như thế nào.
Có lưu lịch sử không?	Có, thường lưu nhiều snapshot.	Không bắt buộc, thường chỉ giữ state hiện tại.
Ví dụ	Undo nội dung editor về bản trước.	Order thay đổi behavior theo Pending/Paid/Shipped/Cancelled.

Memento trả lời câu hỏi: **object trước đây như thế nào?**

State trả lời câu hỏi: **object hiện tại nên hành xử như thế nào theo trạng thái của nó?**

3.5.18. So sánh với Prototype Pattern

Tiêu chí	Memento	Prototype
Mục đích chính	Lưu snapshot để restore state.	Clone object để tạo object mới nhanh hơn.
Kết quả tạo ra	Memento đại diện cho trạng thái cũ.	Object mới có cấu trúc giống object mẫu.
Ai dùng object tạo ra?	Originator dùng để restore.	Client dùng object clone như một object bình thường.
Có bảo vệ encapsulation không?	Có, đây là mục tiêu quan trọng.	Không phải trọng tâm chính.
Ví dụ	Lưu trạng thái editor trước khi sửa.	Clone một enemy mẫu thành enemy mới trong game.

Hai pattern đều có thể liên quan đến việc copy dữ liệu, nhưng mục đích khác nhau. Memento copy state để quay lại quá khứ; Prototype copy object để tạo instance mới.

3.5.19. So sánh với Event Sourcing

Tiêu chí	Memento	Event Sourcing
Cách lưu	Lưu snapshot trạng thái tại một thời điểm.	Lưu chuỗi event đã xảy ra.
Khôi phục	Restore trực tiếp từ snapshot.	Replay event để dựng lại state.
Audit trail	Không thể hiện rõ từng hành động nếu chỉ lưu snapshot.	Rất tốt vì lưu đầy đủ lịch sử event.
Chi phí đọc	Nhanh nếu snapshot đầy đủ.	Có thể cần replay nhiều event, thường kết hợp snapshot.

Phù hợp khi	Cần undo/rollback đơn giản.	Cần audit, trace nghiệp vụ và lịch sử thay đổi chi tiết.
-------------	-----------------------------	--

Trong thực tế, Event Sourcing đôi khi kết hợp với snapshot để tránh replay toàn bộ event từ đầu. Snapshot trong trường hợp đó có ý tưởng gần với Memento, nhưng Event Sourcing là một kiến trúc/lối lưu dữ liệu rộng hơn.

3.5.20. Lưu ý thiết kế

Khi áp dụng Memento, cần chú ý:

- Không nên để Caretaker sửa nội dung bên trong Memento.
- Nên giới hạn số lượng snapshot để tránh tốn bộ nhớ.
- Với object lớn, cân nhắc lưu diff hoặc chỉ lưu phần thay đổi.
- Với object có reference phức tạp, cần phân biệt shallow copy và deep copy.
- Snapshot nên immutable để tránh bị thay đổi sau khi đã lưu.
- Nếu có nhiều loại state, nên đặt tên snapshot hoặc gắn timestamp để dễ quản lý.

3.5.21. Kết luận

Memento Pattern là giải pháp phù hợp khi hệ thống cần lưu và khôi phục trạng thái object mà vẫn bảo vệ tính đóng gói. Pattern này đặc biệt hữu ích cho undo/redo, checkpoint, rollback và snapshot.

Điểm mạnh lớn nhất của Memento là giúp object tự kiểm soát việc lưu và restore state của chính nó, trong khi Caretaker chỉ quản lý lịch sử snapshot. Nhờ vậy, client không cần biết chi tiết nội bộ của object.

Tuy nhiên, Memento có thể gây tốn bộ nhớ nếu state lớn hoặc snapshot quá thường xuyên. Vì vậy, khi sử dụng pattern này cần cân nhắc kích thước state, tần suất lưu, cơ chế xóa snapshot cũ và khả năng dùng diff hoặc Command Pattern thay thế.

3.6. Observer (Đạt)

3.6.1. Tên và Phân loại

Observer Pattern là một mẫu thiết kế thuộc nhóm **Behavioral Pattern** (nhóm hành vi).

Mẫu này định nghĩa mối quan hệ phụ thuộc một-nhiều giữa các đối tượng. Khi trạng thái của một đối tượng trung tâm thay đổi, mọi đối tượng phụ thuộc sẽ được thông báo và tự động cập nhật.

3.6.2. Mục đích, ý định

Mục đích chính của Observer là xây dựng cơ chế publish-subscribe (phát-thông báo/đăng ký lắng nghe) để tách nơi phát sinh sự kiện khỏi nơi xử lý sự kiện.

Ý định của pattern:

- Cho phép nhiều đối tượng đăng ký nhận thay đổi từ một nguồn dữ liệu.
- Giảm coupling giữa đối tượng phát sự kiện (Subject) và các đối tượng phản ứng (Observer).
- Hỗ trợ cập nhật tự động khi trạng thái thay đổi.
- Dễ mở rộng thêm loại phản ứng mới mà không sửa mã Subject.

3.6.3. Motivation

Giả sử ta có một đối tượng trung tâm Subject chứa trạng thái nghiệp vụ (ví dụ trạng thái cảm biến, trạng thái đơn hàng hoặc giá cổ phiếu).

Nhiều thành phần khác nhau cần phản ứng khi trạng thái này đổi:

- Thành phần A ghi log hoặc cảnh báo khi giá trị nhỏ hơn ngưỡng.
- Thành phần B cập nhật giao diện người dùng khi có thay đổi.

Nếu Subject gọi trực tiếp từng thành phần cụ thể, hệ thống nhanh chóng bị phụ thuộc chặt:

- Subject biết quá nhiều lớp xử lý cụ thể.
- Mỗi lần thêm kiểu phản ứng mới phải sửa Subject.

- Khó tái sử dụng Subject ở ngữ cảnh khác.

Observer giải quyết bằng cách để Subject chỉ quản lý danh sách observer thông qua interface và gọi `Notify()` khi trạng thái đổi. Observer nào quan tâm sẽ tự xử lý trong `Update()`.

3.6.4. Khả năng ứng dụng

Nên dùng Observer khi:

- Một thay đổi dữ liệu cần lan truyền đến nhiều thành phần.
- Cần cơ chế event-driven để giảm phụ thuộc giữa module.
- Danh sách thành phần lắng nghe có thể thay đổi ở runtime (attach/detach).
- Muốn tách logic nghiệp vụ cốt lõi khỏi logic hiển thị, log, gửi thông báo.

Ví dụ điển hình:

- Hệ thống GUI (button click, state change, data binding).
- Hệ thống thông báo theo sự kiện trong microservice.
- Theo dõi trạng thái đơn hàng để gửi email/SMS/push notification.
- Giám sát cảm biến IoT và kích hoạt cảnh báo theo ngưỡng.

3.6.5. Cấu trúc

Các vai trò chính của Observer Pattern:

- `ISubject`: khai báo `Attach`, `Detach`, `Notify`.
- `Subject`: lưu trạng thái và danh sách observer, phát thông báo khi đổi trạng thái.
- `IObserver`: khai báo `Update(ISubject subject)`.
- `ConcreteObserver`: cài đặt phản ứng cụ thể khi nhận thông báo.
- `Client`: tạo subject, observer và thiết lập đăng ký.

Sơ đồ lớp tổng quát:

classDiagram

```
class ISubject {
```

```

<<interface>>

+Attach(observer)

+Detach(observer)

+Notify()

}

```

```

class Subject {

    -observers: List~IObserver~

    +State: int

    +Attach(observer)

    +Detach(observer)

    +Notify()

    +SomeBusinessLogic()

}

```

```

class IObserver {

    <<interface>>

    +Update(subject)

}

```

```

class ConcreteObserverA {

    +Update(subject)

}

```

```
class ConcreteObserverB {
    +Update(subject)
}
```

```
ISubject <|.. Subject
```

```
IObserver <|.. ConcreteObserverA
```

```
IObserver <|.. ConcreteObserverB
```

```
Subject --> IObserver : notifies
```

3.6.6. Các thành viên

ISubject

- Giao diện cho đối tượng phát sự kiện.
- Định nghĩa các thao tác quản lý vòng đời observer.

Subject

- Chứa trạng thái nghiệp vụ (State).
- Quản lý danh sách observer (_observers).
- Cung cấp Attach, Detach, Notify.
- Trong ví dụ, SomeBusinessLogic() thay đổi State ngẫu nhiên rồi phát thông báo.

IObserver

- Giao diện chuẩn cho mọi observer.
- Nhận callback Update(ISubject subject) khi Subject thay đổi.

ConcreteObserverA, ConcreteObserverB

- Cài đặt logic phản ứng riêng dựa trên State của Subject.

- ConcreteObserverA phản ứng khi State < 3.
- ConcreteObserverB phản ứng khi State == 0 hoặc State >= 2.

Client

- Tạo Subject và các observer.
- Đăng ký/hủy đăng ký observer.
- Kích hoạt luồng nghiệp vụ để sinh sự kiện.

3.6.7. Sự cộng tác

Luồng cộng tác tiêu biểu trong demo:

1. Client tạo Subject.
2. Client tạo ConcreteObserverA và ConcreteObserverB.
3. Client gọi Attach() để đăng ký lắng nghe.
4. Subject chạy SomeBusinessLogic(), cập nhật State.
5. Subject gọi Notify().
6. Từng observer nhận Update(subject) và tự quyết định có phản ứng hay không.
7. Client có thể gọi Detach() để hủy đăng ký một observer bất kỳ.

3.6.8. Các hệ quả mang lại, Ưu nhược điểm

3.6.9. Ưu điểm

- Giảm coupling giữa thành phần phát sự kiện và thành phần xử lý.
- Mở rộng tốt: thêm observer mới không cần sửa Subject.
- Hỗ trợ kiến trúc event-driven tự nhiên.
- Dễ tái sử dụng Subject trong nhiều ngữ cảnh khác nhau.
- Có thể đăng ký/hủy đăng ký linh hoạt trong runtime.

3.6.10. Nhược điểm

- Nếu số observer lớn, chi phí thông báo có thể tăng.

- Thứ tự gọi observer thường không đảm bảo nếu không quy định rõ.
- Dễ phát sinh vòng lặp sự kiện nếu observer tiếp tục kích hoạt thay đổi mới.
- Khó debug khi luồng sự kiện phức tạp và phân tán.

3.6.11. Chú ý liên quan đến việc cài đặt

- Nên quy định rõ chính sách thứ tự notify nếu nghiệp vụ phụ thuộc thứ tự.
- Cần xử lý ngoại lệ trong từng observer để tránh một observer lỗi làm gây toàn bộ chuỗi notify.
- Tránh memory leak bằng cách Detach() đúng lúc, đặc biệt với đối tượng sống lâu.
- Với hệ thống đa luồng, cần đồng bộ truy cập danh sách observer.
- Cân nhắc notify bất đồng bộ (queue/event bus) khi tải lớn.

3.6.12. Một số so sánh nếu có

3.6.13. Observer và Mediator

- Observer tập trung vào phát-thông báo một-nhiều theo subscription.
- Mediator tập trung điều phối tương tác nhiều-nhiều qua một trung gian.

3.6.14. Observer và Pub/Sub qua message broker

- Observer thường in-process, trực tiếp trong cùng ứng dụng.
- Pub/Sub broker (RabbitMQ, Kafka...) thường dùng liên tiến trình/liên dịch vụ, bất đồng bộ mạnh hơn.

3.6.15. Observer và Callback trực tiếp

- Callback trực tiếp phù hợp tình huống đơn giản, ít bên nghe.
- Observer phù hợp khi số bên nghe thay đổi linh hoạt và cần quản lý vòng đời đăng ký.

3.6.16. Mã nguồn minh họa

Đoạn mã dưới đây tóm lược theo đúng demo trong project:

```

public interface IObserver
{
    void Update(ISubject subject);
}

public interface ISubject
{
    void Attach(IObserver observer);
    void Detach(IObserver observer);
    void Notify();
}

public class Subject : ISubject
{
    public int State { get; set; } = 0;
    private List<IObserver> _observers = new List<IObserver>();

    public void Attach(IObserver observer) => _observers.Add(observer);
    public void Detach(IObserver observer) => _observers.Remove(observer);

    public void Notify()
    {
        foreach (var observer in _observers)

```

```

        {
            observer.Update(this);
        }
    }

    public void SomeBusinessLogic()
    {
        State = new Random().Next(0, 10);

        Notify();
    }
}

class ConcreteObserverA : IObserver
{
    public void Update(ISubject subject)
    {
        if ((subject as Subject).State < 3)
            Console.WriteLine("ObserverA reacted.");
    }
}

class ConcreteObserverB : IObserver
{

```



```

public void Update(ISubject subject)
{
    if ((subject as Subject).State == 0 || (subject as Subject).State >= 2)
        Console.WriteLine("ObserverB reacted.");
}
}

```

Kết quả chạy sẽ thay đổi theo giá trị ngẫu nhiên của State, nhưng luôn thể hiện:

- Subject cập nhật trạng thái.
- Subject thông báo toàn bộ observer đang đăng ký.
- Mỗi observer tự quyết định phản ứng theo điều kiện riêng.

3.6.17. Ví dụ về các hệ thống thực tế

- Cơ chế event trong WinForms/WPF/JavaFX.
- Data binding trong MVVM (property change notification).
- Listener trong Android (OnClickListener, TextWatcher).
- Event emitter trong Node.js.
- Hệ thống domain events trong kiến trúc DDD.

3.6.18. Các mẫu liên quan

- **Mediator**: dùng khi cần điều phối tương tác phức tạp giữa nhiều đối tượng.
- **Singleton**: Subject trung tâm đôi khi được triển khai dạng singleton trong ứng dụng nhỏ.
- **State**: khi state object thay đổi có thể phát sự kiện cho observer.
- **MVC/MVP/MVVM**: thường dùng Observer để đồng bộ Model và View.

3.7. State (Phức)

3.7.1. Tên và Phân loại

State - Mẫu hành vi (Behavioral Pattern)

3.7.2. Mục đích, ý định

Cho phép một đối tượng thay đổi hành vi của nó khi trạng thái nội bộ của nó thay đổi. Tách logic trạng thái khỏi lớp chính.

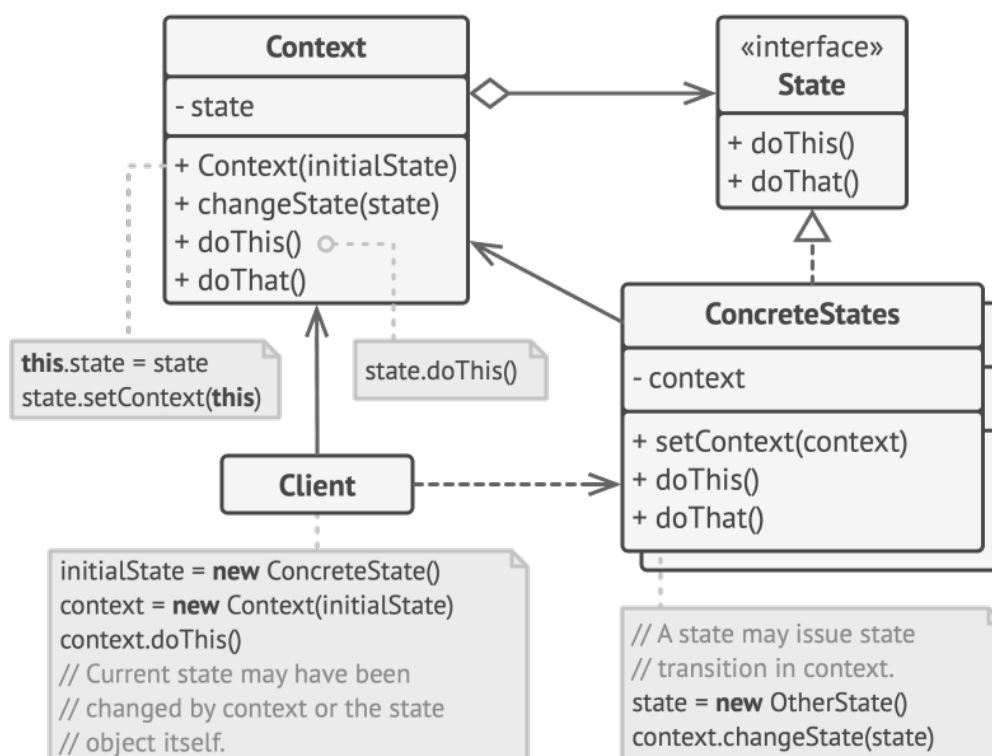
3.7.3. Motivation

Khi một đối tượng có hành vi phụ thuộc vào trạng thái và cần thay đổi hành vi khi trạng thái thay đổi, code sẽ chứa nhiều if-else. State pattern giải quyết bằng cách tạo các lớp trạng thái.

3.7.4. Khả năng ứng dụng

- Hành vi của đối tượng phụ thuộc vào trạng thái và thay đổi tại runtime
- Có các nhánh lớn phụ thuộc vào trạng thái hiện tại
- Mã có nhiều if-else hoặc switch-case kiểm tra trạng thái

3.7.5. Cấu trúc



Hình 3.35: Cấu trúc của mẫu State

State định nghĩa giao diện cho các hành vi trạng thái. ConcreteState triển khai hành vi cụ thể cho từng trạng thái. Context chứa một State hiện tại.

3.7.6. Các thành viên

- State: giao diện định nghĩa các phương thức cho mỗi trạng thái
- ConcreteState: triển khai hành vi cho một trạng thái cụ thể
- Context: định nghĩa giao diện cho client, giữ tham chiếu ConcreteState

3.7.7. Sự cộng tác

Context ủy nhiệm yêu cầu cho ConcreteState hiện tại. State có thể chuyển Context sang state khác.

3.7.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Loại bỏ if-else; dễ thêm trạng thái mới; tách logic trạng thái. **Nhược điểm:** Tăng số lớp; có thể phức tạp hóa code nếu trạng thái ít.

3.7.9. Chú ý liên quan đến việc cài đặt

- Xác định rõ tất cả trạng thái có thể
- Xem xét ai chuyển trạng thái: Context hay State

3.7.10. Mã nguồn minh họa

Máy trạng thái: Connection (Connected, Disconnected, Connecting).

3.7.11. Ví dụ về các hệ thống thực tế

- TCP Connection: Established, Listen, Closed
- Order: Pending, Processing, Shipped, Delivered
- Traffic light: Red, Yellow, Green
- Media player: Playing, Paused, Stopped

3.7.12. Các mẫu liên quan

Liên quan đến Strategy (chọn thuật toán). Khác ở chỗ State tự chuyển đổi.

3.8. Strategy (Đạt)

3.8.1. Tên và Phân loại

Strategy Pattern là một mẫu thiết kế thuộc nhóm **Behavioral Pattern**.

Trong tiếng Việt, mẫu này thường được gọi là **mẫu chiến lược**. Ý tưởng cốt lõi là đóng gói một họ thuật toán dưới các class riêng biệt, có cùng interface, để có thể hoán đổi linh hoạt trong runtime.

3.8.2. Mục đích, ý định

Mục đích chính của Strategy là tách phần “thuật toán có thể thay đổi” khỏi context sử dụng nó.

Ý định cụ thể:

- Đóng gói từng thuật toán thành strategy độc lập.
- Tránh chuỗi if/else hoặc switch lớn theo kiểu xử lý.
- Cho phép thay đổi hành vi lúc runtime mà không sửa context.
- Tăng khả năng mở rộng, kiểm thử và tái sử dụng thuật toán.

3.8.3. Motivation

Xét hệ thống thương mại điện tử cần tính phí vận chuyển theo nhiều loại:

- Standard Shipping.
- Express Shipping.
- Same-Day Shipping.

Nếu đặt toàn bộ logic vào một lớp ShippingService, code dễ gặp vấn đề:

- Nhiều điều kiện rẽ nhánh khó bảo trì.
- Mỗi lần thêm phương thức vận chuyển phải sửa mã cũ.
- Khó unit test từng thuật toán độc lập.

Strategy giải quyết bằng cách:

- Định nghĩa interface IShippingStrategy.
- Mỗi thuật toán tính phí là một ConcreteStrategy.
- CheckoutContext chỉ giữ reference strategy và gọi thực thi.

3.8.4. Khả năng ứng dụng

Strategy phù hợp khi:

- Có nhiều thuật toán cùng mục tiêu nhưng khác cách xử lý.
- Muốn chọn thuật toán theo cấu hình, user input, environment.
- Muốn tránh class lớn chứa quá nhiều nhánh điều kiện.
- Muốn benchmark hoặc A/B test nhiều thuật toán.

Ví dụ điển hình:

- Tính phí vận chuyển/thuế/chiết khấu.
- Thuật toán sắp xếp hoặc lọc dữ liệu.
- Chọn cơ chế nén (zip, gzip, lz4).
- Chọn cổng thanh toán (VNPay, MoMo, Stripe, COD).

3.8.5. Cấu trúc

Các vai trò chính của Strategy Pattern:

- Strategy: interface chung cho các thuật toán.
- ConcreteStrategy: cài đặt thuật toán cụ thể.
- Context: giữ strategy hiện hành và dùng nó để xử lý.
- Client: chọn strategy phù hợp và inject vào context.

UML tổng quát:

```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
interface IStrategy {
```

```
    +Execute(data)
```

```
}
```

```
class ConcreteStrategyA {
    +Execute(data)
}
```

```
class ConcreteStrategyB {
    +Execute(data)
}
```

```
class Context {
    -strategy: IStrategy
    +SetStrategy(strategy: IStrategy)
    +Run(data)
}
```

```
class Client
```

```
IStrategy <|.. ConcreteStrategyA
```

```
IStrategy <|.. ConcreteStrategyB
```

```
Context --> IStrategy
```

```
Client --> Context
```

```
Client --> IStrategy
```

```
@enduml
```

Class diagram (Mermaid):

classDiagram

```
class IShippingStrategy {
    <<interface>>
    +CalculateFee(orderTotal, weight, distance): decimal
}

class StandardShipping
class ExpressShipping
class SameDayShipping

class CheckoutContext {
    -strategy: IShippingStrategy
    +SetStrategy(strategy)
    +CalculateShipping(orderTotal, weight, distance): decimal
}

IShippingStrategy <|.. StandardShipping
IShippingStrategy <|.. ExpressShipping
IShippingStrategy <|.. SameDayShipping
CheckoutContext --> IShippingStrategy
```

3.8.6. Các thành viên

Strategy (IShippingStrategy)

- Khai báo contract cho thuật toán.
- Giúp context làm việc qua abstraction.

ConcreteStrategy

- Cài đặt từng thuật toán cụ thể.
- Ví dụ:
 - StandardShipping: phí thấp, tốc độ thường.
 - ExpressShipping: phí cao hơn, giao nhanh.
 - SameDayShipping: phí cao nhất, có thêm điều kiện phạm vi.

Context (CheckoutContext)

- Nắm strategy hiện tại.
- Cung cấp API nghiệp vụ cho client.
- Ủy quyền xử lý thuật toán cho strategy thay vì tự làm.

Client

- Quyết định chọn strategy nào.
- Có thể thay strategy runtime theo điều kiện đơn hàng.

3.8.7. Sự cộng tác

Luồng cộng tác chuẩn:

1. Client tạo context.
2. Client chọn strategy phù hợp.
3. Client set strategy vào context (SetStrategy).
4. Client gọi API trên context (CalculateShipping).
5. Context gọi strategy hiện tại để xử lý.
6. Client có thể thay strategy và chạy lại mà không sửa context.

Sequence diagram:

```
@startuml
```

```
actor User
```

```
participant Client
```

```
participant Context
```

```
participant Strategy
```

```
User -> Client : choose shipping type
```

```
Client -> Context : SetStrategy(ExpressShipping)
```

```
Client -> Context : CalculateShipping(...)
```

```
Context -> Strategy : CalculateFee(...)
```

```
Strategy --> Context : fee
```

```
Context --> Client : fee
```

```
@enduml
```

3.8.8. Các hệ quả mang lại, Ưu nhược điểm

3.8.9. Ưu điểm

- **Open/Closed**: thêm strategy mới mà ít sửa mã cũ.
- **Single Responsibility**: mỗi thuật toán ở class riêng.
- Dễ unit test từng strategy độc lập.
- Dễ thay thế thuật toán theo runtime hoặc config.
- Giảm độ phức tạp ở context.

3.8.10. Nhược điểm

- Tăng số lượng class.
- Client cần hiểu để chọn strategy phù hợp.

- Nếu interface strategy quá chung chung, có thể phát sinh nhiều tham số không cần thiết.

3.8.11. Chú ý liên quan đến việc cài đặt

- Giữ interface strategy đủ nhỏ và rõ nghĩa.
- Tránh để context biết chi tiết nội bộ của từng strategy.
- Nếu chọn strategy theo nhiều điều kiện, nên dùng factory/resolver để tránh if/else dồn về client.
- Với chiến lược có state nội bộ, cần chú ý thread safety.
- Có thể dùng DI để đăng ký danh sách strategy và resolve theo key.

3.8.12. Một số so sánh nếu có

3.8.13. Strategy và State

- Strategy: client/chính sách bên ngoài chọn thuật toán.
- State: context tự chuyển đổi hành vi theo trạng thái nội bộ.

3.8.14. Strategy và Template Method

- Strategy dùng composition để thay đổi toàn bộ thuật toán.
- Template Method dùng kế thừa để thay đổi một số bước trong khung thuật toán cố định.

3.8.15. Strategy và Command

- Strategy đóng gói **cách làm** (thuật toán).
- Command đóng gói **việc cần làm** (request) để thực thi, queue, undo.

3.8.16. Strategy và Policy-based configuration

- Strategy thường là hiện thực OOP của policy.
- Policy config có thể map 1-1 sang concrete strategy.

3.8.17. Mã nguồn minh họa

Ví dụ C# tính phí vận chuyển theo strategy:

```
using System;
```

```

public interface IShippingStrategy
{
    decimal CalculateFee(decimal orderTotal, decimal weight, decimal
distanceKm);
}

public class StandardShipping : IShippingStrategy
{
    public decimal CalculateFee(decimal orderTotal, decimal weight, decimal
distanceKm)
    => 15000m + weight * 2000m + distanceKm * 500m;
}

public class ExpressShipping : IShippingStrategy
{
    public decimal CalculateFee(decimal orderTotal, decimal weight, decimal
distanceKm)
    => 30000m + weight * 3000m + distanceKm * 900m;
}

public class SameDayShipping : IShippingStrategy
{
    public decimal CalculateFee(decimal orderTotal, decimal weight, decimal

```

```

distanceKm)

{
    if (distanceKm > 20) throw new InvalidOperationException("Same-day only
supports <= 20km");

    return 50000m + weight * 3500m + distanceKm * 1200m;
}
}

```

```

public class CheckoutContext
{
    private IShippingStrategy _strategy;

    public CheckoutContext(IShippingStrategy strategy)
    {
        _strategy = strategy;
    }

    public void SetStrategy(IShippingStrategy strategy) => _strategy =
strategy;

    public decimal CalculateShipping(decimal orderTotal, decimal weight,
decimal distanceKm)
    => _strategy.CalculateFee(orderTotal, weight, distanceKm);
}

```

```
}
```

```
public class Program
```

```
{
```

```
    public static void Main()
```

```
    {
```

```
        var ctx = new CheckoutContext(new StandardShipping());
```

```
        Console.WriteLine($"Standard fee: {ctx.CalculateShipping(500000m, 2.5m, 8m)}");
```

```
        ctx.SetStrategy(new ExpressShipping());
```

```
        Console.WriteLine($"Express fee: {ctx.CalculateShipping(500000m, 2.5m, 8m)}");
```

```
    }
```

```
}
```

Ý nghĩa ví dụ:

- Context giữ contract `IShippingStrategy`, không phụ thuộc class cụ thể.
- Thuật toán đổi bằng `SetStrategy()`.
- Business flow không cần đổi khi thêm strategy mới.

3.8.18. Ví dụ về các hệ thống thực tế

- Cổng thanh toán đa nhà cung cấp.
- Hệ thống đề xuất (recommendation) thay mô hình theo user segment.
- Engine định tuyến chọn thuật toán shortest path theo loại dữ liệu bản đồ.

- Công cụ nén/chuyển mã media theo quality profile.
- Rule pricing động trong ride-hailing theo khu vực/khung giờ.

3.8.19. Các mẫu liên quan

- **State**: gần nhất về cấu trúc, khác mục tiêu chọn hành vi.
- **Template Method**: thay đổi theo kế thừa thay vì composition.
- **Factory Method/Abstract Factory**: thường dùng để tạo strategy phù hợp.
- **Command**: có thể dùng strategy bên trong command để đổi cách thực thi.

Tóm lại, Strategy Pattern đặc biệt hiệu quả khi hệ thống có nhiều thuật toán thay thế cho cùng một nghiệp vụ và cần thay đổi linh hoạt theo runtime. Khi kết hợp tốt với DI/factory, mẫu này giúp code sạch hơn, mở rộng dễ hơn và kiểm thử thuận tiện hơn.

3.9. Template Method (Phát)

3.9.1. Tên và phân loại

Template Method Pattern là một mẫu thiết kế thuộc nhóm **Behavioral Pattern**. Nhóm Behavioral tập trung vào cách các object hoặc class phối hợp hành vi với nhau.

Template Method giải quyết bài toán khi nhiều class có chung một quy trình tổng quát, nhưng một số bước trong quy trình đó lại khác nhau tùy từng class con.

Nói ngắn gọn, Template Method giúp cố định “khung thuật toán” ở class cha, còn các bước chi tiết có thể được class con tùy biến.

3.9.2. Vấn đề cần giải quyết

Trong nhiều hệ thống, ta thường gặp nhiều class thực hiện cùng một workflow tổng quát. Nếu mỗi class tự viết lại toàn bộ workflow, code sẽ bị trùng lặp và dễ sai lệch thứ tự xử lý.

Ví dụ hệ thống xử lý thanh toán. Dù thanh toán bằng thẻ, ví điện tử hay tiền mặt, quy trình tổng quát có thể giống nhau:

1. Validate đơn hàng.

2. Tính tổng tiền.
3. Xử lý thanh toán.
4. Cập nhật trạng thái đơn hàng.
5. Gửi thông báo.

Điểm khác nhau thường chỉ nằm ở bước xử lý thanh toán:

- Thẻ tín dụng cần gọi cổng thanh toán ngân hàng.
- Ví điện tử cần gọi API của ví.
- Tiền mặt có thể không gọi API thanh toán mà chỉ ghi nhận trạng thái chờ thu tiền.

Nếu không dùng Template Method, ta có thể viết lặp như sau:

```
public class CreditCardPayment
{
    public void Pay(Order order)
    {
        ValidateOrder(order);

        var amount = CalculateAmount(order);

        ProcessCreditCard(amount);

        UpdateOrderStatus(order);

        SendNotification(order);
    }
}
```

```
public class MomoPayment
{
```

```

public void Pay(Order order)
{
    ValidateOrder(order);

    var amount = CalculateAmount(order);

    ProcessMomo(amount);

    UpdateOrderStatus(order);

    SendNotification(order);
}
}

```

Các bước `ValidateOrder`, `CalculateAmount`, `UpdateOrderStatus`, `SendNotification` bị lặp lại. Nếu sau này muốn thêm một bước mới như ghi audit log, ta phải sửa nhiều class.

Vấn đề đặt ra là:

Làm sao tái sử dụng được phần quy trình chung, đảm bảo thứ tự thuật toán không bị thay đổi lung tung, nhưng vẫn cho phép class con tùy biến một số bước cụ thể?

3.9.3. Định nghĩa

Template Method Pattern định nghĩa bộ khung của một thuật toán trong class cha, nhưng cho phép class con override một số bước cụ thể mà không làm thay đổi cấu trúc tổng thể của thuật toán.

Trong pattern này:

- Class cha chứa một method chính gọi là **template method**.
- Template method định nghĩa thứ tự các bước của thuật toán.
- Một số bước được cài sẵn trong class cha.
- Một số bước là abstract/virtual để class con triển khai hoặc tùy biến.

- Client thường gọi template method mà không cần biết chi tiết từng bước bên trong.

Nói đơn giản:

Template Method cố định quy trình chung, nhưng mở ra các điểm tùy biến bên trong quy trình đó.

3.9.4. Ý tưởng cốt lõi

Ý tưởng chính của Template Method là:

Đưa những phần giống nhau lên class cha, còn những phần thay đổi thì để class con override.

Class cha đóng vai trò người “điều phối quy trình”. Nó biết bước nào chạy trước, bước nào chạy sau. Class con chỉ chịu trách nhiệm cài đặt các phần khác nhau.

Ví dụ:

```
public abstract class PaymentProcessor
{
    public void Pay(Order order)
    {
        ValidateOrder(order);

        var amount = CalculateAmount(order);

        ProcessPayment(amount);

        UpdateOrderStatus(order);

        SendNotification(order);
    }
}
```

```

        protected abstract void ProcessPayment(decimal amount);
    }

```

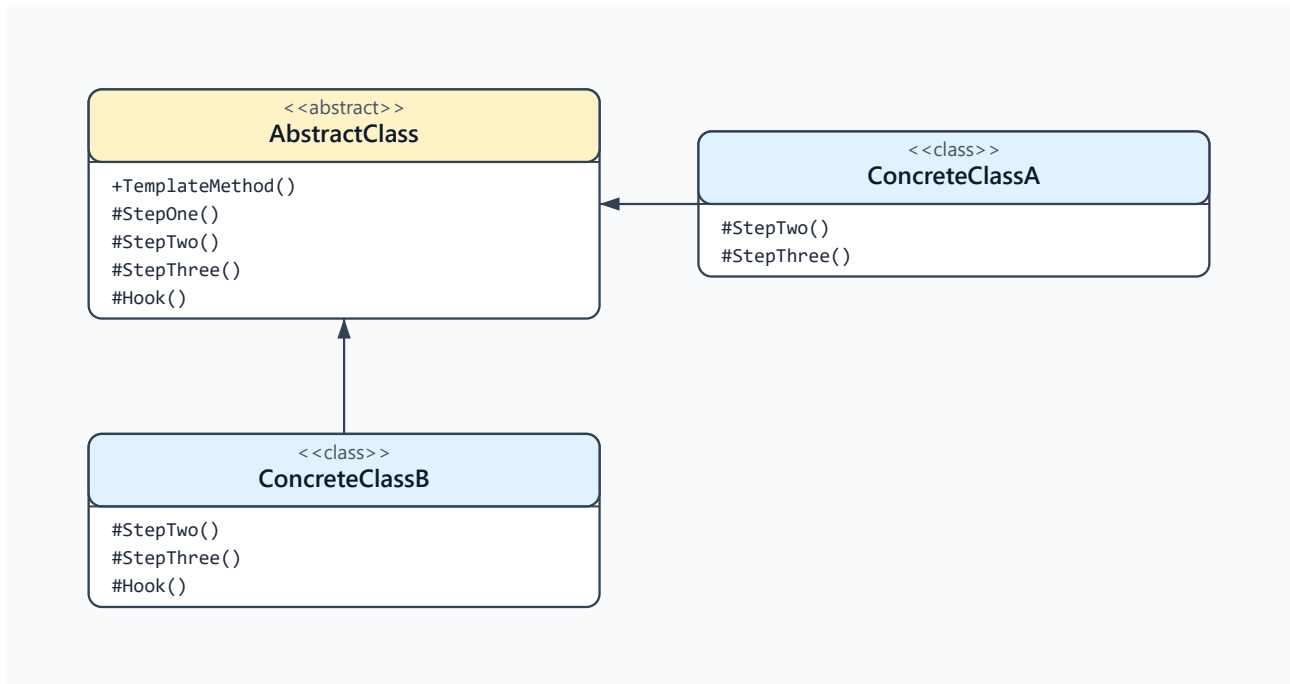
Ở đây, `Pay()` là template method. Nó cố định workflow thanh toán. Các class con chỉ cần override `ProcessPayment()`.

3.9.5. Thành phần chính

Thành phần	Vai trò
AbstractClass	Class cha định nghĩa template method và các bước của thuật toán.
Template Method	Method cố định thứ tự thực hiện các bước. Thường không nên override.
Primitive Operations	Các bước cụ thể mà class con bắt buộc phải triển khai.
ConcreteClass	Class con triển khai các bước còn thiếu hoặc tùy biến bước đã có.
Hook Methods	Các điểm mở rộng tùy chọn; class con có thể override hoặc bỏ qua.

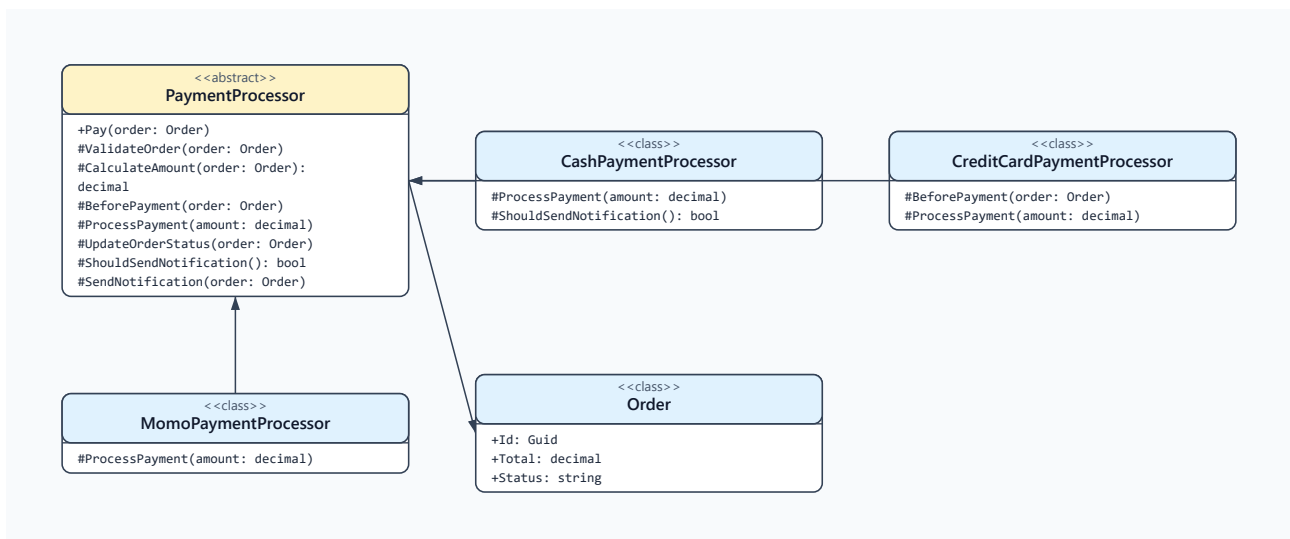
3.9.6. UML bằng PlantUML

3.9.7. UML tổng quát



Hình 3.36: Cấu trúc UML tổng quát của Template Method

3.9.8. UML ví dụ PaymentProcessor



Hình 3.37: Class diagram của ví dụ PaymentProcessor

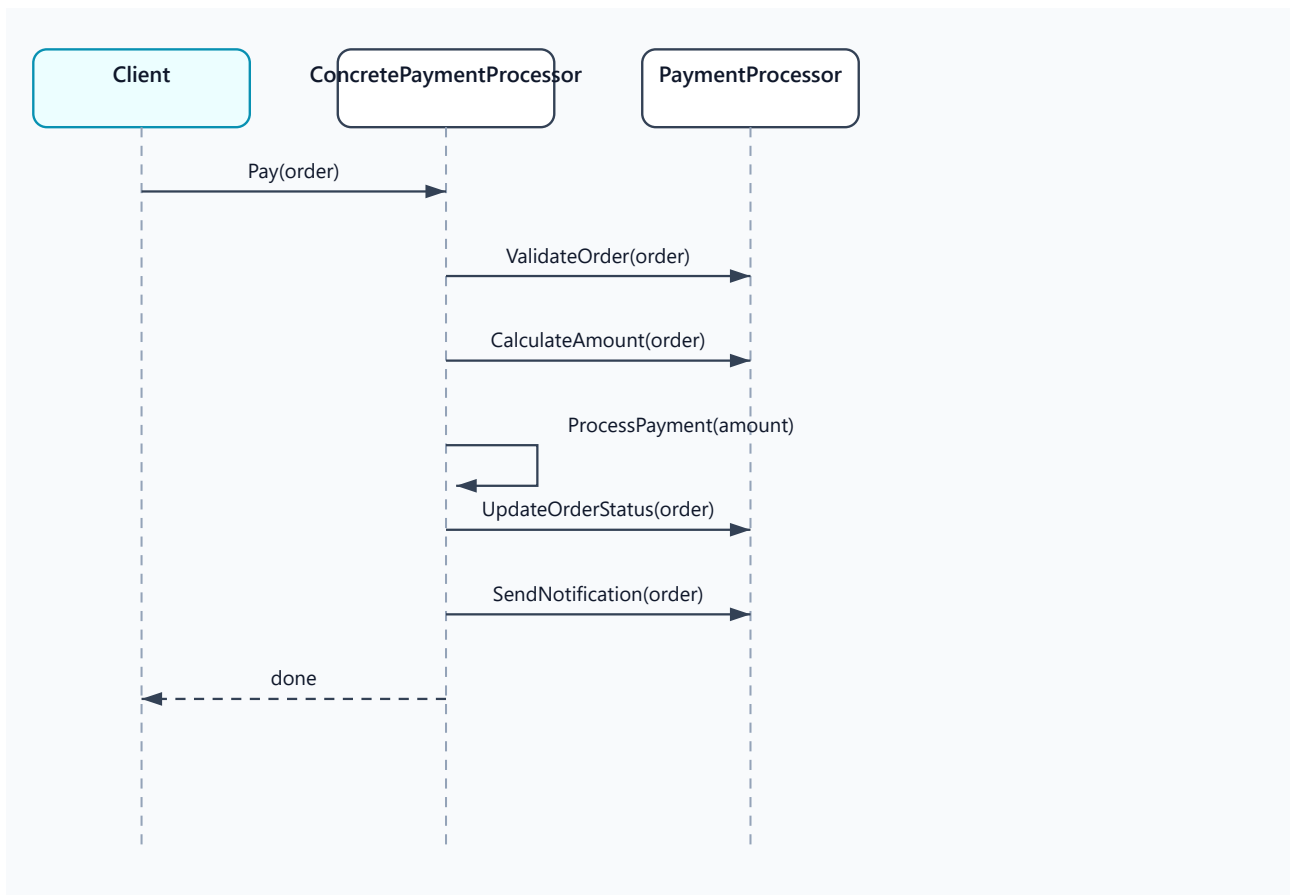
3.9.9. Luồng hoạt động

Luồng hoạt động của Template Method:

1. Client tạo object của class con.

2. Client gọi template method trên object đó.
3. Template method ở class cha bắt đầu chạy.
4. Class cha thực hiện các bước chung.
5. Khi đến bước cần tùy biến, class cha gọi method đã được override ở class con.
6. Sau đó template method tiếp tục chạy các bước còn lại.
7. Client nhận kết quả mà không cần biết chi tiết từng bước.

Sequence diagram:



Hình 3.38: Luồng chạy template method trong PaymentProcessor

3.9.10. Code minh họa C#

3.9.11. Domain đơn giản

```
public class Order
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public decimal Total { get; set; }
    public string Status { get; set; } = "Pending";
    public string CustomerEmail { get; set; } = string.Empty;
}
```

3.9.12. Abstract class chứa template method

```
public abstract class PaymentProcessor
{
    // Template Method: cô'định workflow thanh toán.
    public void Pay(Order order)
    {
        ValidateOrder(order);

        decimal amount = CalculateAmount(order);

        BeforePayment(order);           // Hook method
        ProcessPayment(amount);         // Primitive operation

        UpdateOrderStatus(order);

        if (ShouldSendNotification()) // Hook method
    }
```

```

    {
        SendNotification(order);
    }
}

```

```
protected virtual void ValidateOrder(Order order)
```

```

{
    if (order == null)
        throw new ArgumentNullException(nameof(order));

```

```

    if (order.Total <= 0)

```

```

        throw new InvalidOperationException("Order total must be greater
than zero.");
    }

```

```
protected virtual decimal CalculateAmount(Order order)
```

```

{
    return order.Total;
}

```

```
protected virtual void BeforePayment(Order order)
```

```

{
    // Hook method: mặc định không làm gì.

```

```

        // Class con có thể override nếu cần.
    }

    protected abstract void ProcessPayment(decimal amount);

    protected virtual void UpdateOrderStatus(Order order)
    {
        order.Status = "Paid";

        Console.WriteLine($"Order {order.Id} status updated to Paid.");
    }

    protected virtual bool ShouldSendNotification()
    {
        return true;
    }

    protected virtual void SendNotification(Order order)
    {
        Console.WriteLine($"Send payment confirmation to
{order.CustomerEmail}.");
    }
}

```

3.9.13. Concrete class cho từng loại thanh toán

```
public class CreditCardPaymentProcessor : PaymentProcessor
{
    protected override void BeforePayment(Order order)
    {
        Console.WriteLine("Checking credit card fraud risk...");
    }

    protected override void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Charging credit card: {amount:N0} VND");
    }
}

public class MomoPaymentProcessor : PaymentProcessor
{
    protected override void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Calling MoMo API: {amount:N0} VND");
    }
}

public class CashPaymentProcessor : PaymentProcessor
```



```

{
    protected override void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Mark as cash on delivery: {amount:N0} VND");
    }

    protected override bool ShouldSendNotification()
    {
        return false;
    }
}

```

3.9.14. Client sử dụng

```

var order = new Order
{
    Total = 150_000,
    CustomerEmail = "customer@example.com"
};

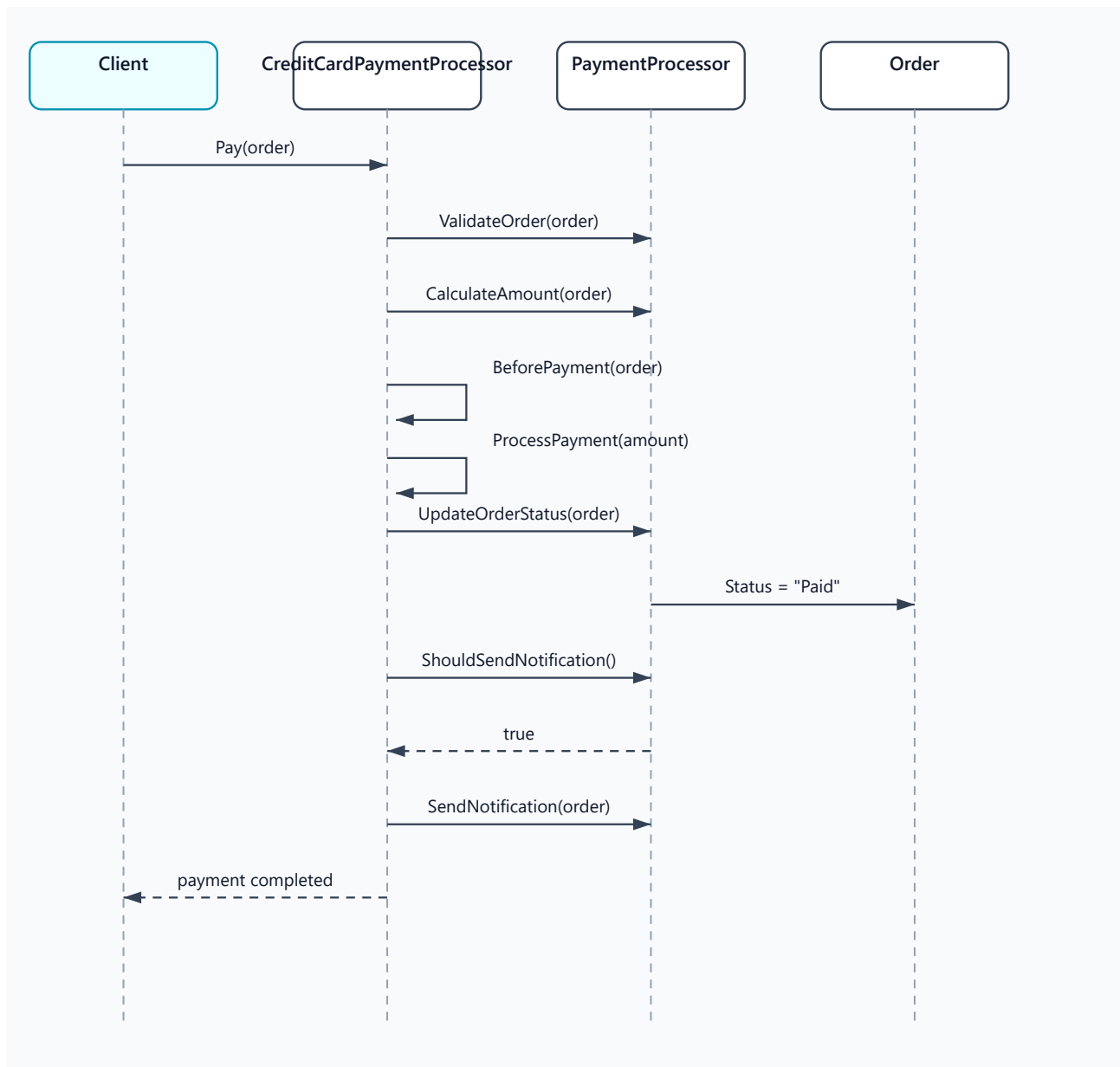
```

```

PaymentProcessor processor = new MomoPaymentProcessor();
processor.Pay(order);

```

Client chỉ gọi `Pay(order)`. Nó không cần quan tâm bên trong có những bước nào và thứ tự ra sao. Class cha đảm bảo workflow chung, class con chỉ thay đổi phần thanh toán cụ thể.



Hình 3.39: Luồng chạy code ví dụ `PaymentProcessor.Pay`

3.9.15. Đánh giá

3.9.16. Ưu điểm

Ưu điểm	Giải thích
Tái sử dụng code chung	Các bước giống nhau được đưa lên class cha, tránh lặp code ở nhiều class con.

Đảm bảo thứ tự thuật toán	Template method cố định workflow, class con khó làm sai thứ tự xử lý.
Cho phép tùy biến một phần thuật toán	Class con chỉ override các bước cần thay đổi.
Tuân thủ Open/Closed ở mức nhất định	Có thể thêm class con mới mà ít phải sửa workflow chung.
Tách phần ổn định và phần thay đổi	Phần ổn định nằm ở class cha, phần biến đổi nằm ở primitive operation/hook.

3.9.17. Nhược điểm

Nhược điểm	Giải thích
Phụ thuộc vào kế thừa	Class con bị ràng buộc vào class cha, kém linh hoạt hơn composition.
Class cha có thể khó hiểu	Nếu workflow lớn và có nhiều hook, abstract class dễ trở nên phức tạp.
Class con bị giới hạn bởi khung thuật toán	Nếu class con cần workflow rất khác, Template Method không còn phù hợp.
Có thể vi phạm LSP	Nếu class con override sai ý đồ, behavior có thể không còn đúng với kỳ vọng của class cha.
Khó đổi thuật toán lúc runtime	Vì variation nằm trong class con, không linh hoạt bằng Strategy.

3.9.18. Khi nào nên dùng và không nên dùng

3.9.19. Nên dùng khi

- Nhiều class có chung một quy trình tổng quát.
- Một số bước trong quy trình thay đổi tùy class con.

- Muốn tránh lặp code ở nhiều class.
- Muốn đảm bảo thứ tự xử lý luôn nhất quán.
- Workflow tương đối ổn định, chỉ có một vài điểm mở rộng.

Ví dụ phù hợp:

- Import file: open file → parse → validate → save → close.
- Xử lý thanh toán: validate → calculate → pay → update → notify.
- Data mining: load data → clean → transform → analyze → export.
- Report generation: fetch data → format → render → export.
- Game AI turn: select target → move → attack → end turn.

3.9.20. Không nên dùng khi

- Các thuật toán thay đổi nhiều và cần đổi lúc runtime.
- Không muốn phụ thuộc vào kế thừa.
- Mỗi class có workflow rất khác nhau.
- Class cha phải chứa quá nhiều hook để đáp ứng mọi trường hợp.
- Có thể dùng composition linh hoạt hơn, ví dụ Strategy hoặc pipeline.

3.9.21. So sánh với các pattern liên quan

3.9.22. Template Method vs Strategy

Tiêu chí	Template Method	Strategy
Nhóm pattern	Behavioral.	Behavioral.
Cơ chế chính	Kế thừa.	Composition.
Phần thay đổi nằm ở	Class con override method.	Object strategy được truyền vào.
Workflow tổng thể	Thường cố định trong class cha.	Context có thể thay đổi strategy linh hoạt.

Đổi thuật toán runtime	Khó hơn.	Dễ hơn.
Coupling	Class con phụ thuộc class cha.	Context phụ thuộc abstraction strategy.
Ví dụ	PaymentProcessor.Pay() cố định quy trình.	IPaymentStrategy.Pay() thay đổi cách thanh toán.

3.9.23. Template Method vs Factory Method

Tiêu chí	Template Method	Factory Method
Nhóm pattern	Behavioral.	Creational.
Mục đích	Định nghĩa khung thuật toán.	Tạo object thông qua method cho class con quyết định.
Class con override để	Tùy biến một bước xử lý.	Quyết định object nào được tạo.
Trọng tâm	Luồng xử lý.	Khởi tạo object.
Ví dụ	Import() gồm open, parse, save, close.	CreateButton() trả về WindowsButton/MacButton.

3.9.24. Template Method vs Hook Method

Tiêu chí	Template Method	Hook Method
Vai trò	Method chính định nghĩa workflow.	Điểm mở rộng tùy chọn trong workflow.
Có bắt buộc override không?	Không nên override template method nếu muốn giữ workflow.	Không bắt buộc override.
Thường nằm ở đâu?	Class cha.	Class cha.

Ví dụ	Pay()	BeforePayment(), ShouldSendNotification()
-------	-------	--

3.9.25. Template Method vs Chain of Responsibility

Tiêu chí	Template Method	Chain of Responsibility
Mục đích	Cố định các bước của một thuật toán.	Truyền request qua chuỗi handler.
Thứ tự xử lý	Được class cha định nghĩa cố định.	Do cách nối chain quyết định.
Ai xử lý?	Nhiều method trong cùng object/class hierarchy.	Nhiều object handler khác nhau.
Có thể dừng giữa chừng không?	Có thể, nhưng không phải trọng tâm.	Có, đây là đặc điểm phổ biến.
Ví dụ	Quy trình import file.	Middleware xử lý request.

3.9.26. Template Method vs Decorator

Tiêu chí	Template Method	Decorator
Nhóm pattern	Behavioral.	Structural.
Mục đích	Cố định workflow và cho class con tùy biến bước.	Bọc object để thêm hành vi mới.
Cơ chế	Kế thừa.	Composition/wrapping.
Thời điểm mở rộng	Thông qua subclass.	Thông qua wrapper object.
Ví dụ	Các class thanh toán override ProcessPayment().	LoggingPaymentGateway bọc payment gateway để thêm logging.

3.9.27. Kết luận

Template Method là pattern phù hợp khi nhiều class có cùng một quy trình xử lý tổng quát nhưng khác nhau ở một vài bước. Nó giúp tái sử dụng code chung, tránh lặp workflow và đảm bảo thứ tự thuật toán nhất quán.

Tuy nhiên, pattern này dựa nhiều vào kế thừa. Nếu hệ thống cần thay đổi thuật toán linh hoạt lúc runtime hoặc muốn giảm phụ thuộc vào class cha, Strategy hoặc các thiết kế dùng composition có thể phù hợp hơn.

3.10. Visitor (Phức)

3.10.1. Tên và Phân loại

Visitor - Mẫu hành vi (Behavioral Pattern)

3.10.2. Mục đích, ý định

Cho phép định nghĩa các thao tác mới trên các phần tử của một cấu trúc đối tượng mà không thay đổi các lớp của những phần tử đó. Tách logic xử lý khỏi cấu trúc dữ liệu.

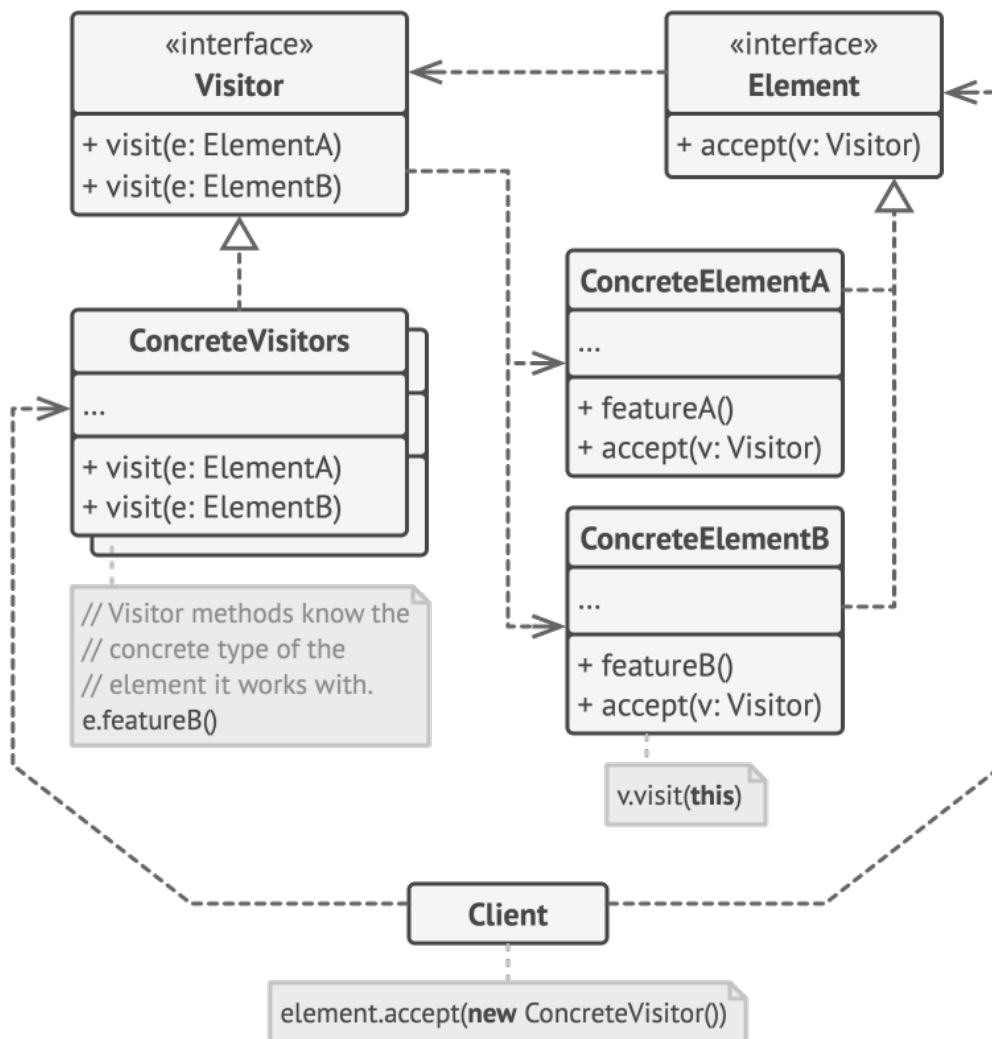
3.10.3. Motivation

Khi cần thực hiện nhiều thao tác khác nhau trên cùng một cấu trúc đối tượng phức tạp, thêm mỗi thao tác mới vào từng lớp sẽ làm cho code trở nên cồng kềnh. Visitor cho phép tách các thao tác ra khỏi cấu trúc.

3.10.4. Khả năng ứng dụng

- Cấu trúc đối tượng ít thay đổi nhưng thao tác hay thay đổi
- Cần thực hiện các phép toán khác nhau trên các phần tử không liên quan
- Cần thực hiện phép tính lặp lại hoặc báo cáo trên cấu trúc phức tạp

3.10.5. Cấu trúc



Hình 3.40: Cấu trúc của mẫu Visitor

Visitor khai báo một giao diện `visit()` cho mỗi loại phần tử. Các phần tử chấp nhận visitor và gọi phương thức `visit` tương ứng. **ConcreteVisitor** triển khai logic cho từng loại phần tử.

3.10.6. Các thành viên

- **Visitor**: giao diện định nghĩa phương thức `visit` cho mỗi loại phần tử
- **ConcreteVisitor**: triển khai các phương thức `visit` cụ thể
- **Element**: giao diện chấp nhận visitor
- **ConcreteElement**: phần tử cụ thể, triển khai `accept()`

- ObjectStructure: quản lý tập hợp các phần tử

3.10.7. Sự cộng tác

Phần tử gọi accept(visitor), visitor gọi lại phương thức visit tương ứng của phần tử đó.

3.10.8. Các hệ quả mang lại, Ưu nhược điểm

Ưu điểm: Dễ dàng thêm thao tác mới; nhóm logic thao tác tại một chỗ. **Nhược điểm:** Khó thêm phần tử mới; vi phạm cấu trúc lớp.

3.10.9. Chú ý liên quan đến việc cài đặt

- Hãy xác định rõ tập hợp các phần tử sẽ được visitor xử lý
- Visitor nên được thiết kế để có thể xử lý các phần tử tương ứng

3.10.10. Mã nguồn minh họa

Visitor được sử dụng trong trình biên dịch, parser, hoặc khi cần in chi tiết cấu trúc dữ liệu phức tạp.

3.10.11. Ví dụ về các hệ thống thực tế

- Trình biên dịch: AST visitor cho mỗi loại nút
- IDE: refactoring tool sử dụng visitor pattern
- Document processors: tính toán thống kê tài liệu

3.10.12. Các mẫu liên quan

Khác với Composite (quản lý cấu trúc) về mục đích, nhưng thường dùng cùng nhau. Liên quan đến Strategy.